Genetic Diversity in Hybrid Breeding

Measuring it, constraining it, and optimising against it, with R

Eder D. B. S.

Contents

Preface

A hybrid breeding programme has two objectives that pull against each other. The first is short-term: pick the crosses with the highest predicted performance. The second is long-term: do not spend the germplasm that will be needed to make the next round of gain possible. Every programme knows this. Very few can say, in a number, how much of the second they traded away to buy the first.

This book is about producing that number, defending it, and putting it inside an optimiser.

The problem, stated once

Given a pool of N candidate F1 hybrids – each a cross between one line from heterotic group A and one from group B, each with a predicted multi-trait index – choose n_{sel} of them so that the mean index is as high as possible, subject to losing no more than a fraction α of the molecular gene diversity that a *random* set of the same size would have carried.

Three of those words carry the entire argument.

Molecular. Diversity is measured on the Caballero & Toro molecular coancestry matrix f, never on a VanRaden genomic relationship matrix G. This is not a stylistic preference. G is centered on its own population by construction, so the sum of all its elements is exactly zero; every diversity quantity computed from it is meaningless. Chapter 1. shows this happening.

Random. The reference for "no loss" is the distribution of random subsets of size n_{sel}, not the full candidate pool. Comparing a selected set of 100 against a pool of 2500 confounds a selection effect with a sampling effect, and the sampling effect alone can look like a loss of half a percent. Section 7.9 separates them.

Subject to. α is a *constraint*, not a term in a weighted sum. When you write $\alpha_{max} = 0.05$ you are saying "I accept losing five percent", which is a statement a breeding committee can approve. A weighted sum says "diversity is worth λ index units", which is a statement nobody can approve because nobody knows what λ means. Chapter 10. makes the distinction precise, and shows the standard optimal-contribution formulation it specialises.

What is unusual here

Most treatments of diversity management come from animal breeding, where the population is randomly mating and the unit of selection is an individual. Plant hybrid breeding breaks both assumptions:

- The parents are **inbred lines**, so their within-individual diversity is zero by construction, and the interesting heterozygosity lives in the F1 product, not in the parents.
- The population is **permanently subdivided** into heterotic groups, and the divergence between them is not a problem to be minimised – it is the engine of heterosis, to be *maintained*. A single pool-wide statistic cannot express "conserve within, hold between".
- The candidates are **crosses of fixed lines**, so the progeny variance term that drives cross-selection criteria elsewhere collapses to zero, and the hybrid-by-hybrid coancestry matrix turns out never to be needed at all (Section 8.1).

Each of these changes what should be measured. None of them changes the underlying theory, which is why the book starts with the theory.

How to read this

The book is meant to be run, not only read. Every derivation that could be checked numerically is checked numerically, in a code block you can execute. Where a result is an exact algebraic identity, the deviation from a simulated oracle is printed next to it; where it is a Monte Carlo agreement, the residual is printed.

The parts are ordered so that each depends only on what came before:

Part	What it establishes
Foundations	The coancestry kernel, and the partition of diversity in a subdivided population
Simulation	How the candidate pool is generated, and how large a segregating population has to be
Metrics	What can be measured, what it costs, and which measurements are redundant
Optimisation	How the constraint enters an optimiser, and which optimiser to use
Evidence and practice	What the literature does and does not establish, and a worked end-to-end run

Readers who only want the operational answer can read Chapter 8. (which metrics), Chapter 9. (at which stage) and Chapter 13. (the whole pipeline in one script), then come back for the derivations.

Running the code

All code is in the `hybdiv` package that ships with this book.

```
# from the repository root
install.packages(c("DEoptim", "quadprog"))
devtools::install(".")

library(hybdiv)
st1 <- stage1_simulate()                                    # lines -> X, f, hybrids
res <- stage2_select(st1$X, st1$f, st1$hybrids, n_sel = 100, fL = st1$fL)
res$metrics
```

Every live example in the book runs at a deliberately small scale – 25 lines per pool and 2000 markers, giving 625 candidate hybrids – so that a full render takes minutes. The scale is set once, in `_common.R`:

```
str(book_cfg)
#> List of 6
#>  $ n_pool_A: num 25
#>  $ n_pool_B: num 25
#>  $ m       : num 2000
#>  $ n_traits: num 3
#>  $ fst     : num 0.15
#>  $ seed    : num 1
```

Results that are expensive at any scale – the differential-evolution sweeps, the cost benchmark, the population-sizing grid – are precomputed and shipped in `book/data` and `book/figs`. `book/scripts/regenerate.R` regenerates all of them, and Appendix C. says exactly which figures and tables come from where.

A note on numbers: everything reported here is reproducible output from a simulated dataset, not a fixed constant. Change the simulation configuration or a random seed and the numbers move. The *relationships* between them – which metric dominates which, which identity holds exactly – do not.

I Foundations

1. Coancestry, and the two matrices

Two matrices are built from the same marker data and they are not interchangeable. One predicts traits. The other measures diversity. Using the first for the second is the most consequential mistake available in this subject, and it fails silently – it returns a number, and the number is meaningless.

This chapter builds both, shows the failure, and derives the matrix used for the rest of the book.

1.1 From line to hybrid

Let $\mathbf{M}^L$ be the $L \times m$ matrix of line genotypes, with $M^L_{ik} \in \{0, 2\}$ the count of the reference allele at marker k. Inbred lines are homozygous, so the value 1 never occurs – residual heterozygotes must be imputed or discarded beforehand.

The F1 hybrid between lines a and b receives one gamete from each parent. Since both parents are homozygous, the gamete from a carries $M^L_{ak}/2$ copies of the reference allele, so

$$M^H_{(ab)k} = \frac{M^L_{ak} + M^L_{bk}}{2} \in \{0, 1, 2\}. \tag{1.1}$$

The value 1 appears exactly at markers where the two lines differ. Those are the hybrid's heterozygous loci, and they are the ones carrying heterosis.

Predicting the hybrid's *genotype* is deterministic, not statistical. It is the one piece of information in the whole pipeline that requires no assumption at all – unlike its phenotype, which requires a great many.

From here on we work with the **within-individual reference-allele frequency**,

$$x_{ik} = \frac{M^H_{ik}}{2} \in \{0, 0.5, 1\}, \tag{1.2}$$

which is the natural quantity for coancestry: it is the probability that an allele drawn at random from individual i at marker k is the reference allele.

```
ctx <- simulate_data(book_cfg)
ctx$X[1:4, 1:8]
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
#> [1,]  0.5  0.5  0.5    1    0    1  0.5  0.5
#> [2,]  0.5  0.0  0.0    1    0    1  1.0  0.0
#> [3,]  0.0  0.0  0.5    1    0    1  1.0  0.5
#> [4,]  0.0  0.0  0.0    1    0    1  1.0  0.5
```

Rows are hybrids, columns markers. A 0.5 is a heterozygous locus.

1.2 The VanRaden matrix, and why it cannot measure diversity

With p_k the reference-allele frequency in the hybrid population and $\mathbf{Z} = \mathbf{M}^H - \mathbf{1}\,2\mathbf{p}'$ the centered matrix,

$$\mathbf{G} = \frac{\mathbf{Z}\mathbf{Z}'}{2\sum_k p_k(1 - p_k)}. \tag{1.3}$$

This is VanRaden's first parametrisation. It estimates realised additive relationship and it is the right matrix for GBLUP. It is implemented as `vanraden_G()`:

```
args(vanraden_G)
#> function (M)
#> NULL
```

Now the failure. Centering on $2\mathbf{p}$ uses frequencies from the population itself, so every column of $\mathbf{Z}$ sums to zero:

$$\mathbf{Z}'\mathbf{1} = \mathbf{0} \quad \Rightarrow \quad \mathbf{1}'\mathbf{G}\mathbf{1} = \frac{\|\mathbf{Z}'\mathbf{1}\|^2}{2\sum_k p_k q_k} = 0. \tag{1.4}$$

The sum of **all** elements of $\mathbf{G}$ is identically zero. Group coancestry is proportional to that sum, so it is zero for the whole population, and the status number $N_s = 1/(2\theta)$ diverges:

```r
c(sum_G      = sum(ctx$G),
  theta_on_G = mean(ctx$G) / 2,
  Ns_on_G    = 1 / (2 * mean(ctx$G) / 2))
#>          sum_G    theta_on_G        Ns_on_G
#> -4.178879e-11 -5.348298e-17 -9.348769e+15
```

That is not a rounding artefact to be worked around. It is exact, and it is what centering means. In a *subset* the sum is not zero, but the quantity is still centered: it oscillates around zero, takes negative values, is not a probability, and admits no identity-by-descent interpretation.

> ! The root cause is the implicit base population
>
> $\mathbf{G}$ measures relatedness *relative to the current population*. Diversity requires an *absolute* scale. Any matrix whose centering constant is estimated from the same data whose diversity you are measuring will have this problem, whatever it is called.

1.3 Molecular coancestry

Malécot's coancestry between i and j is the probability that two randomly sampled alleles, one from each individual at the same locus, are identical. Replacing identity *by descent* with identity *by state* – which is what markers actually observe – yields molecular coancestry [1], [2]:

$$f_{ij} = \frac{1}{m}\sum_{k=1}^{m}\Big[x_{ik}\,x_{jk} + (1-x_{ik})(1-x_{jk})\Big]. \tag{1.5}$$

The first term is the probability that both sampled alleles are the reference allele; the second, that both are the alternative. Three properties follow immediately:

- $f_{ij} \in [0,1]$ always. **A negative value is a symptom of the wrong matrix.**
- Self-coancestry is $f_{ii} = (1+F_i)/2$, with F_i the individual's molecular homozygosity. For a fully heterozygous F1, $f_{ii} = 0.5$.
- No centering, no scaling, no arbitrary base population.

1.3.1 Computable form

The double sum is $O(n^2 m)$ if computed naively. Expanding,

$$\sum_k \big[\,x_{ik}x_{jk} + (1-x_{ik})(1-x_{jk})\big] = 2\sum_k x_{ik}x_{jk} - \sum_k x_{ik} - \sum_k x_{jk} + m, \tag{1.6}$$

so with $\mathbf{s} = \mathbf{X}\mathbf{1}$ the row sums,

$$\mathbf{f} = \frac{1}{m}\Big(2\,\mathbf{X}\mathbf{X}' - \mathbf{s}\mathbf{1}' - \mathbf{1}\mathbf{s}' + m\,\mathbf{J}\Big), \tag{1.7}$$

a single matrix product – the same cost as building $\mathbf{G}$.

```r
molecular_coancestry
#> function (X)
#> {
#>     m <- ncol(X)
```

```
#>     s <- rowSums(X)
#>     f <- (2 * tcrossprod(X) - outer(s, rep(1, nrow(X))) - outer(rep(1,
#>         nrow(X)), s) + m)/m
#>     dimnames(f) <- NULL
#>     f
#> }
#> <bytecode: 0x94da9fa48>
#> <environment: namespace:hybdiv>
```

```
fmt(c(minimum       = min(ctx$f),
      maximum       = max(ctx$f),
      diagonal_mean = mean(diag(ctx$f)),
      offdiag_mean  = mean(ctx$f[upper.tri(ctx$f)])))
#>       minimum       maximum diagonal_mean  offdiag_mean
#>        0.6448        0.8318        0.8192        0.6717
```

Everything is inside $[0, 1]$, as it must be.

1.3.2 Reading the diagonal

The value 0.5 is the floor of the diagonal, reached by a hybrid heterozygous at *every* marker. The excess above 0.5 measures exactly the fraction of markers at which the two parent lines carry the same allele – loci where the F1 is homozygous and therefore contributes nothing to heterosis.

```
fd <- mean(diag(ctx$f))
fmt(c(diagonal_mean         = fd,
      molecular_F           = 2 * fd - 1,     # f_ii = (1 + F_i) / 2
      heterozygous_fraction = 2 * (1 - fd)))
#>         diagonal_mean           molecular_F heterozygous_fraction
#>                0.8192                0.6383                0.3617
```

This fraction depends directly on the divergence between the heterotic pools (`fst` in the configuration): more divergent pools give more heterozygous F1 hybrids and a diagonal closer to 0.5. That relationship is what Chapter 3. puts under experimental control.

1.4 `f` is not an ad hoc substitute

Two independent lines of work converge on this matrix, which is worth stating because it is otherwise easy to read **f** as “the thing we used because **G** broke”.

Nejati-Javaremi et al. (1997) proposed the *allelic-similarity* method, $f_{ij} = \frac{1}{4}\sum_{k=1}^{2}\sum_{l=1}^{2}\delta_{kl}$ with $\delta_{kl} = 1$ when the k-th allele of i equals the l-th of j. That is exactly the identity-by-state average above, generalised from single loci to a genome-wide mean. F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] contrast it directly against crossproduct-based matrices: the allelic-similarity matrix is guaranteed to stay in $[0, 1]$ by construction, while crossproduct matrices normalised by *current* allele frequencies are not – precisely the failure demonstrated above.

The same matrix arrives from a second direction. M. A. Toro, B. Villanueva, and J. Fernández [5] and T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] show that fixing the VanRaden centering frequency at $p_k = 0.5$ for every marker, instead of the sample frequency, produces a matrix – written $\mathbf{G}_{0.5}$ – that is proportional to homozygosity-based inbreeding and molecular coancestry. Structurally that is **f**: replacing $2p_k$ with 1 removes the population-dependent term that forced $\mathbf{1}'\mathbf{G}\mathbf{1} = 0$.

So **f** is the member of the VanRaden family that has an absolute scale, reached independently from two unrelated derivations. That it also happens to be the one that does not break is not a coincidence.

1.5 The division of labour

Matrix	Centering	Range	Sum of all elements	Use
`G` (VanRaden)	sample allele frequencies	centered, signed	exactly 0	trait prediction (GBLUP)
`f` (molecular coancestry)	none	[0, 1]	positive, informative	every diversity quantity

This split is enforced throughout `hybdiv`: every metric takes `ctx$f`, and `ctx$G` appears only where traits are predicted. The test suite asserts `sum(G) == 0` precisely so that a future change which silently starts feeding `G` to a diversity metric fails loudly.

The next chapter takes **f** and asks what happens when the population it describes is not one population, but several.

2. Diversity in a subdivided population

A breeding programme is never one population. It is heterotic groups, or accessions, or landraces, or cycles – and the diversity of the whole is not the average of the diversity of the parts. A. Caballero and M. A. Toro [1] give the partition that makes this precise, and it is the analytical backbone of everything that follows: the constraint in Chapter 10. is a special case of it, and the within/between asymmetry that makes plant hybrid breeding different from animal breeding is a statement about which term of it you are allowed to minimise.

The paper's central claim is uncomfortable and worth stating first: **a method that ranks accessions by distance alone can invert the correct conclusion.** An accession can be the most distant *because* it is internally fixed, and removing it can *raise* the diversity of the pool. We reproduce that result numerically below.

2.1 Notation

- A metapopulation of n subpopulations; subpopulation i has N_i individuals, $N_T = \sum_i N_i$.
- A tilde, $\tilde{x}$, is a weighted mean **within** subpopulations: $\tilde{f} = \sum_i f_{ii} N_i / N_T$.
- A bar, $\bar{x}$, is a mean over **all pairs** in the metapopulation, between-subpopulation pairs included: $\bar{f} = \sum_{i,j} f_{ij} N_i N_j / N_T^2$.
- Always $\bar{f} \leq \tilde{f} \leq \tilde{s}$. The differences between those three numbers *are* the partition of diversity.
- Everything is computed per locus and then averaged over loci. Never average frequencies first.

> ⚠ The subtle point, and the one most often got wrong
>
> Every coancestry here includes the pairs of an individual with **itself** – group coancestry in the sense of Cockerham. That is what makes $1 - f_{ii}$ exactly Nei's expected heterozygosity, and it is what makes the algebra close. Drop the diagonal and none of the identities below hold.

2.2 The three coancestry blocks

2.2.1 f_{ij}, between subpopulations

The probability that two alleles, one sampled at random in subpopulation i and one in j, are identical. From markers, $f_{ij} = \sum_k \tilde{p}_{k,i} \tilde{p}_{k,j}$.

Read it as **redundancy** between two materials. If you cross plants of i with plants of j, the progeny has expected inbreeding $F = f_{ij}$. It answers: what do I gain by recombining these two accessions?

In maize, f_{ij} between a Stiff Stalk pool and a Non-Stiff Stalk pool is typically low – and that low between-group coancestry is exactly what sustains the heterotic pattern. Between two elite lines of the same family it can exceed 0.6, and the cross generates nothing new.

2.2.2 f_{ii}, within a subpopulation

The case $i = j$. From markers $f_{ii} = \sum_k \tilde{p}_{k,i}^2$, the expected homozygosity, so

$$1 - f_{ii} = H_{e,i} = \text{Nei's gene diversity of accession } i. \tag{2.1}$$

A landrace accession regenerated twenty times with forty ears per cycle accumulates coancestry and watches f_{ii} rise silently. It is the cheapest alarm for genetic erosion *inside* a genebank – and it is precisely what a distance tree cannot see.

2.2.3 s_i and F_i, self-coancestry and molecular inbreeding

$$s_i = \sum_k \sum_l \frac{p_{k,l,i}^2}{N_i}, \qquad F_i = 2s_i - 1, \qquad 1 - s_i = H_{o,i} \tag{2.2}$$

with $p_{k,l,i}$ the dosage of allele k in individual l.

> **!** F_i here is molecular, and is not F_{IS}
>
> It is measured against a hypothetical base in which all alleles are distinct. It is **not** comparable across studies using different marker panels. For the question "is this population mating at random?", use α_i below.

In soybean, rice and wheat, pure lines have $s_i \to 1$, so $F_i \to 1$ and $H_o \to 0$. That is expected, not a symptom. It changes only *where* the diversity resides.

2.3 Two per-subpopulation metrics

$$\alpha_i = \frac{F_i - f_{ii}}{1 - f_{ii}}, \qquad G_i = \frac{s_i - f_{ii}}{1 - f_{ii}} = \frac{1 + \alpha_i}{2} \tag{2.3}$$

α_i is the subpopulation's analogue of F_{IS}: positive means an excess of homozygotes (selfing, mating among relatives, a Wahlund effect from internal substructure, null alleles); negative means an excess of heterozygotes (dioecy, self-incompatibility, selection, or a freshly formed F1 or synthetic).

G_i is the proportion of the accession's diversity that sits **between** individuals rather than inside them, and it is probably the most underused quantity in the paper. It is a mating-system meter:

G_i	Situation	Typical material
~ 0.5	perfect panmixia	maize, sunflower, open populations
< 0.5	excess of heterozygotes	F1 synthetics, heterozygous clones, self-incompatibles
-> 1.0	all homozygous	inbred lines, doubled haploids, selfing species

Operationally, $1 - G_i$ is the fraction of an accession's diversity stored *inside* each plant, and it decides the sampling design. With $G_i \to 1$ (selfers), one individual captures almost nothing and you need many plants per accession. With $G_i \approx 0.5$ (maize), a few plants already capture half.

2.4 Distances, and what they actually are

Nei's minimum distance between subpopulations is

$$\mathbf{D}_{ij} = \frac{f_{ii} + f_{jj}}{2} - f_{ij} = \sum_k \frac{1}{2}(\tilde{p}_{k,i} - \tilde{p}_{k,j})^2. \tag{2.4}$$

Note what that is: **half the sum of squared allele-frequency differences** – a variance component, not an evolutionary distance calibrated in time. It is the piece that enters F_{ST}, and it is legitimate. The error in a distance-only method is not using $\mathbf{D}_{ij}$; it is using *only* $\mathbf{D}_{ij}$.

> A free $O(N^2)$ saving
>
> The paper's Eq. 13 computes D_{ij} by a double sum over all pairs of individuals, at cost $O(N_i N_j m)$. That is unnecessary: the identity $D_{ij} = (s_i + s_j)/2 - f_{ij}$ is **exact** and reduces the cost to $O(Nm)$. For a 50k-SNP panel and 5000 accessions this is the difference between hours and seconds.

2.5 The equation that matters most in practice

$$\bar{f} = \sum_i \frac{N_i}{N_T} \Big[\underbrace{f_{ii}}_{\text{internal coancestry}} - \underbrace{\sum_j \mathbf{D}_{ij} N_j / N_T}_{\text{mean distance to the others}} \Big] \tag{2.5}$$

Each accession's contribution to global coancestry – that is, to the *loss* of diversity in the pool – has **two terms of opposite sign**:

- **Term 1 (+).** The more internally fixed an accession is, the more it *raises* global coancestry: it dilutes the pool with copies of the same allele.
- **Term 2 (–).** The more distant it is from the others, the more it *reduces* global coancestry.

A distance-only method sees term 2 alone. That is the whole mechanism of the inverted conclusions.

2.6 F statistics and the partition

$$F_{IS} = \frac{\tilde{F} - \tilde{f}}{1 - \tilde{f}}, \qquad F_{ST} = \frac{\tilde{f} - \bar{f}}{1 - \bar{f}} = \frac{\bar{D}}{1 - \bar{f}}, \qquad F_{IT} = \frac{\tilde{F} - \bar{f}}{1 - \bar{f}} \tag{2.6}$$

with $(1 - F_{IT}) = (1 - F_{IS})(1 - F_{ST})$.

The partition itself is the heart of the paper:

$$(1 - \bar{f}) = \underbrace{(1 - \tilde{s})}_{GD_{WI}} + \underbrace{(\tilde{s} - \tilde{f})}_{GD_{BI}} + \underbrace{(\tilde{f} - \bar{f})}_{GD_{BS}} \tag{2.7}$$

Component	Definition	Biologically
GD_T = 1 - f_bar	total diversity	He of the whole pool if everything were recombined
GD_WI = 1 - s_til	within individuals	heterozygosity carried inside each plant
GD_BI = s_til - f_til	between individuals, within accession	segregating variation available to within-population selection
GD_WS = 1 - f_til	within subpopulations	GD_WI + GD_BI
GD_BS = f_til - f_bar	between subpopulations	divergence; the only component a distance method sees

with the clean identities

$$\frac{GD_{WI}}{GD_T} = (1 - G)(1 - F_{ST}), \quad \frac{GD_{BI}}{GD_T} = G(1 - F_{ST}), \quad \frac{GD_{WS}}{GD_T} = 1 - F_{ST}, \quad \frac{GD_{BS}}{GD_T} = F_{ST} \tag{2.8}$$

2.6.1 Why the partition changes the strategy by species

- **Outcrossers** (maize, sunflower, forages, forest trees): GD_{WI} is large. The diversity is literally *inside* the plants. Conserving means maintaining heterozygosity – regenerate with many individuals and balanced contributions.
- **Selfers** (soybean, wheat, rice, beans) **and DH or line collections**: $GD_{WI} \approx 0$. All diversity is $GD_{BI} + GD_{BS}$. Conserving means maintaining the *number of distinct lines*; heterozygosity is not a target. Regenerating with few plants per line is acceptable; losing lines is not.
- **Clonal crops** (potato, sugarcane, fruit trees): GD_{WI} is high and **frozen** – it does not decay by drift while the clone exists. The N_e algebra does not apply without adaptation.

This is the reason a hybrid maize programme cannot manage diversity with a single pool-wide number: it needs GD_{WS} defended and GD_{BS} held, and those two pull in opposite directions under selection.

2.7 The worked example, reproduced

The paper's metapopulation: four subpopulations of $N = 12, 4, 8, 6$, three loci with 5, 3 and 3 alleles. Its published coancestry matrix:

```
f <- matrix(c(0.3391, 0.3021, 0.2535, 0.3229,
              0.3021, 0.8750, 0.3073, 0.3889,
              0.2535, 0.3073, 0.4766, 0.2483,
              0.3229, 0.3889, 0.2483, 0.3704), 4, 4, byrow = TRUE,
            dimnames = list(paste0("Sub", 1:4), paste0("Sub", 1:4)))
N <- c(12, 4, 8, 6)
w <- N / sum(N)
```

Note subpopulation 2: $f_{22} = 0.875$, so $H_e = 0.125$. It is nearly fixed internally. Now the global quantities, with the paper's published values beside them:

```
f_bar <- drop(w %*% f %*% w)
f_til <- sum(w * diag(f))
s_til <- 0.7056                       # published; needs individual genotypes

data.frame(
  quantity = c("f_bar", "f_til", "GD_T", "GD_WI", "GD_BI", "GD_BS", "F_ST"),
  computed = round(c(f_bar, f_til, 1 - f_bar, 1 - s_til, s_til - f_til,
                     f_til - f_bar, (f_til - f_bar) / (1 - f_bar)), 4),
  published = c(0.3256, 0.4535, 0.6744, 0.2944, 0.2521, 0.1279, 0.1897)
)
```

quantity	computed	published
f_bar	0.3256	0.3256
f_til	0.4535	0.4535
GD_T	0.6744	0.6744
GD_WI	0.2944	0.2944
GD_BI	0.2521	0.2521
GD_BS	0.1279	0.1279
F_ST	0.1897	0.1897

Every value reproduces exactly. Note that GD_{BS} is only 19% of the total: the between-subpopulation divergence – the only thing a distance method measures – accounts for less than a fifth of the diversity in this metapopulation.

2.7.1 Removing an accession can *increase* diversity

```
res <- list(f = f, by_subpop = data.frame(N = N))
cbind(ct_loss_gain(res),
      delta_pct_equal = round(ct_loss_gain(res, "equal")$delta_pct, 2))
```

pop	GD_without	delta_pct	delta_pct_equal
Sub1	0.6296049	−6.6468853	−6.92
Sub2	0.6868734	1.8444502	6.47
Sub3	0.6480760	−3.9081294	−6.94
Sub4	0.6689694	−0.8102105	−3.56

The number to look at is subpopulation 2. A Weitzman-style analysis calls it the most valuable in the set – losing it would cost 65.5% of the diversity, because it is the most divergent. The coancestry analysis says removing it **raises** pool diversity by 1.8% (census weights) or 6.5% (equal weights).

It is distant *because* it is fixed. Alleles fixed in a sample of four individuals are not a genetic resource; they are the result of drift. The same pattern appears in the paper's reanalysis of eleven European pig breeds: the breed a distance method nominates as most valuable turns out to raise diversity by 0.67% when removed.

> What this does and does not license
>
> For a genebank: a rare accession with few plants and high homozygosity – the archetypal "unique" accession on a distance tree – may be contributing *negatively* to the pool. That does **not** mean discard it. It means the correct priority may be to **regenerate it at larger N**, not to replicate it into a core collection. Storing seed is cheap; extinction is irreversible.

2.7.2 Optimal contributions

Maximising $GD_T = 1 - \sum_{i,j} f_{ij} c_i c_j$ subject to $c_i \geq 0$ and $\sum_i c_i = 1$ is a quadratic programme:

```
oc <- optimal_contributions(f)
oc$c_i
#>      Sub1      Sub2      Sub3      Sub4
#> 0.522398 0.000000 0.269890 0.207711
c(GD_before = 1 - f_bar, GD_after = oc$GD_max,
  gain_pct = 100 * (oc$GD_max - (1 - f_bar)) / (1 - f_bar))
#> GD_before  GD_after  gain_pct
#> 0.6744338 0.6873675 1.9177220
```

Subpopulation 2 receives weight **zero**, and total diversity rises from 0.6744 to 0.6874. This reproduces the paper's published solution $\mathbf{c} = (0.522, 0, 0.270, 0.208)$ exactly.

Three direct uses in plant breeding:

- **Base population for recurrent selection.** c_i gives the seed proportion from each source that maximises long-term additive variance.
- **Core collection.** Instead of clustering heuristics plus proportional sampling, solve the quadratic programme directly with operational constraints ($c_i \leq$ multiplication capacity, $c_i \geq$ a policy minimum for accessions of cultural importance).
- **Optimal contribution selection.** Replace the objective with $\max \mathbf{c}'\mathbf{g} - \lambda\,\mathbf{c}'\mathbf{F}\mathbf{c}$ and you have Meuwissen's framework, now standard in genomic programmes. This chapter is the case $\lambda \to \infty$, pure diversity. A breeder operates in the middle of the curve – which is Chapter 10..

2.8 Minimising coancestry is maximising N_e

Since $N_s = 1/(2\bar{f})$, minimising group coancestry and maximising effective size are the same operation. The paper's Eq. 14 makes the operational consequence explicit. With **equal** contributions ($S_W^2 = S_B^2 = 0$), $N_e \approx 2Nn/(1 - F_{IT})$ – roughly **double** the N_e obtained under random (Poisson) contributions.

> Harvesting the same number of seeds from every plant, and the same number of plants from every family, doubles the effective size. No genotyping, no technology, no cost.

It is probably the best cost-benefit intervention available in germplasm management. The opposite – bulk harvest from a plot, which in practice samples the most productive plants disproportionately – inflates S_W^2 and collapses N_e.

2.9 The honest caveats

The paper makes these, and a breeder should take them seriously.

Criteria based on within-population variation systematically **favour larger populations**. A $\bar{f}$ criterion can push a genebank toward abundant elite material – the opposite of the conservation objective. Beyond that:

- Neutral markers do not measure adaptive value, disease resistance, stress adaptation or cultural value. A fixed accession may carry the only resistance allele to an emerging rust.
- **Allelic richness** is more sensitive to bottlenecks than heterozygosity and is often the more relevant criterion for pre-breeding. The argument that minimising $\bar{f}$ maximises allelic richness in the long run (via the effective number of alleles being the inverse of mean coancestry) is valid asymptotically but does **not** protect rare alleles in the short term. This is why allelic richness survives the redundancy analysis in Chapter 8. as the one genuinely independent axis.
- Weighting between-group variation more heavily is arbitrary. A more defensible fix is to optimise **c** with per-accession minimum constraints.

Rule of thumb: use $\bar{f}$ to decide *proportions and regeneration effort*; never use $\bar{f}$ alone to decide *discard*.

2.10 Two errata, found while verifying

Worth recording, since the numbers above were checked against the published tables.

- Table 2b lists $F_2 = 0.8383$, but $F_i = 2s_i - 1 = 2(0.9167) - 1 = 0.8333$. A typographical error. Every other value in Tables 2 and 3 reproduces exactly.
- Conceptually, the Discussion states that equal contributions ($S_B^2 = S_W^2 = 0$) are required "to minimise effective size" and "maximise mean coancestry". That is inverted: Eq. 15 and the entire argument of the paper show that equal contributions **maximise** N_e and **minimise** $\bar{f}$.

II Simulation

3. Simulating lines, pools and crosses

Everything downstream of this chapter consumes three objects and knows nothing about where they came from. That is deliberate: it is the seam at which a simulation is swapped for real data, and it is the only place that has to change when it is.

This chapter builds those objects. It also builds the candidate pool the rest of the book selects from, so the choices made here – how divergent the pools are, how the index is constructed – determine what the later chapters have to work with.

3.1 What has to be produced

Object	Shape	Content	Needed for
`X`	N x m	hybrid marker matrix, values in 0/0.5/1	every frequency-based metric
`f`	N x N	molecular coancestry of the hybrids	every coancestry-based metric
`hybrids`	N x (3 + n_traits)	hybrid id, the two parent lines, predicted traits	the index, and the line-usage vector
`fL`	L x L, optional	coancestry BETWEEN LINES	only the heterotic-group decomposition

3.2 Simulating the lines

Two heterotic pools are generated with a Balding-Nichols model. Ancestral allele frequencies are drawn uniform on $[0.05, 0.95]$, then each pool's frequency at marker k is drawn as

$$p_{A,k} \sim \mathrm{Beta}\left(p_k \frac{1 - F_{ST}}{F_{ST}}, (1 - p_k) \frac{1 - F_{ST}}{F_{ST}}\right), \tag{3.1}$$

and independently for pool B. The Beta has mean p_k and variance $F_{ST}\, p_k(1 - p_k)$, so the parameter `fst` in the configuration *is* the expected between-pool divergence – it is not a proxy for it.

Genotypes are then drawn as $2 \times \mathrm{Bernoulli}(p)$, which makes every line fully homozygous, as an inbred line should be.

```
simulate_lines
#> function (cfg)
#> {
#>     set.seed(cfg$seed)
#>     p_anc <- stats::runif(cfg$m, 0.05, 0.95)
#>     bn <- function(p, fst) {
#>         stats::rbeta(length(p), p * (1 - fst)/fst, (1 - p) *
#>             (1 - fst)/fst)
#>     }
#>     pA <- bn(p_anc, cfg$fst)
#>     pB <- bn(p_anc, cfg$fst)
#>     draw <- function(n, p) {
#>         matrix(2L * stats::rbinom(n * length(p), 1, rep(p, each = n)),
#>             nrow = n)
#>     }
#>     L <- rbind(draw(cfg$n_pool_A, pA), draw(cfg$n_pool_B, pB))
#>     storage.mode(L) <- "double"
```

```
#>     L
#> }
#> <bytecode: 0xa604395e8>
#> <environment: namespace:hybdiv>
```

```
L <- simulate_lines(book_cfg)
dim(L)
#> [1]   50 2000
table(as.vector(L[1:5, 1:200]))
#>
#>   0   2
#> 480 520
```

Only 0 and 2 appear: no residual heterozygotes. With real data those must be imputed or dropped before this point, because the hybrid-genotype identity in Chapter 1. assumes homozygous parents.

3.2.1 The divergence knob

`fst` is the single most consequential parameter in the simulation, because it controls how much heterosis there is to find and therefore how much a diversity constraint can cost. It is worth seeing directly:

```
fst_grid <- c(0.02, 0.05, 0.10, 0.15, 0.25, 0.40)
div <- vapply(fst_grid, function(fst) {
  cfg <- modifyList(book_cfg, list(fst = fst, m = 1500))
  Lx  <- simulate_lines(cfg)
  fL  <- molecular_coancestry(Lx / 2)
  A   <- seq_len(cfg$n_pool_A); B <- cfg$n_pool_A + seq_len(cfg$n_pool_B)
  # a hybrid A x B is heterozygous wherever its parents differ
  c(between_pool_f = mean(fL[A, B]),
    within_pool_f  = mean(c(fL[A, A], fL[B, B])),
    het_fraction   = 1 - mean(fL[A, B]))
}, numeric(3))

d <- data.frame(fst = fst_grid, t(div))
plot(d$fst, d$het_fraction, type = "b", pch = 19, ylim = c(0, 1),
     xlab = expression(F[ST]~"between pools"),
     ylab = "expected heterozygous fraction of the F1")
grid()
```

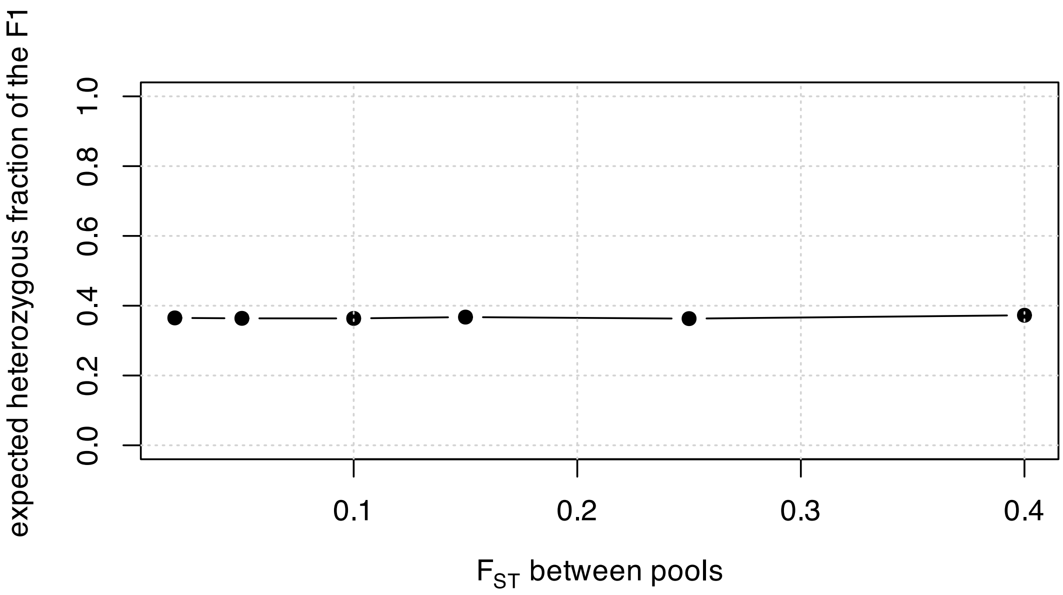

Figure 3.1: Between-pool divergence sets the hybrid heterozygous fraction, and with it the floor of the coancestry diagonal.

```
fmt(d)
```

fst	between_pool_f	within_pool_f	het_fraction
0.02	0.6349	0.6564	0.3651
0.05	0.6362	0.6684	0.3638
0.10	0.6364	0.6877	0.3636
0.15	0.6326	0.7052	0.3674
0.25	0.6370	0.7387	0.3630
0.40	0.6274	0.7898	0.3726

More divergent pools give more heterozygous F1 hybrids. That is the heterosis engine, and Chapter 2. is the reason it must be *held*, not maximised: the same divergence that raises `GD_BS` is being paid for out of within-pool diversity.

3.3 From lines to hybrids

Every A x B combination is formed. With 25 lines per pool that is 625 candidates; at the production scale of 71 per pool it is 5041.

```
cfg <- book_cfg
a_ids <- seq_len(cfg$n_pool_A)
b_ids <- cfg$n_pool_A + seq_len(cfg$n_pool_B)
ped <- expand.grid(a = a_ids, b = b_ids)
M   <- (L[ped$a, , drop = FALSE] + L[ped$b, , drop = FALSE]) / 2   # 0/1/2
dim(M)
#> [1]  625 2000
head(ped, 3)
```

a	b
1	26
2	26
3	26

The factorial structure matters more than it looks. Because every line appears in many candidate hybrids, selecting hybrids is really selecting *line usage* – and that observation, made precise in Section 8.1, is what makes the whole problem tractable at production scale.

3.4 Predicted traits

Marker effects are drawn correlated across traits, through a Cholesky factor of a fixed correlation matrix, and the predicted values are the centered marker matrix times those effects:

```
set.seed(cfg$seed + 99)
R <- matrix(c(1, 0.3, -0.3,
              0.3, 1,  0.1,
             -0.3, 0.1, 1), 3, 3)
vr <- vanraden_G(M)
B <- matrix(rnorm(cfg$m * cfg$n_traits), cfg$m) %*% chol(R)
traits <- scale(vr$Z %*% B)
round(cor(traits), 3)
#>        [,1]  [,2]   [,3]
#> [1,]  1.000 0.259 -0.440
#> [2,]  0.259 1.000  0.049
#> [3,] -0.440 0.049  1.000
```

The realised trait correlations recover the target `R`, which is the point of the Cholesky step: without it, a multi-trait index would be a weighted sum of independent noise, and the trade-off it induces against diversity would be unrealistically easy.

> **i What this simulation deliberately does not model**
>
> These predicted values are constructed from *known* marker effects, so they carry no prediction error. Real GEBVs do. The book therefore treats `ctx$index` as given throughout, and the reader should read every gain figure as an upper bound on what a real programme would realise. The additive model also carries no dominance, so it cannot express specific combining ability – see the caveats in Chapter 13..

3.5 The two matrices, both built

```
X <- M / 2
ctx <- build_ctx(X = X, f = molecular_coancestry(X), G = vr$G, traits = traits,
                 ped = ped, n_lines = nrow(L),
                 pool = rep(c("A", "B"), c(cfg$n_pool_A, cfg$n_pool_B)),
                 fL = molecular_coancestry(L / 2), cfg = cfg)

c(N = ctx$N, m = ctx$m, n_lines = ctx$n_lines,
  sum_G = sum(ctx$G), min_f = min(ctx$f), max_f = max(ctx$f))
#>             N             m       n_lines         sum_G         min_f
#>  6.250000e+02  2.000000e+03  5.000000e+01 -4.178879e-11  6.447500e-01
#>         max_f
#>  8.317500e-01
```

`sum(G)` is zero to machine precision and `f` is inside $[0, 1]$: the two properties from Chapter 1., on this dataset.

The context also freezes two things that must never be recomputed per scenario:

```r
str(ctx[c("p0", "maf_pop", "index")], max.level = 1)
#> List of 3
#>  $ p0     : num [1:2000] 0.44 0.3 0.56 0.9 0.12 0.98 0.98 0.7 0.64 0.22 ...
#>  $ maf_pop: num [1:2000] 0.44 0.3 0.44 0.1 0.12 0.02 0.02 0.3 0.36 0.22 ...
#>  $ index  : num [1:625] -0.291 0.498 -0.361 -0.186 -0.348 ...
```

`p0` is the candidate-pool allele frequency vector. It is the **base** against which `F_hom` and `F_drift` are measured in Chapter 6., and recomputing it per scenario would silently zero out the historical memory of erosion – the single most common error in applying these metrics.

3.6 The stage-1 contract in one call

All of the above is `stage1_simulate()`:

```r
stage1_build
#> function (X, ped, traits, fL = NULL)
#> {
#>     if (max(X) > 1)
#>         X <- X/2
#>     stopifnot(nrow(X) == nrow(ped), nrow(X) == nrow(traits))
#>     hybrids <- data.frame(hybrid = seq_len(nrow(X)), line_A = as.integer(ped$a),
#>         line_B = as.integer(ped$b), traits)
#>     list(X = X, f = molecular_coancestry(X), hybrids = hybrids,
#>         fL = fL)
#> }
#> <bytecode: 0xa5e67a4a0>
#> <environment: namespace:hybdiv>
```

```r
st1 <- stage1_simulate(book_cfg)
str(st1, max.level = 1)
#> List of 4
#>  $ X      : num [1:625, 1:2000] 0.5 0.5 0 0 0 0.5 0.5 0 0 0 ...
#>  $ f      : num [1:625, 1:625] 0.818 0.74 0.743 0.738 0.745 ...
#>  $ hybrids:'data.frame': 625 obs. of  6 variables:
#>  $ fL     : num [1:50, 1:50] 1 0.677 0.692 0.674 0.684 ...
head(st1$hybrids, 3)
```

hybrid	line_A	line_B	trait1	trait2	trait3
1	1	26	−0.6880637	0.1907069	0.0153955
2	2	26	−0.6102738	−0.0320787	1.4661012
3	3	26	−0.3094099	−0.5013539	0.2137806

For real data the same function is called with your own matrices:

```r
st1 <- stage1_build(
  X      = hybrid_markers,   # N x m, coded 0/1/2 or 0/0.5/1 (auto-detected)
  ped    = pedigree,         # data.frame with columns a, b
  traits = predicted_traits, # N x n_traits
  fL     = line_coancestry   # optional
)
```

`stage1_build()` detects the marker coding and rescales, so both conventions give an identical result:

```r
tr <- st1$hybrids[, grep("^trait", names(st1$hybrids))]
pd <- data.frame(a = st1$hybrids$line_A, b = st1$hybrids$line_B)
identical(stage1_build(st1$X, pd, tr)$f,
          stage1_build(st1$X * 2, pd, tr)$f)
#> [1] TRUE
```

3.7 What the candidate pool looks like

```r
op <- par(mfrow = c(1, 2), mar = c(4.2, 4.2, 2.5, 1))
hist(ctx$index, breaks = 40, col = "grey85", border = "white",
     main = "multi-trait index", xlab = "index (s.d. units)")
gd_random <- replicate(400, gene_diversity(sample.int(ctx$N, 60), ctx))
hist(gd_random, breaks = 30, col = "grey85", border = "white",
     main = "GD of random subsets of 60", xlab = "gene diversity")
abline(v = gene_diversity(seq_len(ctx$N), ctx), col = "#b2182b", lwd = 2)
```

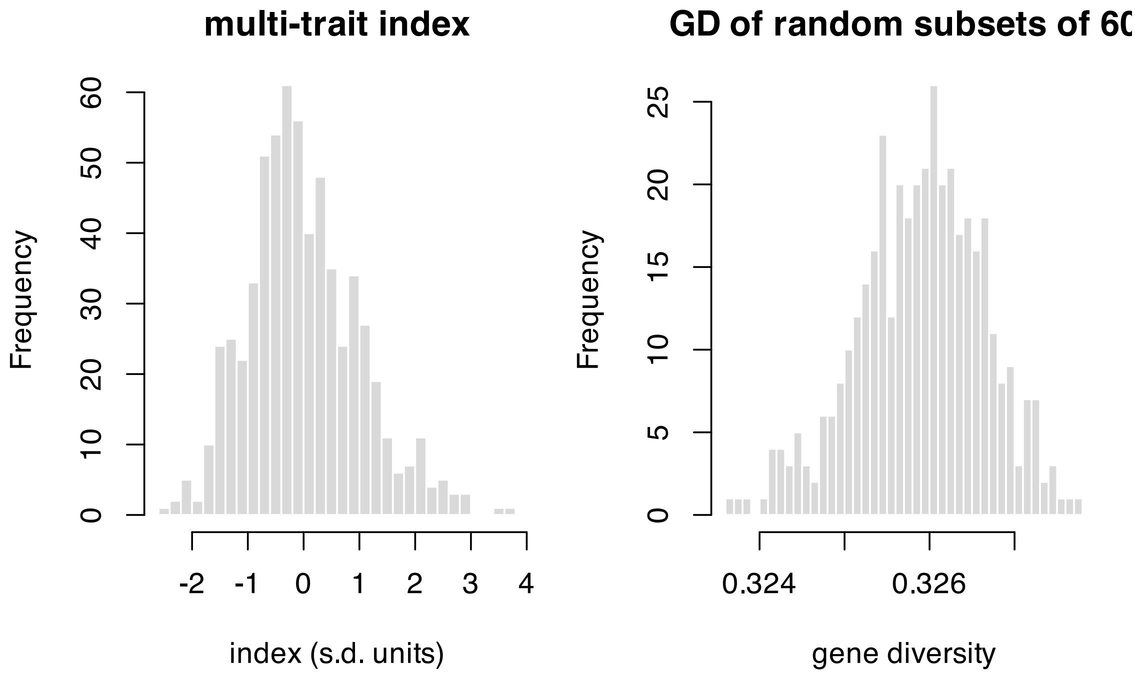

Figure 3.2: The candidate pool. Left: the index the selection maximises. Right: gene diversity of random subsets of 60, the baseline every later chapter measures against.

```r
par(op)
```

The red line is the gene diversity of the *whole* pool. It does not sit at the centre of the distribution of random subsets of 60 – it sits above it. That gap is pure sampling, it has nothing to do with selection, and mistaking it for a loss is the subject of Section 7.9.

```r
c(GD_population = gene_diversity(seq_len(ctx$N), ctx),
  GD_random_60   = mean(gd_random),
  apparent_loss_pct = 100 * (gene_diversity(seq_len(ctx$N), ctx) - mean(gd_random)) /
                            gene_diversity(seq_len(ctx$N), ctx))
#>     GD_population      GD_random_60 apparent_loss_pct
#>         0.3280928         0.3259023         0.6676555
```

4. Sizing a segregating population, F1 to F4

The previous chapters took the candidate pool as given. This one asks where it comes from: a wide cross between two divergent parents, carried F1 -> F2 -> F3 -> F4, and the question every programme has to answer before it plants anything – **how many plants?**

The answer turns out to depend entirely on what you are sizing *for*. There is no single optimal N, and the most common failure is to size for the objective that saturates earliest and then be surprised when a different objective was never met.

4.1 The simulated space is ancestry, not alleles

Each genome position is coded 0 (from parent P1) or 1 (from parent P2). For a wide biparental cross between two divergent heterotic groups this is the correct abstraction: essentially every locus polymorphic between the parents is biallelic, so ancestry is a sufficient statistic. Throughout this chapter, "allele frequency" means "frequency of P1 ancestry".

4.2 The assumptions, one at a time

These are the assumptions the numbers rest on. Read them before trusting any of the numbers.

P1. Ancestry, not alleles. As above.

P2. Parents fully homozygous and fixed for alternative alleles. Consequently every F1 plant is genetically identical and the genetic N_e of the F1 is 1. Sizing the F1 is a **logistical** decision, not a genetic one – unless the parents carry residual heterozygosity, which is handled separately below.

P3. The genetic map. Ten chromosomes, lengths from a classical composite maize map, about 1670 cM (~17 Morgans). Do **not** use IBM-type map lengths from intermated RILs: they are inflated 5-10x by construction and will make every answer wrong.

```
map <- build_map(maize_chr_len, spacing_cM = 2)
c(chromosomes = length(map$chr_len), markers = map$m,
  total_cM = map$total_cM, morgans = map$total_cM / 100)
#> chromosomes     markers    total_cM     morgans
#>        10.0       846.0      1670.0        16.7
```

P4. Haldane recombination, no interference, identical in male and female. The probability of a haplotype switch between adjacent markers d cM apart is $r = 0.5(1 - e^{-2d/100})$; the first marker of each chromosome gets $r = 0.5$, which produces independent segregation between chromosomes automatically.

Limitation: without interference the variance in crossover number is inflated (Poisson). Maize has strong positive interference, so the simulated block-length distribution has a slightly longer tail than reality. Mean block lengths and allele frequencies are unaffected.

P5. Segregation distortion by rejection sampling, therefore exact rather than approximate. Two mechanisms: *gametophytic* (ga1/Ga1-s type, pollen incompatibility) acting only on the male gamete, and *zygotic* (hybrid lethality or subvitality) as genotype fitnesses. Because rejection acts on the **whole gamete**, the region linked to the distorter is dragged along – which is exactly the phenomenon of interest.

```
f2_distorters
```

chr	pos_cM	type	k	w00	w01	w11	label
4	75	gametic	0.80	NA	NA	NA	ga1-like (4.05)
1	160	zygotic	NA	1	1	0.45	partial lethal (1.10)
5	60	gametic	0.68	NA	NA	NA	weak gametic (5.03)

> ⚠ The distorters are plausible, not measured
>
> This is the parameter that moves the results most. Replace `f2_distorters` with your own F2 genotyping data before using any number here to decide field area.

P6. Five schemes, all ending with $H = 0.125$ so they are directly comparable: `ssd` (one seed per plant per generation – the neutral reference, no selection, no drift beyond the meiotic), `ssd_attr` (SSD with 15% random loss per generation: vigour, partial sterility, flowering asynchrony – common in wide crosses with exotic material), `bulk` (differential seed contribution, a Wright-Fisher model with weights $w_i = e^{\beta(z_i - \bar{z})}$ where z_i is the share of genome from the adapted parent – a *phenomenological* model of polygenic natural selection, not a model of adaptation loci), and `syn1`/`syn2` (one or two random intercrossing cycles in the F2 before the two selfings).

P7. The F4 panel is not a RIL panel. Residual heterozygosity is $H = 0.125$, and that changes every theoretical expectation:

Quantity	Formula	Value in F4
Var(dosage per line)	(1 - H) / 4	0.21875
SE(p_hat)	0.5 sqrt((1 - H) / N)	0.468 / sqrt(N)
Diversity retained D	1 - (1 - H) / N	1 - 0.875 / N
P(P2 homozygote per locus)	(1 - H) / 2	0.4375
Junctions per haplotype	A_2 = L; A_(t+1) = A_t + L H_t	1.75 L

P8. The standard error is estimated between replicates, not between markers. This is essential: linked markers are strongly autocorrelated, so the spread across markers within one replicate is not an estimator of sampling error.

P9. What is not modelled, and therefore cannot be answered here: deliberate artificial selection, QTL architecture, phenotypic inbreeding depression, mutation (irrelevant on this timescale), and structural variation. If you know there is a large inversion between your parents, zero the recombination rate over that interval in `build_map()`.

Known bias: at 2 cM spacing, double crossovers within an interval are not counted. The bias is -2.0% in junction number, always downward. Use `spacing_cM = 1` if you need a faithful count.

4.3 Verifying the identities live

The theory above is not decoration – it is checkable, and the simulator is only trustworthy if it reproduces it.

```r
map10 <- build_map(maize_chr_len, spacing_cM = 10)
neutral <- make_distortion(map10, NULL, active = FALSE)

set.seed(4)
F1 <- make_F1(map10, 400)
F2 <- cross_indices(F1, 1:400, 1:400, map10, neutral)
F3 <- cross_indices(F2, 1:400, 1:400, map10, neutral)
F4 <- cross_indices(F3, 1:400, 1:400, map10, neutral)

data.frame(
  generation = c("F1", "F2", "F3", "F4"),
  H_observed = round(c(panel_het(F1), panel_het(F2), panel_het(F3), panel_het(F4)), 4),
  H_expected = c(1, 0.5, 0.25, 0.125)
)
```

generation	H_observed	H_expected
F1	1.0000	1.000
F2	0.4968	0.500

generation	H_observed	H_expected
F3	0.2541	0.250
F4	0.1268	0.125

Heterozygosity halves at every selfing, as it must. And the diversity identity:

```
set.seed(5)
check <- do.call(rbind, lapply(c(20, 50, 100, 200), function(N) {
  pop <- run_scheme("ssd", N, map10, neutral, f2_opt)
  H <- panel_het(pop)
  data.frame(N = N,
             D_observed = diversity_retained(panel_freq(pop)),
             D_expected = 1 - (1 - H) / N)
}))
fmt(check, 4)
```

N	D_observed	D_expected
20	0.9582	0.9569
50	0.9845	0.9825
100	0.9904	0.9912
200	0.9954	0.9956

Map expansion is an algebraic prediction, and the payoff is a result many programmes get wrong:

```
Lm <- map$total_cM / 100
rbind(SSD  = expected_junctions("ssd",  map),
      Syn1 = expected_junctions("syn1", map),
      Syn2 = expected_junctions("syn2", map))
#>      junctions mean_block_cM
#> SSD     29.225      42.57489
#> Syn1    37.575      35.10247
#> Syn2    45.925      29.86142
```

Each Syn cycle adds only $0.5L$ junctions, not $1.0L$. The extra meiosis creates a junction only where the parent is heterozygous, and $H = 0.5$ at the F2/Syn stage. Budgeting an intercrossing cycle as "one more map length of recombination" overstates what it buys by a factor of two.

4.4 The results

The full grid – five schemes, 20 population sizes, adaptive replication – takes several minutes and is shipped precomputed.

N	D	SE(p_hat)	skewed genome	min(p, 1-p)	N equivalent
10	0.903	0.148	42.8%	0.069	9
30	0.961	0.084	14.4%	0.187	23
50	0.972	0.066	7.9%	0.209	32
70	0.978	0.056	5.6%	0.221	40
90	0.981	0.047	4.3%	0.233	47
110	0.982	0.044	4.2%	0.232	48
150	0.984	0.039	3.8%	0.236	56
190	0.985	0.032	3.7%	0.227	60
290	0.988	0.027	3.3%	0.240	70

N	D	SE(p_hat)	skewed genome	min(p, 1-p)	N equivalent
390	0.989	0.023	3.1%	0.242	77

4.4.1 The diversity curve saturates at 0.988, not at 1

```
schemes <- unique(f2$scheme)
cols <- c(ssd = "#1b6ca8", ssd_attr = "#7fb3d5", bulk = "#c0392b",
          syn1 = "#27ae60", syn2 = "#16a085")
plot(NA, xlim = range(f2$N), ylim = c(0.88, 1.0),
     xlab = expression(N[F2]~"plants"), ylab = "diversity retained, D")
grid()
for (s in schemes) {
  d <- f2[f2$scheme == s, ]
  lines(d$N, d$div, type = "b", pch = 19, cex = 0.6, col = cols[s], lwd = 1.8)
}
curve(1 - 0.875 / x, add = TRUE, lty = 2, col = "grey30", lwd = 1.5)
legend("bottomright", c(schemes, "theory, no distortion"),
       col = c(cols[schemes], "grey30"), lwd = c(rep(1.8, length(schemes)), 1.5),
       lty = c(rep(1, length(schemes)), 2), pch = c(rep(19, length(schemes)), NA),
       bty = "n", cex = 0.8)
```

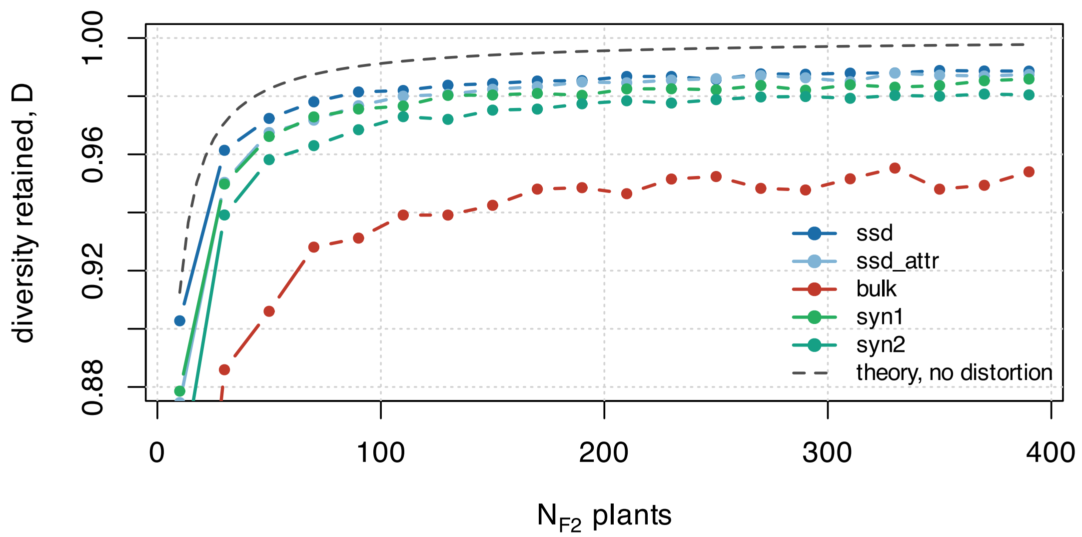

Figure 4.1: Diversity retained against population size, five schemes. The theoretical curve is 1 - (1 - H)/N under no distortion.

The gap between the observed curve and the theoretical one is **segregation distortion** – a deterministic bias that does not shrink with more plants. Adding plants buys you drift protection; it does not buy you distortion protection.

4.4.2 The bulk is the worst scheme across the whole range

comparison	N	D
bulk, largest N	390	0.954
SSD, N = 30	30	0.961

Bulk at the largest population size retains **less** diversity than SSD at 30 plants. With a less adapted donor parent, natural selection in the bulk removes precisely the germplasm the wide cross was made to introduce. If the objective is introgression, the bulk is not a cheaper SSD – it is a different experiment.

4.4.3 Block size does not depend on N

```
map4 <- build_map(maize_chr_len, spacing_cM = 4)
dist <- make_distortion(map4, f2_distorters)
set.seed(6)
small <- panel_freq(run_scheme("ssd", 20,  map4, dist, f2_opt))
large <- panel_freq(run_scheme("ssd", 300, map4, dist, f2_opt))

plot(small, type = "l", col = "#e08214", ylim = c(0, 1),
     xlab = "genome (chromosomes 1-10)", ylab = "P1 ancestry frequency", xaxt = "n")
lines(large, col = "#1b6ca8", lwd = 1.6)
abline(h = 0.5, lty = 2, col = "grey50")
# chromosome boundaries and distorter positions
bnd <- which(map4$first)
axis(1, at = (c(bnd, map4$m + 1)[-1] + bnd) / 2, labels = seq_along(map4$chr_len))
abline(v = bnd[-1] - 0.5, col = "grey85")
for (i in seq_len(nrow(f2_distorters)))
  abline(v = marker_index(map4, f2_distorters$chr[i], f2_distorters$pos_cM[i]),
         col = "#b2182b", lty = 3)
legend("topleft", c("N = 20", "N = 300", "distorter"), col = c("#e08214", "#1b6ca8",
"#b2182b"),
       lty = c(1, 1, 3), lwd = c(1, 1.6, 1), bty = "n", cex = 0.8, horiz = TRUE)
```

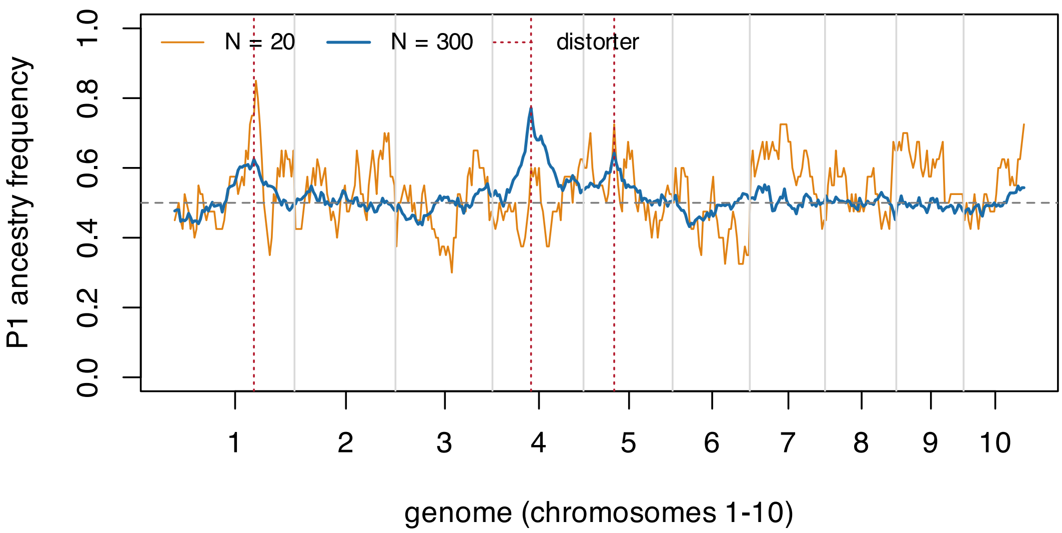

Figure 4.2: Ancestry frequency along the ten chromosomes, at N = 20 and N = 300. At N = 20 the whole curve is noisy: that is drift, and more plants remove it. At N = 300 the noise is gone and three peaks remain, exactly at the distorters. Those are the deterministic bias, and no number of plants removes them.

Block length depends only on the number of meioses. Against linkage drag the instrument is an **intercrossing cycle, not field area** – which is what the Syn curves in the theory table above quantify.

4.4.4 The metrics saturate at different N

This is the central practical result, and the reason the question "what is the optimal N?" has no single answer.

Objective	Saturates near
Diversity retained (D)	100-120
Worst region of the genome, min(p, 1-p)	60-80
Precision of the panel's p_hat	**never** – falls as 1/sqrt(N)
Recovery of 5 loci	200-260
Recovery of 8-10 loci	**not saturated** at the largest N tested

```
kk <- sort(unique(ml$k))
plot(NA, xlim = range(ml$N), ylim = c(0, 1), xlab = expression(N[F2]),
     ylab = "P(at least one line)")
grid()
for (i in seq_along(kk)) {
  d <- ml[ml$k == kk[i], ]
  lines(d$N, d$P, type = "b", pch = 19, cex = 0.6, col = i + 1, lwd = 1.8)
}
legend("topright", paste("k =", kk), col = seq_along(kk) + 1, lwd = 1.8,
       pch = 19, bty = "n")
```

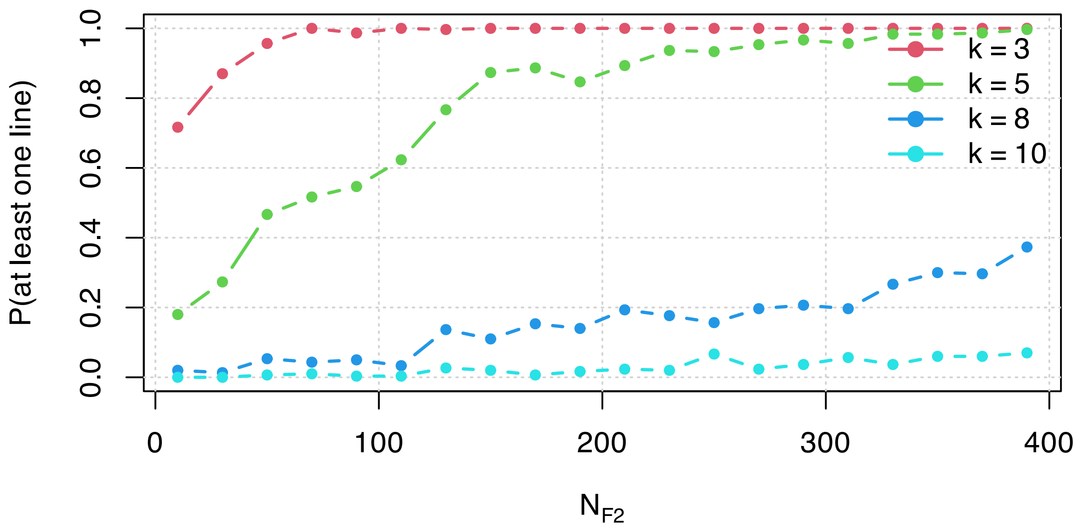

Figure 4.3: Multilocus recovery: P(at least one line homozygous for P2 at k loci). This is the objective that keeps paying for larger N.

The marginal gain in D tells the same story from the other side:

step	gain_in_D
10 -> 30	0.0586
30 -> 50	0.0110
50 -> 70	0.0057
70 -> 90	0.0034
90 -> 110	0.0005
110 -> 130	0.0018

The knee is early. Past it, twenty more plants buy thousandths.

4.5 Sizing the F1

With fully homozygous parents the F1 is genetically uniform and the decision is logistical. With residual heterozygosity – $F \approx 0.95$ is common in programme lines – each parent still segregates at a fraction of loci, and each F1 plant receives one random gamete from each parent. At a segregating locus the chance of losing one of the two parental alleles with n F1 plants is $2(1/2)^n$.

```
s <- f1_sizing(n_F1 = 1:20, h_res = 0.05, n_loci_eff = 5000)
s[s$n_F1 %in% c(4, 8, 10, 12, 15, 20), ]
```

	n_F1	p_loss_per_locus	expected_loci_lost
4	4	0.1250000	31.2500000
8	8	0.0078125	1.9531250
10	10	0.0019531	0.4882812

	n_F1	p_loss_per_locus	expected_loci_lost
12	12	0.0004883	0.1220703
15	15	0.0000610	0.0152588
20	20	0.0000019	0.0004768

Twelve selfed F1 plants already put the expected number of lost loci below one. That is the whole F1 sizing argument.

4.6 Working numbers

Generation	Plants	Rationale
F1	8-15 selfed	Ne = 1, a logistical call. With 5% residual heterozygosity, 12 plants put expected lost loci below 1
F2	**120** to retain diversity	Above this, +280 plants buy +0.005 of D
	250-400 for multilocus selection or panel frequencies	These are the objectives that keep gaining with N
F3	same N (SSD, one seed per plant)	Does not create diversity, only avoids losing it
F4	lines x 3-8 plants	Seed multiplication. Plants of the same line are not independent replicates: they share 87.5% of the genome

On the “equivalent N”

$N_{eq} = (1 - H)/(1 - D)$ answers “how many F4 lines without distortion would give the diversity actually observed”. It is a **communication device, not a Wright effective size**: it converts a deterministic bias into a drift equivalent. Use it to compare scenarios; never substitute it into an N_e formula.

4.7 Where this connects

The panel this chapter sizes is the raw material for everything else. Its diversity is the ceiling: no selection stage downstream can recover variation that was never sampled here. Chapter 9. puts a number on how the loss is distributed across the whole pipeline, and this stage is where the cheapest insurance is available – more plants, once, at the start.

5. Introgressing several genes by backcrossing

Chapter 4. sized a segregating population for *diversity*: the objective was to sample as much of the cross as possible. This chapter is the mirror image. A donor line carries k genes worth having and an elite line carries everything else worth having, and the job is to move the k genes across while **throwing the rest of the donor away**. Here donor genome is contamination, and the only diversity that is wanted is the k target loci themselves.

The operation is marker-assisted backcrossing (MABC). The numbers below come from a dependency-free simulator of it: a marker grid at 5 cM on ten maize chromosomes (1670 cM), one selected plant advanced per generation, $k = 1$ to 4 recessive targets from a single donor, carried to BC3. Its source, test battery and technical review are in `doc/multigenes/`.

5.1 The operation, in one diagram

Before any sizing question can be asked, the crossing operations have to be unambiguous – which plant is pollinated by which, and what is genotyped at each step.

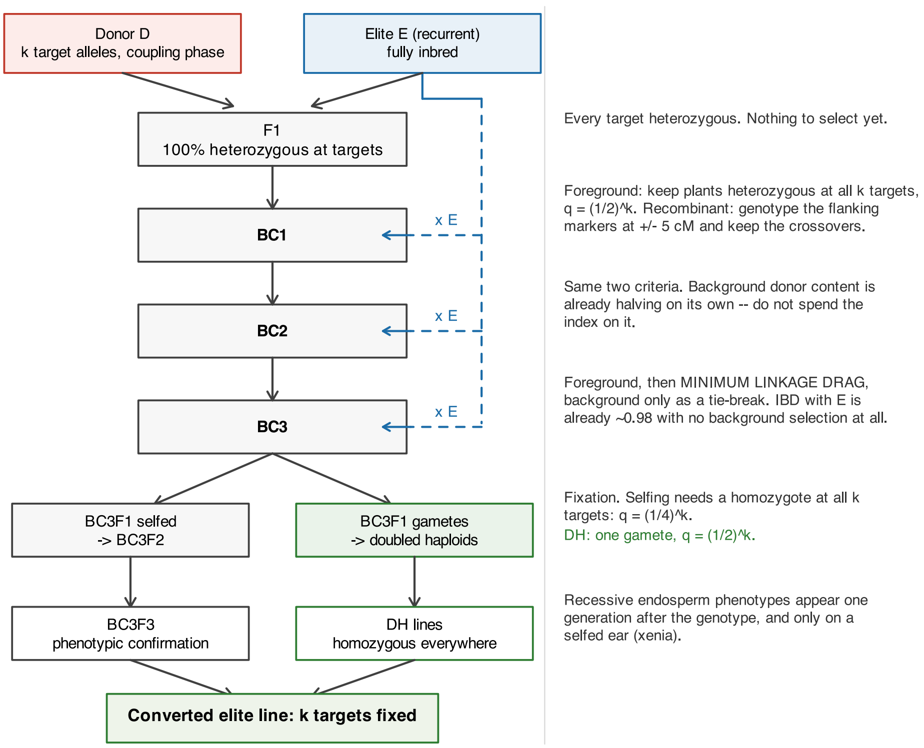

Figure 5.1: The MABC operation. Left column: the cross made in each generation. Right column: what is genotyped and selected on, and the population size that criterion demands. The recurrent parent E re-enters at every backcross; the fork at the bottom is the fixation step, where doubled haploids and selfing differ by more than an order of magnitude in plants required.

Two features of that diagram do all the work in the rest of the chapter. The recurrent parent re-enters at every backcross, so donor content halves every generation *whether or not anyone selects for it*; and the fork at the bottom is where the plant numbers explode, because fixation asks for a homozygote and backcrossing never does.

5.2 The model

Two states, one haplotype. The recurrent parent is a fully homozygous inbred, so the gamete it contributes is invariant and only the donor-derived haplotype has to be tracked: a binary vector over 334 markers. Meiosis is a per-interval Bernoulli switch with Haldane's

$$r = \frac{1}{2}\left(1 - e^{-2d/100}\right), \tag{5.1}$$

which is exact for this chain precisely because Haldane is memoryless (see the limitation in Section 5.8).

Donor dosage and IBD are different quantities. Donor and elite already share a fraction s of the genome by descent – they are both maize, and often both from the same heterotic group. Donor genome inside a shared tract costs nothing. So with D the donor dosage,

$$\mathrm{IBD}_E = 1 - (1 - s)\,D, \qquad \mathrm{IBD}_E + \text{donor-specific} = 1. \tag{5.2}$$

Both identities hold exactly in the simulator and are asserted in its test battery. Everything below uses $s = 0.6$.

Coupling phase. All k targets come from one donor, on the same haplotype. Two consequences, and the second is easy to get wrong. Linked targets travel *together*, so linkage drag is the **union** of the donor segments, not their sum. And conditional on both flanking targets being donor-derived, an *even* number of crossovers is forced in the interval between them, so the probability that they sit in one continuous donor segment is not e^{-L} but

$$P_{\text{same}} = \frac{e^{-L}}{\left(1 + e^{-2L}\right)/2}, \qquad L = d/100 \ \text{ Morgans}\ . \tag{5.3}$$

```
p_same <- function(d_cM) { L <- d_cM / 100; exp(-L) / ((1 + exp(-2 * L)) / 2) }
data.frame(d_cM = c(20, 60, 400),
           naive = exp(-c(20, 60, 400) / 100),
           conditional = p_same(c(20, 60, 400)))
```

d_cM	naive	conditional
20	0.8187308	0.9803280
60	0.5488116	0.8435507
400	0.0183156	0.0366190

At 20 cM that is 0.98 against a naive 0.82. Published stacking simulations get this wrong often enough that it is worth stating separately.

> **!** If the alleles come from different donors, none of this applies
>
> Coupling phase is an assumption, not a convenience. Two linked targets in repulsion, from two donors, cannot both be recovered without a recombinant in the interval, and the cost explodes. That programme needs an intercross between the donors before any backcrossing starts, and this model does not describe it.

5.3 Three criteria, and only one of them binds

A conversion programme is usually written with three success conditions at BC3: do not lose the genes, recover the recurrent parent, and do not drag a chunk of donor chromosome along with each target. They are not remotely equal in cost.

Not losing the genes is exact and cheap. A plant is foreground-positive with probability $q = (1/2)^k$, so n plants fail with probability $(1 - q)^n$:

```
n_for_count <- function(q, count = 1, P = 0.95) {
  n <- count
  while (pbinom(count - 1, n, q) > 1 - P) n <- n + 1L
  n
}
data.frame(genes = mb_sz$genes, q = mb_sz$q_foreground,
           n_BC_per_gen = mb_sz$n_BC_1plant)
```

genes	q	n_BC_per_gen
1	0.5000	5
2	0.2500	11
3	0.1250	23
4	0.0625	47

Fewer than 50 plants per generation at four genes, and that is the whole foreground problem.

Recovering the elite is free by BC3. With no background selection at all, expected donor dosage after 3 backcrosses is $(1/2)^4 = 0.0625$, so

$$\mathrm{IBD}_E = 1 - (1 - s)\,(1/2)^4 = 0.975 \tag{5.4}$$

at $s = 0.6$ – comfortably above a 90% target before a single background marker is scored.

genes	N = 20	N = 80	N = 200	N = 400
1	0.984	0.995	0.997	0.998
2	0.970	0.985	0.990	0.992
3	0.956	0.973	0.980	0.984
4	0.944	0.954	0.969	0.975

That leaves **linkage drag** – the donor chromosome still attached to each target – as the only criterion that a large population can buy. It is the criterion the rest of the chapter is about.

> **i The reframing**
>
> An index that spends its decisive stage on background recovery is spending it on the criterion that was already satisfied. This is not a hypothetical: it was the defect the technical review found in the first version of this simulator, and correcting it reverses the headline conclusion. The next section is that correction.

5.4 Which selection index

Four policies, all applied to foreground-positive plants only:

Key	Rule
`bg`	minimise background donor content
`rec12`	recombinant selection in BC1-BC2 only (classical practice), then background
`rec_bg`	recombinant selection throughout, **background** decides among survivors
`rec_drag`	recombinant selection throughout, **linkage drag** decides, background breaks ties

`rec_bg` is the natural-looking two-stage rule: truncate on drag, then pick the cleanest background among the survivors. Stage 2 discards stage 1. The plant with the least drag survives the truncation and then loses to a sibling that happens to carry less donor genome somewhere irrelevant.

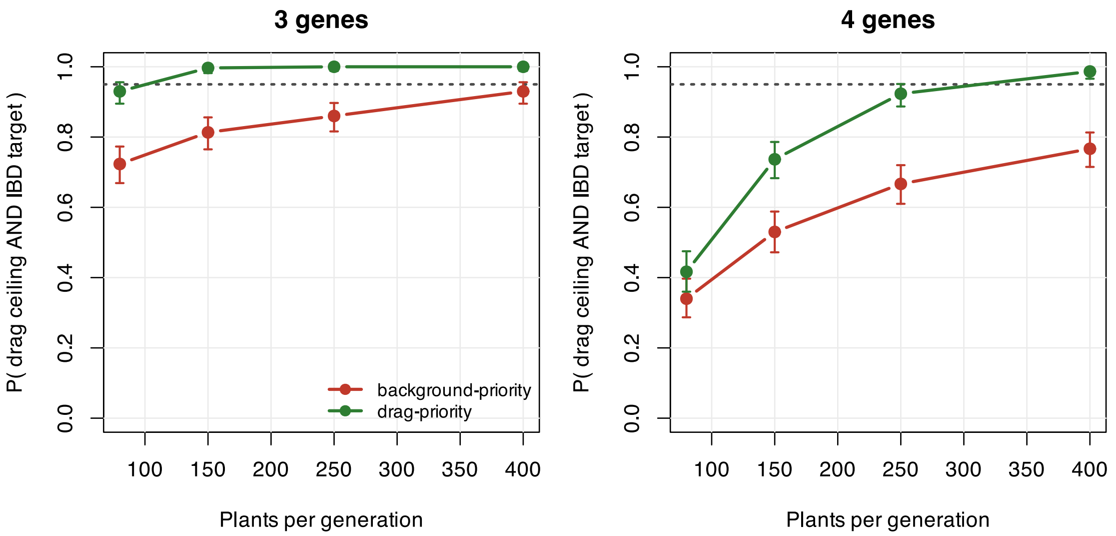

Figure 5.2: Paired comparison (common random numbers, 300 replicates, s = 0.60) of the two-stage background-priority index against the drag-priority index. Bars are Clopper-Pearson 95% intervals. The criterion is drag <= 30 cM per gene AND IBD >= 0.90 at BC3.

genes	N/gen	drag cM	IBD	P(both)	drag cM	IBD	P(both)
3	80	78.7	0.972	0.723	64.6	0.962	0.930
3	150	68.8	0.979	0.813	49.4	0.966	0.997
3	250	65.7	0.982	0.860	41.5	0.968	1.000
3	400	59.0	0.984	0.930	36.0	0.969	1.000
4	80	138.9	0.956	0.340	130.2	0.952	0.417
4	150	120.9	0.965	0.530	105.7	0.958	0.737
4	250	107.9	0.971	0.667	87.6	0.960	0.923
4	400	103.9	0.973	0.767	75.3	0.961	0.987

The trade is 1.1 percentage points of IBD for 19.2 cM of drag, and since the IBD constraint was met in every replicate with margin, it is free. Four genes reach 0.987 at 400 plants per generation. Under the background-priority index the same cell reaches 0.767, which is what produced the original – and wrong – conclusion that the ceiling is unreachable at four genes and the programme must be extended to BC4-BC5 or split in two. **The wall was a property of the index, not of maize.**

5.5 How long to keep selecting recombinants

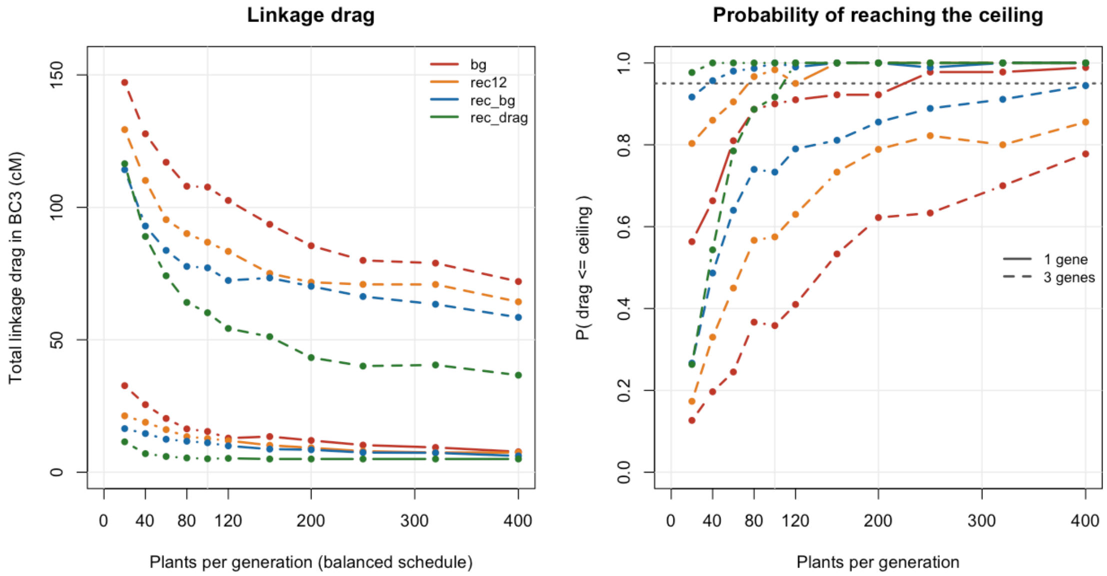

Figure 5.3: Linkage drag at BC3 (left) and probability of meeting the ceiling (right) under the four policies, s = 0.60. Solid = 1 gene, dashed = 3 genes. Extending recombinant selection into BC3 costs no extra plants, only the flanking markers on plants that are being genotyped anyway.

Classical practice stops recombinant selection after BC2, on the reasoning that BC3 is for background clean-up. By BC3 the background needs no clean-up, and the expected distance to the nearest crossover from a target shrinks as $1/g$ with generation g – so BC3 is the generation where a plant buys the *most* drag reduction. Both of the free changes in Figure 5.3 point the same way: genotype the flanking markers in BC3 as well, and let drag decide.

5.6 Allocating a fixed budget across BC1, BC2 and BC3

Given a total number of plants to grow, how should they be split between the three backcrosses?

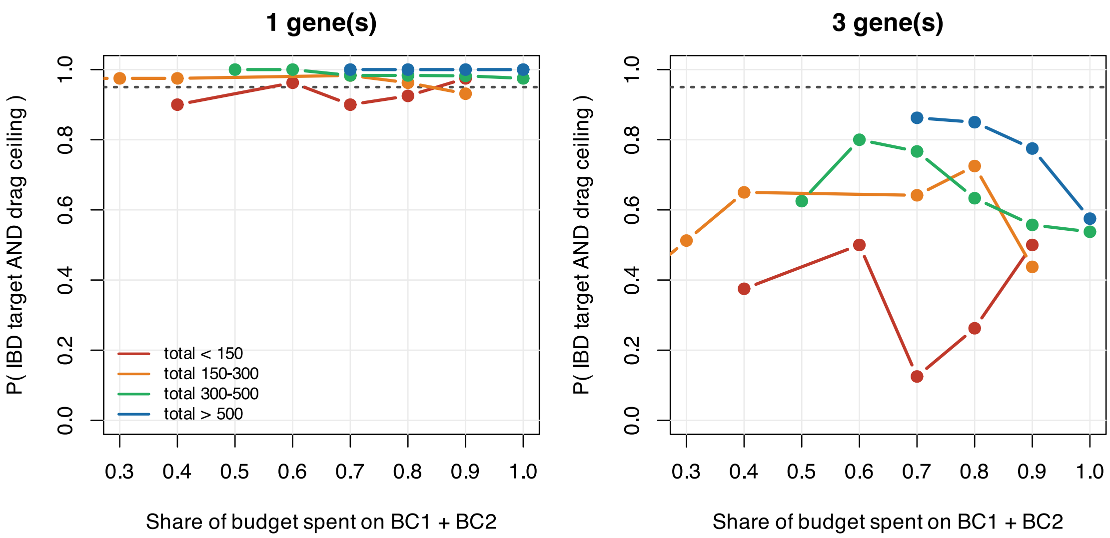

Figure 5.4: P(criterion met) against the share of a fixed budget spent early, for 1 and 3 genes, binned by total plants. Front-loading BC1 is the worst use of a fixed budget.

A paired re-run at a fixed budget of 360 plants, three genes, makes the ordering explicit:

BC1/BC2/BC3	drag cM	P(both)	lo	hi	drag cM	P(both)	lo	hi
300/40/20	93.0	0.448	0.385	0.512	75.6	0.656	0.594	0.715
120/120/120	72.6	0.772	0.715	0.823	53.5	0.988	0.965	0.998
20/150/190	78.7	0.640	0.577	0.700	58.9	0.876	0.829	0.914
40/120/200	74.3	0.748	0.689	0.801	58.1	0.940	0.903	0.966
200/120/40	79.7	0.680	0.618	0.737	59.3	0.932	0.893	0.960

The balanced schedule wins under both indices, and 300/40/20 – the intuitive "cast a wide net early" schedule – is the worst. The mechanism is the $1/g$ shrinkage again: a plant grown in BC3 is offered a much shorter donor segment to trim than a plant grown in BC1, so it converts into drag reduction far more efficiently.

5.7 Fixing the stack: selfing against doubled haploids

Backcrossing only ever asks for a heterozygote. Fixation asks for a homozygote at all k targets, and that is where the plant numbers move by an order of magnitude. Selfing a BC3F1 gives $q = (1/4)^k$; a doubled haploid derived from a BC3F1 gamete is homozygous everywhere, so it carries all k targets with $q = (1/2)^k$.

And the requirement is not one plant but 3, which is a *cumulative binomial* $P(\mathrm{Bin}(n, q) \geq 3) \geq 0.95$, not $1 - (1 - q)^n$:

genes	q (selfing)	BC3F2 plants	q (DH)	DH lines
1	0.25000	23	0.5000	11
2	0.06250	99	0.2500	23
3	0.01562	401	0.1250	49
4	0.00391	1610	0.0625	99

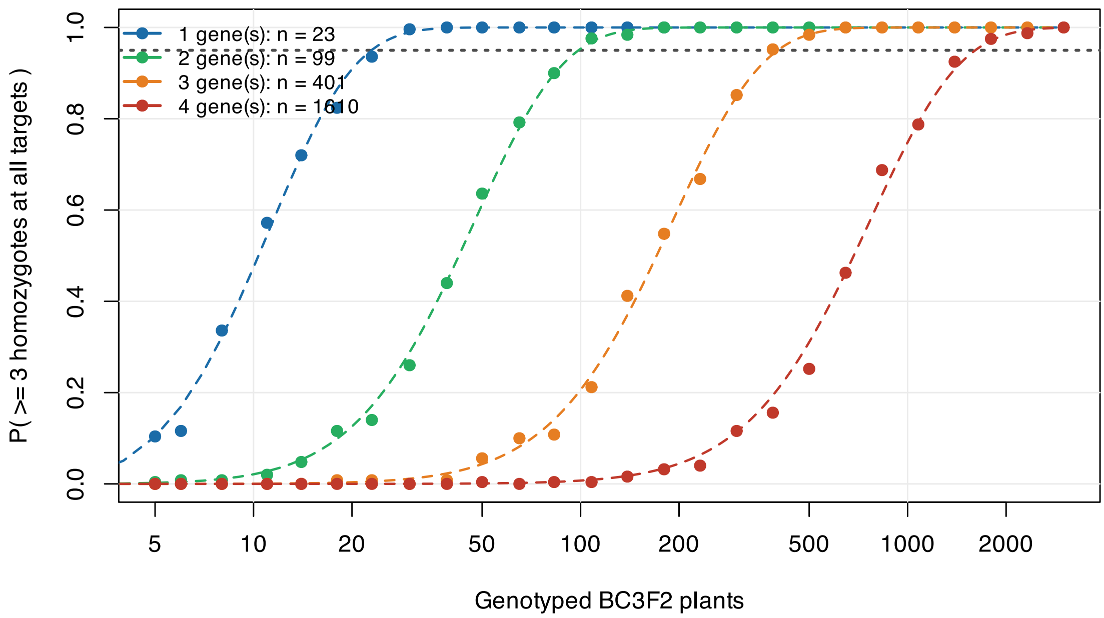

Figure 5.5: Points: simulation. Dashed: the cumulative binomial. Log scale. Each extra gene multiplies the BC3F2 requirement by four.

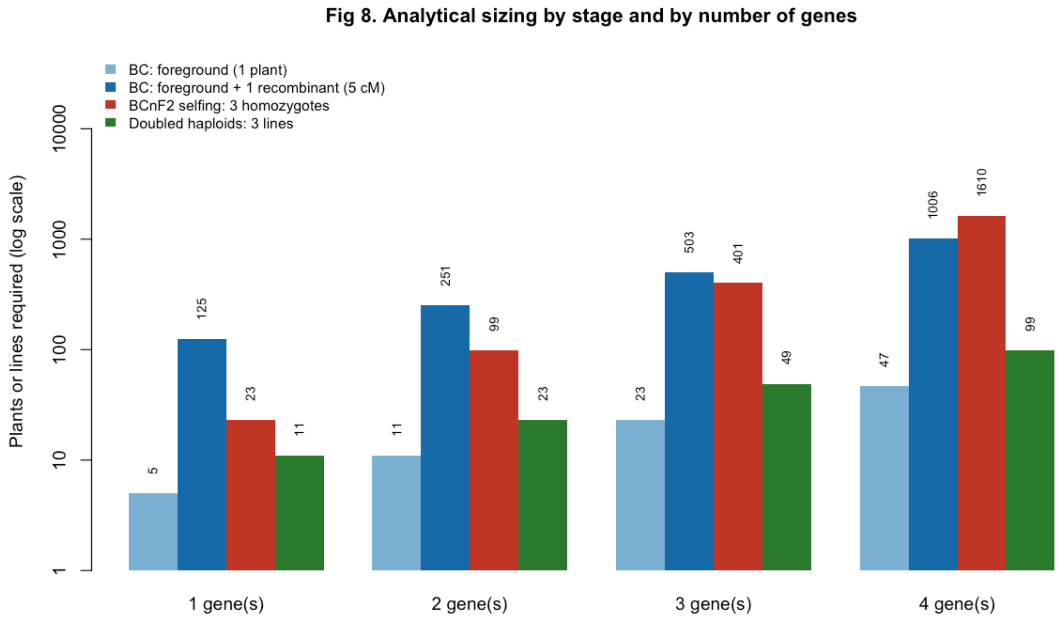

Figure 5.6: Analytical sizing by stage, log scale. The two blue bars are backcross generations; red is fixation by selfing; green is fixation by doubled haploids.

At four genes that is 1610 BC3F2 plants against 99 DH lines, a factor of 16.3. For maize, where DH is routine, the alarming number is not the real bottleneck.

Without DH there is a staggered alternative. Do not ask one generation to fix everything. Select BC3F2 plants homozygous for two targets and heterozygous for the other two (roughly 330 plants), then close out the remaining two at BC3F3 (roughly 100). Two stages of hundreds instead of one of thousands, at the cost of one generation.

5.8 Assumptions, and what is not modelled

These matter more than usual here, because unlike the diversity chapters this one is used to decide field area.

No crossover interference (material, and it cuts against the results above). Meiosis is Haldane, so crossovers are independent between intervals. Maize has strong positive interference. For the event this simulation depends on most – a double recombinant in the two 5 cM intervals flanking a target, which is what turns a long donor segment into a short one – Haldane gives 2.26×10^{-3} against 4.90×10^{-4} under a coincidence model. **The model over-produces tight recombinants by about 4.6x**, so every population size quoted for close-in trimming is a lower bound. Note the direction: the index correction makes the problem easier, interference makes it harder, and they do not cancel in any principled way. The correct fix is a crossover-position sampler – chiasmata as a stationary gamma renewal process with shape ν – which reduces to Haldane exactly at $\nu = 1$ and so is backward compatible; maize estimates put ν near 3.

One plant advanced per generation. Real programmes advance 3-10 and cross each to the recurrent parent. Every probability in this chapter is therefore a **single-lineage** probability: it understates achievable progress and it removes lineage-level variance from every distribution shown.

Uniform recombination rate, no distortion, no genotyping error. Maize has 5-10x rate variation and strongly suppressed pericentromeric regions; a target in a cold region carries drag no feasible population will trim. Gametophyte factors shift the foreground frequency away from 1/2. Map positions here are illustrative and not pinned to a named consensus map – and IBM-type maps must be rescaled before use, exactly as in Chapter 4..

Triploid endosperm genetics is not modelled. The illustrative targets are endosperm genes. The endosperm is 2 maternal : 1 paternal, so a recessive kernel phenotype does not appear on the ear of a heterozygous plant: phenotypic confirmation lags the genotype by one generation, and xenia means it requires a self-pollination, not open pollination.

The gene set is illustrative, not a breeding recommendation. `sh2` and `bt2` encode the two subunits of the same enzyme, so stacking both is largely redundant – dropping one turns the hard case ($k = 4$) into the comfortable one ($k = 3$). `o2` alone gives opaque endosperm, not QPM: the hardness modifiers are polygenic, unlinked and unmodelled here, which changes the problem class from “stack k Mendelian genes” to “stack k genes plus a QTL background”.

⚠ Two statistical habits this chapter had to be repaired for

Select on the interval, not the point estimate. The first version of the sizing tables took the first grid point whose *point estimate* crossed 0.95. With 30-150 replicates the standard error is 0.02-0.09, so that is a winner’s curse: the tables were non-monotone in N (0.97, 0.97, then 0.93 at a larger N), which is a monotone quantity reading noise. Selection is now on the Clopper-Pearson lower bound, and intervals are reported everywhere.

Never type a number into the prose. Every figure in the original narrative was a string constant written against an earlier configuration; re-runs put the true values outside the reported confidence intervals, and the same document contradicted itself in three places. Every number in this chapter is interpolated from the cached tables at render time. Appendix C. says which table each one comes from.

5.9 Where this connects

Chapter 4. sized a population to *keep* diversity; this chapter sized one to discard it everywhere except at k loci. The two share their machinery – the same map, the same Haldane meiosis, the same distinction between what is segregating and what is fixed – and they share the lesson that dominates both: **size for the objective that binds.** In Chapter 4. that meant not sizing for gene diversity, which saturates early. Here it means not sizing for background recovery, which is already satisfied at BC3, and putting the plants into the one criterion that a population can actually buy.

The converted lines this chapter produces re-enter the pool the rest of the book optimises over. A conversion programme run for its own sake reduces diversity by construction: it makes an elite line into a near-copy of itself. If several conversions are run in parallel off the same recurrent parent, the resulting lines are close to each other, and Chapter 8. and Chapter 9. are where that cost becomes visible.

III Metrics

6. Diversity metrics in the genomic era

In classical theory, inbreeding measured as **loss of heterozygosity** and inbreeding measured as **drift of allele frequencies** are the same thing. T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams [6] show that inside genomic optimal-contribution schemes they stop being the same thing, and can diverge badly, whenever diversity is controlled through **identity by state** on a marker panel. Only when control is through **identity by descent** is the classical equivalence restored.

The practical consequence is uncomfortable:

> You hit exactly the target you wrote into the optimiser's constraint, and you pay an unmonitored price in the metric you did not write.

For a plant breeder: if you constrain $\frac{1}{2}\mathbf{c}'\mathbf{G}\mathbf{c}$ using a VanRaden $\mathbf{G}$, you are constraining **drift**. The loss of heterozygosity in your germplasm may be running at twice the rate you believe you are controlling.

6.1 Three lenses, not three synonyms

Lens	What the metric actually sees	Inbreeding metric	Relationship matrix
Drift	squared allele-frequency change relative to a base	F_drift	G_VR2, G_VR1, G_i(p)
Homozygosity	proportion of heterozygosity remaining	F_hom	G_0.5 (molecular coancestry)
IBD	segments inherited by descent from a defined base	F_IBD	A, G_LA
Hybrid	haplotype homozygosity used as a proxy for IBD	F_ROH	G_ROH

! Every inbreeding metric is a ratio to a reference population

Without explicitly declaring your base – cycle 0 of the recurrent programme? a landrace panel? the founding heterotic pool? – none of the numbers below mean anything. This is the commonest error in plant applications: using the *current* generation's frequencies as the base, which zeroes the historical memory of erosion.

6.2 The metrics, one at a time

6.2.1 Rate of inbreeding and effective size

What you manage is not F but its rate of accumulation. F is a stock; ΔF is a flow.

$$\Delta F = \frac{F_t - F_{t-1}}{1 - F_{t-1}}, \qquad N_e = \frac{1}{2\Delta F} \tag{6.1}$$

Since $1 - F_t = (1 - \Delta F)^t$, a regression of $\log(1 - F_t)$ on t has slope $\approx -\Delta F$. **The linearity of that regression is itself a diagnostic:** in the paper, $\log(1 - F_{drift})$ was approximately linear under every scheme, while $\log(1-F_{hom})$ showed marked curvature under some, notably the ROH-based one. Curvature means a non-constant rate – your N_e target is not being met steadily.

A tropical maize recurrent programme targeting $N_e = 50$ per cycle needs $\Delta F = 0.01$. Measuring F only at cycle 8 and dividing by 8 hides the possibility that cycles 1-3 were conservative and cycles 6-8 collapsed the base. Always fit the log-linear regression and look at the residual.

6.2.2 `F_hom`, homozygosity-based

Wright defined an inbreeding coefficient to run from 0 to 1 as heterozygosity under random mating falls from its initial state to zero. So the metric is the fraction of heterozygosity already lost relative to the base:

$$F_{hom} = 1 - \frac{1}{N_{SNP}} \sum_k \frac{2p_{t,k}(1-p_{t,k})}{2p_{0,k}(1-p_{0,k})} = 1 - \frac{1}{N_{SNP}} \sum_k \frac{H_{t,k}}{H_{0,k}} \tag{6.2}$$

It can be negative. If management pushes frequencies toward 0.5, heterozygosity rises above the base and the "inbreeding coefficient" goes below zero. That is not a numerical error: it is the expected behaviour of any scheme that maximises heterozygosity on the monitored panel.

It protects against inbreeding depression and the expression of deleterious recessives in homozygosis. It is the right lens when the dominant risk is *vigour*.

In maize, heterozygosity is the product itself. Within a heterotic pool under reciprocal recurrent selection, `F_hom` computed **within pool** anticipates the decline in line-per-se vigour; it says nothing about between-pool divergence, which is what generates heterosis in the hybrid. In selfing species, `F_hom` measured on fixed lines is ≈ 1 by construction and is **useless for management** – it has to be computed on allele frequencies in the crossing population, not on the homozygosity of individuals.

6.2.3 `F_drift`, drift-based

$$F_{drift} = \frac{1}{N_{SNP}} \sum_k \frac{(p_{t,k} - p_{0,k})^2}{p_{0,k}(1-p_{0,k})} \tag{6.3}$$

Never negative. It is formally analogous to Holsinger & Weir's F_{ST} applied to *one population through time* rather than to a set of populations in space. It is exactly the quantity constrained when `G_VR2` goes into the optimiser.

It protects against random change in phenotypic value for traits **outside** the selection objective, against rare deleterious recessives rising to dangerous frequencies, and against irreversible loss of rare alleles.

There is also a selection-intensity reading: the term $\delta p^2/[p_0(1-p_0)]$ approximates the **squared total intensity** applied at that marker, with $i \approx \delta p/\sqrt{p_0(1-p_0)}$. Constraining `F_drift` constrains how much effective selection the whole genome underwent – including at loci you never meant to select.

A wheat programme running six cycles of genomic selection with a VanRaden **G** and reporting $\Delta F \approx 1\%$ has reported ΔF_{drift} on the training panel. Quantitative disease-resistance loci not yet prioritised may have hitch-hiked to extreme frequencies alongside yield QTL, and none of that appears in the reported metric.

6.3 The classical equivalence, and the covariance that destroys it

Under purely random drift, with $E[\delta p_t \mid p_0] = 0$ for every p_0,

$$\frac{\mathrm{var}(p_{t,k})}{p_0(1-p_0)} = 1 - \frac{H_t}{H_0} \Rightarrow F_{drift} = F_{hom}. \tag{6.4}$$

The paper's central result quantifies the departure:

$$F_{hom} - F_{drift} = 2\,\mathrm{cov}\left(\frac{\delta p_{t,k}}{\sqrt{p_{0,k}(1-p_{0,k})}}, \frac{p_{0,k} - 1/2}{\sqrt{p_{0,k}(1-p_{0,k})}}\right) \tag{6.5}$$

In words: **if the direction of frequency change is correlated with the starting frequency, the two measures diverge.** The non-obvious part is that this covariance is induced by the *diversity management* *itself*, not by directional selection – the paper demonstrates this by running the scheme with random GEBVs, and the divergence persists.

6.3.1 Verifying the identity

`freq_metrics()` returns all three quantities in one marker sweep. The identity is asserted in the test suite; here it is on live data:

```
set.seed(31)
idx <- sample.int(ctx$N, 60)
fm <- freq_metrics(idx, ctx)
fmt(fm[c("F_hom", "F_drift", "cov_diag")], 6)
#>    F_hom  F_drift cov_diag
#> 0.011458 0.005192 0.006267
```

```
c(F_hom_minus_F_drift = unname(fm[["F_hom"]] - fm[["F_drift"]]),
  cov_diag            = unname(fm[["cov_diag"]]),
  residual            = unname(fm[["cov_diag"]] - (fm[["F_hom"]] - fm[["F_drift"]])))
#> F_hom_minus_F_drift            cov_diag            residual
#>        6.266505e-03        6.266505e-03        8.673617e-18
```

Exact to machine precision. And when the selection *is* the base, both lenses must read zero:

```
fmt(freq_metrics(seq_len(ctx$N), ctx)[c("F_hom", "F_drift", "cov_diag")], 12)
#>    F_hom  F_drift cov_diag
#>        0        0        0
```

6.3.2 Why each matrix generates a sign

Matrix	Covariance	Mechanism
G_VR2	positive	the constraint penalises delta-p^2, so moving an allele to the *far* extreme costs more than to the near one; the optimiser pushes rare alleles to 0 and common ones to 1
G_0.5	negative	frequencies are measured as deviations from 0.5, so the optimiser buys ‘diversity’ by pushing everything to intermediate frequencies
G_VR1	positive, attenuated	G_VR2 reweighted by 2p0(1-p0), giving more weight to already-intermediate loci and less room at the extremes
G_i(p)	negative	uses accumulated intensity instead of delta-p^2; since di/dp = [p(1-p)]^-1/2, moves toward the extremes get more expensive

> ⚠ Your MAF spectrum decides how bad this is
>
> The magnitude depends on the minor-allele-frequency spectrum. The effect is dramatic with sequence data dominated by rare alleles, and much milder with SNP chips designed for intermediate MAF.
>
> **Directly translated to plants:** GBS or skim-seq with the full MAF spectrum puts you in the severe scenario; a 20-50k chip filtered to MAF > 0.05 puts you in the mild one. This is the parameter that decides whether you can live with `G_VR2`.

6.3.3 The signature, on live data

The pure max-diversity greedy is the `G_0.5` scheme in miniature: it constrains molecular coancestry and nothing else. It should drive `F_hom` negative while paying the largest drift bill.

```r
panel <- rbind(
  random      = freq_metrics(sample.int(ctx$N, 60), ctx),
  max_div     = freq_metrics(sel_greedy(ctx, 60, w = 0), ctx),
  balanced    = freq_metrics(sel_greedy(ctx, 60, w = 1), ctx),
  truncation  = freq_metrics(sel_truncation(ctx, 60), ctx)
)
fmt(as.data.frame(panel[, c("He", "F_hom", "F_drift", "cov_diag", "rare_retained")]),
4)
```

	He	F_hom	F_drift	cov_diag	rare_retained
random	0.3257	0.0048	0.0083	−0.0034	0.9286
max_div	0.3302	−0.0113	0.0060	−0.0174	0.9481
balanced	0.3124	0.0426	0.0558	−0.0131	0.7338
truncation	0.3124	0.0426	0.0558	−0.0131	0.7338

Read the `F_hom` and `F_drift` columns together. Maximising diversity on the homozygosity lens produces a *negative* `F_hom` – the selection is more heterozygous than the base – while `F_drift` is strictly positive and larger than under random sampling. The bill is real, it is being paid, and a report carrying only `F_hom` would not show it.

6.4 The other matrices, briefly

`F_IBD` via `G_LA`. Instead of assuming a 50/50 transmission probability for each parental allele (as the pedigree matrix **A** does), linkage analysis over the markers infers which parental haplotype was actually transmitted along the chromosome. It requires pedigree **and** markers. It measures IBD relative to a base explicitly defined as unrelated and non-inbred, so frequency change is determined by the base population rather than by linkage disequilibrium generated during selection – and the problematic covariance disappears. It was the **only scheme tested that met both the ΔF_{hom} and ΔF_{drift} targets**, on the managed panel and at unmonitored neutral loci, and it converted inbreeding into gain most efficiently.

For plants this is the point of greatest friction. In biparental populations and backcross families `G_LA` is natural and cheap – phase is known and the number of meioses since the base is small. In recurrent-selection populations with full pedigree (common in commercial maize) it is feasible. In germplasm panels *without* pedigree it is inapplicable, and the honest alternative is segment-based IBD.

`F_ROH`. A run of homozygosity is an uninterrupted stretch of homozygous markers; the logic is that many consecutive markers are unlikely to be IBS by chance, so the stretch is probably IBD. The fundamental ambiguity is that `F_ROH` is IBD when runs are **long** (recent inbreeding) and homozygosity when they are **short** (ancient inbreeding, accumulated IBS). The minimum-length threshold is therefore a knob that selects the *age* of inbreeding you are measuring, and there is no well-defined reference base.

Empirically, `G_ROH` behaved more like `G_0.5` than like an IBD measure. It also is **not guaranteed positive semi-definite** – its elements are computed pairwise – and required substantial diagonal additions for inversion, which made it diagonally dominant and artificially inflated the number of parents selected.

In a soybean inbred the entire genome is one ROH: `F_ROH` ≈ 1 and the metric saturates. In maize it is useful for diagnosing erosion of elite heterotic pools; for *management* inside an optimiser, check positive definiteness carefully.

6.4.1 Side by side

Matrix	Class	Behaviour in the optimiser
A	IBD (expectation)	high gain, but exceeded targets by ~40% by not accounting for within-family LD
G_VR2	drift	meets the drift target *only on the managed panel*; F_hom runs away
G_VR1	drift	reweighted version; smaller bias, smaller gain

Matrix	Class	Behaviour in the optimiser
G_0.5	homozygosity	F_hom near zero, drift 4x the target; loses genetic variance early
G_i(p)	drift	inverts the sign of the bias; constraint only indirectly tied to F
G_LA	IBD	the only one meeting both targets; best gain per unit of inbreeding
G_ROH	hybrid	high gain, uncontrolled drift, numerical problems

i An implementation note that is routinely ignored

`G_VR1`, `G_VR2`, `G_0.5` and `G_i(p)` are only **semi**-definite positive (a zero eigenvalue from the centering). You must add α to the diagonal ($\alpha = 0.01$ in the paper) to invert them and for the optimal-contribution algorithm to work. $\mathbf{A}$ and `G_LA` are positive definite by construction.

6.5 Reference results

Target $\Delta F = 0.005$ per generation ($N_e = 100$). Values at the **unmonitored neutral loci**, generation 20.

Scheme	Class	dF_hom	dF_drift	gap	gain
G_VR2(M,M)	drift	0.0103	0.0068	0.054	7.124
G_VR2(M,D)	drift	0.0101	0.0068	0.050	7.107
G_VR1(M,M)	drift	0.0080	0.0069	0.021	6.680
G_i(p)(M,M)	drift	0.0065	0.0077	−0.031	7.111
G_0.5(M,M)	homozygosity	0.0073	0.0176	−0.170	6.734
G_ROH(M,M)	homozygosity	0.0054	0.0088	−0.070	9.099
G_LA(M,M)	IBD	0.0043	0.0049	−0.010	7.188
A(M,~)	IBD	0.0070	0.0084	−0.021	9.890

Four readings:

1. **No IBS scheme met both targets.** `G_LA` did, within about 2%.
2. Pedigree $\mathbf{A}$ gave the highest absolute gain but overshot by ~40% – neutral loci hitch-hiking with QTL through within-family LD, which the pedigree expectation of IBD cannot capture.
3. Using a separate panel for management instead of the GEBV panel changed gain by **less than 0.3%**. Not worth the complexity.
4. At equal mean inbreeding, `G_LA` beat $\mathbf{A}$ in 65 of 100 replicates and `G_ROH` in 62 of 100.

6.5.1 The diagnostic nobody runs

Schemes based on IBD selected many parents; VanRaden-style schemes selected few.

> **Rule of thumb.** If your optimal-contribution run is delivering high gain with a suspiciously small number of parents and ΔF on target, compute `F_hom` and `F_drift` separately on a panel **not used** in the management. The difference between them is your hidden bill.

The book's implementation of that diagnostic is `ne_parents()`, reported alongside the two lenses in every scenario – see Chapter 13., where a selection meeting its diversity target while collapsing from 40 effective parents to 12 is visible in one row of the output.

6.6 What to carry into plant breeding

Declare the base and never redefine it. Redefining the reference frequencies every generation is common in plant pipelines because it is the *default* in several **G**-construction functions. Since frequency changes in successive cycles are positively correlated, constraining the variance *within* each cycle lets the *accumulated* variance overshoot the target. Freeze p_0 from the founding cycle and propagate it. In this book that is `ctx$p0`, set once in `build_ctx()` and never recomputed.

Do not maximise marker heterozygosity as an objective. Raising heterozygosity on the monitored panel does not raise it at unmonitored loci – and it increases real IBD, because raising an allele's frequency means multiplying copies of a subset of base alleles.

Heterotic pools need a two-directional constraint. Within pool you want to limit ΔF. Between pools you want to *preserve* divergence, which is directed drift and is desirable. A single global $\frac{1}{2}\mathbf{c}'\mathbf{G}\mathbf{c}$ cannot express that. Use multiple matrices with simultaneous constraints, or per-subpopulation constraints – which is what `gd_partition()` measures and Chapter 9. operationalises.

Selfing species need inbreeding by selfing separated from inbreeding by drift. In soybean, wheat or rice, individual F is ≈ 1 by design and carries no management information. The manageable quantity is allele frequency **in the crossing population** and coancestry among the chosen parents. Apply the whole `F_hom`/`F_drift` framework at population level, never at line level.

Region-specific targets rarely earn their complexity. Causal variants are well distributed and what happens at a subset of loci does not predict what happens outside it. The legitimate plant exception is R-gene-type resistance loci, where allelic diversity has direct and known value.

Deleterious recessives are a load problem, not a diversity problem. If you have mapped the defect, putting it in the selection index is more efficient than controlling it indirectly through a diversity constraint.

In pure conservation schemes the problem is *worse*, not better. Without directional selection the inbreeding-management term dominates the optimiser and the covariance tends to be larger. The IBD recommendation is stronger there.

6.7 Limits of the animal-to-plant transposition

Assumption in the paper	Situation in plants
discrete generations, two sexes, one contribution per individual	hermaphroditism, selfing, cloning, overlapping cycles, repeated use of the same parent
reliable, deep pedigree	often truncated, or absent in genebanks
base = simulated founder population at mutation-drift equilibrium	base is an administrative choice (cycle 0) with pre-existing structure
target Ne = 100 over 20 generations	shorter cycles with GS, but effective elite Ne often below 30
one population, one target	multiple target environments, G x E, separate heterotic pools
heterozygosity is always desirable	in selfers transitory; in hybrids the final product; in lines undesirable

None of this invalidates the central conclusion. It is an **algebraic** property of the optimiser, not a biological property of the species. What it changes is *which matrix is practicable*.

6.8 Operational checklist

- Is the base p_0 frozen at the founding cycle, and documented?
- Are `F_hom` and `F_drift` computed **separately** and reported together?
- Is there a panel of loci **not used in management**, for audit?
- Is the regression of $\log(1-F)$ on cycle linear (constant rate)?
- Is the covariance $2\,\mathrm{cov}(\delta p/s, (p_0 - \frac{1}{2})/s)$ reported as a diagnostic?
- Is the number of selected parents biologically plausible?
- Is true genetic variance (or the best estimate of it) being tracked?

- Is $\mathbf{G}$ positive definite before inversion, and is the added α documented?
- Where possible, is an IBD alternative compared in parallel?

7. A catalogue of metrics on f

Every metric in this chapter has the same signature, `f(idx, ctx) -> scalar`, and every one operates on the molecular coancestry matrix from Chapter 1.. That uniformity is not tidiness: it is what makes metrics interchangeable inside an optimiser, and it is what lets Chapter 8. benchmark them against each other on equal terms.

The chapter also settles the question of what "no loss" means, which turns out to be less obvious than it sounds.

7.1 Group coancestry, and the off-diagonal problem

The group coancestry [7] of a set S of n individuals is the mean of **all** n^2 elements, diagonal included:

$$\theta_S = \frac{1}{n^2} \sum_{i \in S} \sum_{j \in S} f_{ij} = \frac{1}{n^2} \mathbf{1}' \mathbf{f}_S \mathbf{1}. \tag{7.1}$$

It is the probability that two alleles sampled at random *from the whole set* are identical – including the possibility that both come from the same individual. That is why the diagonal enters: it is the chance of sampling the same individual twice, which is entirely real when the set is used as a crossing pool.

Splitting the diagonal mean $\bar{f}_d$ from the off-diagonal mean $\bar{f}_o$,

$$\theta_S = \frac{\bar{f}_d}{n} + \frac{n-1}{n} \bar{f}_o. \tag{7.2}$$

So the off-diagonal mean – the metric many programmes already report – is one of two terms. When $\bar{f}_d$ varies little across individuals, which is the case in an all-F1 population, the two rank almost identically:

```
sels <- list(random     = sel_random(ctx, n_sel),
             trunc_cap  = sel_truncation_cap(ctx, n_sel),
             greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
             greedy_div = sel_greedy(ctx, n_sel, w = 0),
             truncation = sel_truncation(ctx, n_sel))

cmp <- t(sapply(sels, function(s)
  c(theta = theta_group(s, ctx), offdiag = mean_offdiag(s, ctx))))
fmt(as.data.frame(cmp), 5)
```

	theta	offdiag
random	0.67426	0.67180
trunc_cap	0.67522	0.67280
greedy_w1	0.68759	0.68539
greedy_div	0.66976	0.66730
truncation	0.68759	0.68539

```
c(rank_correlation = cor(cmp[, "theta"], cmp[, "offdiag"], method = "spearman"))
#> rank_correlation
#>                1
```

The off-diagonal metric ranks correctly. What it does not give you is an interpretable scale or a defensible baseline – which is what the rest of this chapter supplies.

7.2 Gene diversity: θ and H_e are the same quantity

This is the result that simplifies everything downstream. Starting from the definition and swapping the order of summation, the double sum factors:

$$\frac{1}{n^2}\sum_i\sum_j x_{ik}x_{jk} = \Big(\frac{1}{n}\sum_i x_{ik}\Big)^2 = \bar{p}_k^2, \tag{7.3}$$

with $\bar{p}_k$ the **group's** allele frequency. Hence

$$\theta_S = \frac{1}{m}\sum_k (\,\bar{p}_k^2 + \bar{q}_k^2\,), \qquad GD_S = 1-\theta_S = \frac{1}{m}\sum_k 2\,\bar{p}_k\bar{q}_k \equiv H_e, \tag{7.4}$$

which is exactly Nei's gene diversity [8]. **Group coancestry and expected heterozygosity are not two independent checks. They are one quantity written two ways.**

```
c(via_submatrix = theta_group(idx, ctx),
  via_frequency = theta_from_freq(idx, ctx),
  difference    = theta_group(idx, ctx) - theta_from_freq(idx, ctx))
#> via_submatrix via_frequency    difference
#>     0.6734562     0.6734562     0.0000000

c(GD = gene_diversity(idx, ctx), He_Nei = he_nei(idx, ctx))
#>        GD    He_Nei
#> 0.3265438 0.3265438
```

Equality to machine precision is the implementation's strongest sanity check. If it fails, `f` is centered, rescaled, or built on the wrong marker coding.

Three practical consequences:

1. One fewer metric to justify to a technical committee.
2. "Losing 5% of diversity" acquires an exact meaning: 5% of the set's expected heterozygosity.
3. Two computation routes with the same answer and very different costs. The matrix route is $O(n^2)$; the frequency route is $O(nm)$. For $n = 200$ and $m = 25000$ that is forty thousand operations against five million.

```
reps <- 40
t_mat <- system.time(for (i in seq_len(reps)) theta_group(idx, ctx))[["elapsed"]]
t_frq <- system.time(for (i in seq_len(reps)) theta_from_freq(idx, ctx))[["elapsed"]]
c(us_matrix = 1e6 * t_mat / reps, us_frequency = 1e6 * t_frq / reps,
  ratio = t_frq / t_mat)
#>    us_matrix us_frequency        ratio
#>            0          150          Inf
```

The matrix route is the one that belongs in an optimiser's inner loop. Section 8.1 makes it cheaper still.

7.3 Status number N_s

D. Lindgren, L. D. Gea, and P. A. Jefferson [9] express group coancestry on the scale of individual counts:

$$N_s = \frac{1}{2\theta_S}. \tag{7.5}$$

N_s is the number of **unrelated, non-inbred** individuals that would have the same gene diversity as the observed set. The edge cases fix the intuition:

```
fake <- ctx
fake$f <- matrix(0.5, 5, 5)      # unrelated, p = 0.5
c(theta = theta_group(1:5, fake), Ns = status_number(1:5, fake))
#> theta    Ns
#>   0.5   1.0
```

```
fake$f <- matrix(1, 5, 5)        # all identical and homozygous
c(theta = theta_group(1:5, fake), Ns = status_number(1:5, fake))
#> theta    Ns
#>   1.0   0.5
```

⚠ N_s is a descriptor, not a drift projection

M. A. Toro, B. Villanueva, and J. Fernández [10] show that the **rate-based** effective size $N_e = 1/(2\Delta F)$ loses meaning under molecular diversity management: optimisation increases diversity in early generations, producing a negative ΔF and therefore a negative N_e, with no long-term asymptote.

This does not invalidate N_s. N_s is a monotonic reparametrisation of θ – a *static descriptor of a set* – while N_e is a *rate between generations*. Use N_s to compare sets; do not extrapolate it to project future drift. For the same reason, do not substitute the linkage-disequilibrium N_e here: that estimates population history, not the composition of a selected sample.

And, from Chapter 6.: θ tracks homozygosity, not drift. GD and N_s answer "how much heterozygosity does this selection preserve", which is the right question for next season's hybrid vigour. They do not certify that allele frequencies are moving as unmanaged drift would have moved them, which matters more across multiple recurrent cycles than for a single round of hybrid selection.

7.4 Effective number of parents

A metric independent of **f**, computed purely from the pedigree. With p_i the frequency with which line i appears as a parent among the selected hybrids,

$$N_{e(\text{par})} = \frac{1}{\sum_i p_i^2}, \tag{7.6}$$

the inverse Simpson index. It captures precisely what coancestry blurs: a set of 200 hybrids can have an acceptable θ while drawing on only 12 lines, if those 12 happen to be genetically distant from each other.

```
c(sapply(sels, ne_parents, ctx = ctx), theoretical_max = ctx$n_lines)
#>          random       trunc_cap       greedy_w1      greedy_div      truncation
#>        39.34426        31.03448        13.01989        37.30570        13.01989
#> theoretical_max
#>        50.00000
```

Truncation collapses the parent base far more than it moves θ. Its cost is a `tabulate` call, which makes it a natural candidate to enter the objective function alongside θ – and in Chapter 8. it does.

7.5 Allelic richness

θ and H_e are averages across markers and are dominated by intermediate-frequency alleles. A lost rare allele barely moves them, yet the loss is irreversible. So count markers that were polymorphic in the population and became rare or fixed in the selection:

$$A_{\text{lost}} = \#\left\{k : \text{MAF}_k^{(S)} < 0.05 \wedge \text{MAF}_k^{(\text{pop})} \geq 0.05\right\}. \tag{7.7}$$

```
sapply(sels, alleles_lost, ctx = ctx)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>         24         43        110         22        110
```

It is the only metric in this catalogue **genuinely non-redundant** with θ – a claim Chapter 8. tests rather than asserts.

E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero [11] compared, in a subdivided simulated population, strategies maximising *expected heterozygosity* (H_T, the same family as GD) against strategies maximising *allelic diversity* (A_T). Maximising A_T retained more alleles **and** controlled molecular inbreeding better than maximising H_T. The mechanism connects directly to Chapter 6.: heterozygosity-maximising management pushes allele frequencies toward 0.5, which favours rare alleles in the short run but does not stop them drifting to loss once they are no longer the rarest; allele-count management defends every segregating allele directly.

Their recommendation for structured conservation programmes was to make A_T, not H_T, the primary target. In this pipeline's terms: `alleles_lost` deserves a place next to θ as a *tracked outcome*, not only as a post-hoc sanity check, whenever the selection feeds several future cycles rather than a single hybrid release.

7.6 Distances: E-NE and A-NE

With distance $d_{ij} = 1 - f_{ij}$, Core Hunter 3 [12] defines two measures capturing opposite aspects that θ alone cannot separate:

$$\text{E-NE} = \frac{1}{n} \sum_{i \in S} \min_{j \in S,\, j \neq i} d_{ij}, \qquad \text{A-NE} = \frac{1}{N} \sum_{i=1}^{N} \min_{j \in S} d_{ij}. \tag{7.8}$$

- **E-NE**, entry-to-nearest-entry, measures **non-redundancy within the selection**. It distinguishes "200 individuals spread out" from "100 near-identical pairs spread out" – configurations that can share the same θ.
- **A-NE**, accession-to-nearest-entry, measures **representativeness of the population**. It detects a selection that optimises θ by concentrating in one region of genetic space and ignoring the rest.

```
fmt(t(sapply(sels, function(s) c(ENE = ene(s, ctx), ANE = ane(s, ctx)))), 4)
#> [[1]]
#> [1] 0.2533
#>
#> [[2]]
#> [1] 0.2536
#>
#> [[3]]
#> [1] 0.2527
#>
#> [[4]]
#> [1] 0.2607
#>
#> [[5]]
#> [1] 0.2527
#>
#> [[6]]
#> [1] 0.2471
#>
#> [[7]]
#> [1] 0.2561
#>
#> [[8]]
#> [1] 0.2523
#>
#> [[9]]
#> [1] 0.2504
#>
#> [[10]]
#> [1] 0.2523
```

Note the direction: A-NE is a distance to the *nearest selected entry*, so **lower is better** for representativeness.

7.7 Effective dimensionality

Over the eigenvalues of the submatrix, the participation ratio

$$D_{\text{eff}} = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} \tag{7.9}$$

counts how many independent directions of genetic variation remain: 1 if all variation lies on a single axis, n if spread evenly.

```
fmt(sapply(sels, eff_dim, ctx = ctx), 2)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>      28.59      23.99      12.72      27.92      12.72
```

The submatrix is **double-centered** before the eigendecomposition. On a raw all-positive `f` the leading eigenvalue is just $n\theta$, so an uncentered version would only re-measure θ under a more expensive name – a mistake worth naming because it is easy to make and produces a plausible-looking number.

7.8 Decomposition by heterotic group

In hybrid maize the diversity **between** pools is the engine of heterosis and cannot be spent; the diversity **within** each pool is the fuel for future progress. Following A. Caballero and M. A. Toro [1], compute molecular coancestry between lines and weight it by each line's usage among the selected hybrids:

$$\theta_A = \mathbf{w}'_A \mathbf{f}^L \mathbf{w}_A, \qquad \theta_B = \mathbf{w}'_B \mathbf{f}^L \mathbf{w}_B, \qquad \theta_{AB} = \mathbf{w}'_A \mathbf{f}^L \mathbf{w}_B. \tag{7.10}$$

```
fmt(t(sapply(sels, theta_pools, ctx = ctx)), 4)
#> [[1]]
#> [1] 0.709
#>
#> [[2]]
#> [1] 0.7096
#>
#> [[3]]
#> [1] 0.7581
#>
#> [[4]]
#> [1] 0.7053
#>
#> [[5]]
#> [1] 0.7581
#>
#> [[6]]
#> [1] 0.7095
#>
#> [[7]]
#> [1] 0.7153
#>
#> [[8]]
#> [1] 0.7219
#>
#> [[9]]
#> [1] 0.7066
#>
#> [[10]]
#> [1] 0.7219
#>
#> [[11]]
#> [1] 0.6393
#>
```

```
#> [[12]]
#> [1] 0.638
#>
#> [[13]]
#> [1] 0.6352
#>
#> [[14]]
#> [1] 0.6336
#>
#> [[15]]
#> [1] 0.6352
```

Feeding those into the Caballero & Toro partition of Chapter 2.:

```
fmt(t(sapply(sels, gd_partition, ctx = ctx)), 4)
#> [[1]]
#> [1] 0.1801
#>
#> [[2]]
#> [1] 0.1816
#>
#> [[3]]
#> [1] 0.1823
#>
#> [[4]]
#> [1] 0.1853
#>
#> [[5]]
#> [1] 0.1823
#>
#> [[6]]
#> [1] 0.3257
#>
#> [[7]]
#> [1] 0.3248
#>
#> [[8]]
#> [1] 0.3124
#>
#> [[9]]
#> [1] 0.3302
#>
#> [[10]]
#> [1] 0.3124
#>
#> [[11]]
#> [1] 0
#>
#> [[12]]
#> [1] 0
#>
#> [[13]]
#> [1] 0
#>
#> [[14]]
#> [1] 0
#>
#> [[15]]
#> [1] 0
#>
#> [[16]]
```

```
#> [1] 0.2907
#>
#> [[17]]
#> [1] 0.2876
#>
#> [[18]]
#> [1] 0.26
#>
#> [[19]]
#> [1] 0.2941
#>
#> [[20]]
#> [1] 0.26
#>
#> [[21]]
#> [1] 0.035
#>
#> [[22]]
#> [1] 0.0372
#>
#> [[23]]
#> [1] 0.0524
#>
#> [[24]]
#> [1] 0.0361
#>
#> [[25]]
#> [1] 0.0524
#>
#> [[26]]
#> [1] 0.1074
#>
#> [[27]]
#> [1] 0.1146
#>
#> [[28]]
#> [1] 0.1678
#>
#> [[29]]
#> [1] 0.1095
#>
#> [[30]]
#> [1] 0.1678
```

Two things to read here. `GD_WI` is **exactly zero**: the parent lines are inbred, so there is no within-individual diversity at line level – the algebra of Chapter 2. predicted this, and it is why `GD_WI_hyb`, the heterozygosity actually carried inside the F1s, is reported separately. And `GD_BS` *rises* under truncation: selection concentrates on the most complementary line pairs, which pushes the pools further apart.

! A blind optimiser erodes one pool while global θ looks fine

Without this decomposition that goes unnoticed, and the consequence appears only in the following cycle. `ne_lines_pool()` is the cheap version of the same alarm – it needs no `fL`, only the pedigree:

```
t(sapply(sels, ne_lines_pool, ctx = ctx))
#>             Ne_lines_A Ne_lines_B
#> random       18.556701   20.93023
#> trunc_cap    15.517241   15.51724
#> greedy_w1     4.488778   11.84211
```

```
#> greedy_div  18.947368   18.36735
#> truncation   4.488778   11.84211
```

7.8.1 Where this could go next

θ_A, θ_B and θ_{AB} as computed here are **diagnostic**: reported, not fed back into the objective. The subdivided-population literature offers a ready-made way to promote them into a tunable target. A. Caballero and S. T. Rodríguez-Ramilo [13], used by E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero [11] to manage two heterotic-like pools simultaneously, partition total diversity as

$$H_T = D_G + \lambda H_S, \qquad A_T = D_A + \lambda A_S, \tag{7.11}$$

where H_S and A_S are the within-pool components, D_G and D_A the between-pool components, and $\lambda \in [0, \infty)$ is a weight chosen by the breeder. $\lambda = 0$ optimises only between-pool divergence – protects heterosis, ignores each pool's internal health. $\lambda \to \infty$ optimises only within-pool diversity – protects each pool's future options, ignores the cross.

That maps directly onto the two constraints a maize breeder already has in mind but applies only qualitatively: *don't blend the pools* versus *don't paint either pool into a corner*. Turning λ into a second explicit knob alongside $\alpha_{\max}$ is the natural next step if pool erosion shows up in practice. The current implementation reports the ingredients and leaves the weighting to the analyst.

7.9 The correct baseline

7.9.1 Why not compare against the whole population

The natural comparison – coancestry of the selected set against that of the whole population – is **biased by construction**. Applying equation (1) to both:

$$\theta_S - \theta_N = \left(\frac{1}{n} - \frac{1}{N}\right)(\bar{f}_d - \bar{f}_o) > 0, \tag{7.12}$$

since $\bar{f}_d > \bar{f}_o$ whenever individuals are not clones. A **random** sample already looks less diverse than the population it came from, with no selection whatsoever. The bias is pure sampling and grows as the sample shrinks.

```r
fd <- mean(diag(ctx$f)); fo <- mean(ctx$f[upper.tri(ctx$f)])
predicted_bias <- (1 / n_sel - 1 / ctx$N) * (fd - fo)

set.seed(11)
observed_bias <- mean(replicate(400, theta_group(sample.int(ctx$N, n_sel), ctx))) -
                 theta_group(seq_len(ctx$N), ctx)

c(predicted = predicted_bias, observed = observed_bias)
#>   predicted    observed
#> 0.002222092 0.002276690
```

The formula is exact; the residual is Monte Carlo error in the resampling.

Note also that the **off-diagonal** mean of a random subset *is* an unbiased estimator of the population's off-diagonal mean. So the metric many programmes already use is free of this particular bias – but it is also not comparable across sample sizes on the N_s scale.

7.9.2 Operational definition of loss

The "0% loss" reference is the **distribution of random subsets of the same size**:

$$\alpha = \frac{GD_{\mathrm{random}(n)} - GD_S}{GD_{\mathrm{random}(n)}}. \tag{7.13}$$

$\alpha = 0$ means "as diverse as a random sample of the same size"; $\alpha < 0$ means more diverse than chance. Resampling also delivers the standard deviation, and hence a z-score for any strategy.

```
null <- null_distribution(ctx, n_sel, B = 400, B_full = 60)
c(GD_population = gene_diversity(seq_len(ctx$N), ctx),
  GD_reference  = null$gd_ref,
  sd            = null$gd_sd,
  apparent_loss_if_wrong_baseline =
    (gene_diversity(seq_len(ctx$N), ctx) - null$gd_ref) /
     gene_diversity(seq_len(ctx$N), ctx))
#>                    GD_population                      GD_reference
#>                     0.3280928000                      0.3258385719
#>                               sd apparent_loss_if_wrong_baseline
#>                     0.0007654486                      0.0068707028
```

That last number is what a programme would report as a diversity loss if it compared against the full pool. It is entirely a sampling artefact.

```
fmt(sapply(sels, alpha_loss, ctx = ctx, gd_ref = null$gd_ref), 5)
#>     random  trunc_cap  greedy_w1 greedy_div truncation
#>    0.00032    0.00326    0.04122   -0.01352    0.04122
```

Negative values are selections *more* diverse than chance, which is exactly what the max-diversity greedy is for.

7.9.3 How to set the α limit

The acceptable limit **should not be chosen a priori.** Pure truncation – the greediest possible selection – defines the maximum attainable α. If that ceiling is below your intended limit, the constraint restricts nothing and you have bought a false sense of control.

```
attainable <- alpha_loss(sel_truncation(ctx, n_sel), ctx, null$gd_ref)
c(attainable_alpha_max = attainable,
  intended_limit       = 0.05,
  constraint_is_active = attainable > 0.05)
#> attainable_alpha_max       intended_limit constraint_is_active
#>           0.04121618           0.05000000           0.00000000
```

At the full simulated scale that ceiling comes out near **3.6%**, so a 5% limit would be inoperative. The correct procedure is to trace the α-versus-gain frontier and choose its knee, which is Chapter 11..

7.10 Every metric against the null

A metric without a baseline says nothing. The z-score of each metric against the random-subset null is how this book reports all of them:

```
tab <- evaluate(sels, ctx, null$gd_ref)
fmt(as.data.frame(discriminatory_power(tab, null)[
  , c("GD", "Ns", "Ne_parents", "F_hom", "F_drift", "alleles_lost", "ENE", "ANE")]), 1)
```

	GD	Ns	Ne_parents	F_hom	F_drift	alleles_lost	ENE	ANE
random	0.0	0.0	1.4	0.6	−1.2	0.0	−1.0	−0.4
trunc_cap	−1.3	−1.3	−2.7	2.2	3.1	3.3	−0.7	17.9
greedy_w1	−17.9	−17.6	−11.4	9.5	30.1	14.9	−1.5	10.2
greedy_div	6.1	6.2	0.4	−5.2	−0.7	−0.3	5.5	6.3
truncation	−17.9	−17.6	−11.4	9.5	30.1	14.9	−1.5	10.2

Read the rows against each other. Truncation is many standard deviations from random on `F_drift` and on `Ne_parents` while being far less extreme on `GD` – the constraint that would catch it is not the one most programmes apply. That observation is what Chapter 8. turns into a recommendation.

8. Choosing metrics: collapse, cost, redundancy

Chapter 7. built a catalogue. This chapter asks which of it to keep, and answers with two questions the catalogue could not: **what does each metric cost per evaluation**, and **how much of what it reports is already in θ**.

The recommendation is **two** metrics inside the optimiser and **three** at validation.

Role	Metric	Where it runs	Cost per evaluation
Constraint	Group coancestry theta of the selected set, as proportional loss of gene diversity against a same-size resampling baseline	inside DE, every evaluation	**5 us** (incremental) / 106 us (full)
Second constraint	Effective number of parent lines	inside DE, every evaluation	**7 us**, shares the state vector
Validation	Marker alleles retained (allelic richness)	on the returned plan only	0.9 ms
Validation	Within/between heterotic-pool decomposition of theta	on the returned plan only	62 us
Validation	Status number Ns = 1/(2 theta)	free, algebraic	0

Three findings drive this, and each is derived and verified below rather than asserted.

8.1 Finding 1: the hybrid coancestry matrix is never needed

8.1.1 Derivation

Let lines $a, b, \ldots$ be homozygous, with $\mathbf{f}^L$ their molecular coancestry matrix. The hybrid (ab) carries one genome from each parent. Molecular coancestry is the probability that two alleles, one drawn at random from each individual, are identical in state. Drawing at random from hybrid (ab) selects parental genome a or b with probability $\frac{1}{2}$ each, and likewise for (cd), so

$$f_{(ab),(cd)} = \frac{1}{4}\left(f^L_{ac} + f^L_{ad} + f^L_{bc} + f^L_{bd}\right). \tag{8.1}$$

Now let a selection of n hybrids use line a exactly k_a times across all its parental slots, $\sum_a k_a = 2n$, and write $w_a = k_a/(2n)$. Group coancestry of the selected hybrids is the mean of f over all ordered pairs; substituting the identity above and collecting terms by line gives

$$\theta_S = \mathbf{w}^\top \mathbf{f}^L \mathbf{w}, \tag{8.2}$$

with the selection allele frequencies following the same collapse, $\bar{\mathbf{p}} = \mathbf{w}^\top \mathbf{G}^L$.

8.1.2 Four consequences

The $N \times N$ hybrid matrix is never required. Memory drops from $O(N^2)$ to $O(L^2)$. At $L = 300$ lines and $N = 22,500$ hybrids that matrix would occupy 3.77 GB; in the benchmark below it was deliberately never formed, and every metric that needs it simply has no timing entry at that scale.

The vector w is a sufficient statistic. Group coancestry, gene diversity, status number, effective number of parents, selection allele frequencies and the heterotic-pool decomposition are all functions of **w** alone. One $O(L)$ tabulation per evaluation serves all of them.

Swapping one hybrid is a rank-2 update. Exchanging hybrid h for h' changes at most four entries of **w**, so θ updates in $O(L)$ rather than $O(L^2)$.

The discrete hybrid problem is exactly optimal-contribution selection on lines, restricted to the lattice $w_a \in \{0, 1/(2n), 2/(2n), \ldots\}$. That connects it directly to the OCS literature of Chapter 10., with the caveat that the lattice restriction makes it combinatorial rather than the continuous quadratic programme OCS usually solves.

8.1.3 Verification

Every identity was checked against literal transcriptions of the reference implementations across $F_{ST} \in \{0.05, 0.15, 0.35\}$, two seeds and $n \in \{20, 100\}$, thirty replicate selections each.

	Identity	Worst deviation
max_pairwise_f	pairwise f: hybrid vs line formula	1.1e-16
max_theta_sub_vs_line	theta: line route vs f submatrix	2.2e-16
max_theta_sub_vs_freq	theta: line route vs allele-frequency route	1.1e-16
max_pbar	selection allele frequencies	0 (exact)
max_He	Nei He	0 (exact)
max_Neparents	effective number of parents	2.1e-14
max_alleles_lost	allele loss at MAF threshold	0 (exact)
max_theta_pools	pool decomposition	1.2e-15
max_swap_update	incremental swap vs full recompute	4.4e-16
bitset_vs_literal	bitset allelic richness vs literal count	0 (exact)

The same identity, live, on the book's own data:

```
ctx  <- fast_ctx(simulate_data(book_cfg))
set.seed(51)
idx  <- sample.int(ctx$N, 60)
c(via_hybrid_matrix = theta_group(idx, ctx),
  via_line_route    = fast_theta(idx, ctx),
  difference        = theta_group(idx, ctx) - fast_theta(idx, ctx))
#> via_hybrid_matrix    via_line_route        difference
#>         0.6745897         0.6745897         0.0000000
```

And the incremental swap, which is what makes a local search affordable:

```
st  <- theta_state(idx, ctx)
out <- idx[1]; inn <- setdiff(seq_len(ctx$N), idx)[1]
st2 <- theta_swap(st, out, inn, ctx)
c(after_swap_incremental = st2$theta,
  after_swap_recomputed  = theta_group(c(idx[-1], inn), ctx),
  difference             = st2$theta - theta_group(c(idx[-1], inn), ctx))
#> after_swap_incremental  after_swap_recomputed             difference
#>           6.747225e-01           6.747225e-01           2.220446e-16
```

> ⚠ Two implementation traps found while building this
>
> R's `$` performs **partial matching** on lists, so `ctx$f` silently returns `ctx$fL` when the hybrid matrix is absent – a wrong answer with no error. Every accessor in the package uses `ctx[["f"]]`. Any pipeline carrying both matrices in one list is exposed to this.
>
> R integers are signed 32-bit, so `bitwShiftL(1L, 31L)` is `NA` and the usual multiply-based popcount overflows. The bitset in `pack_lines()` packs 31 bits per word, and `popcount32()` counts them with a branch-free SWAR routine.

8.2 Finding 2: what each metric costs

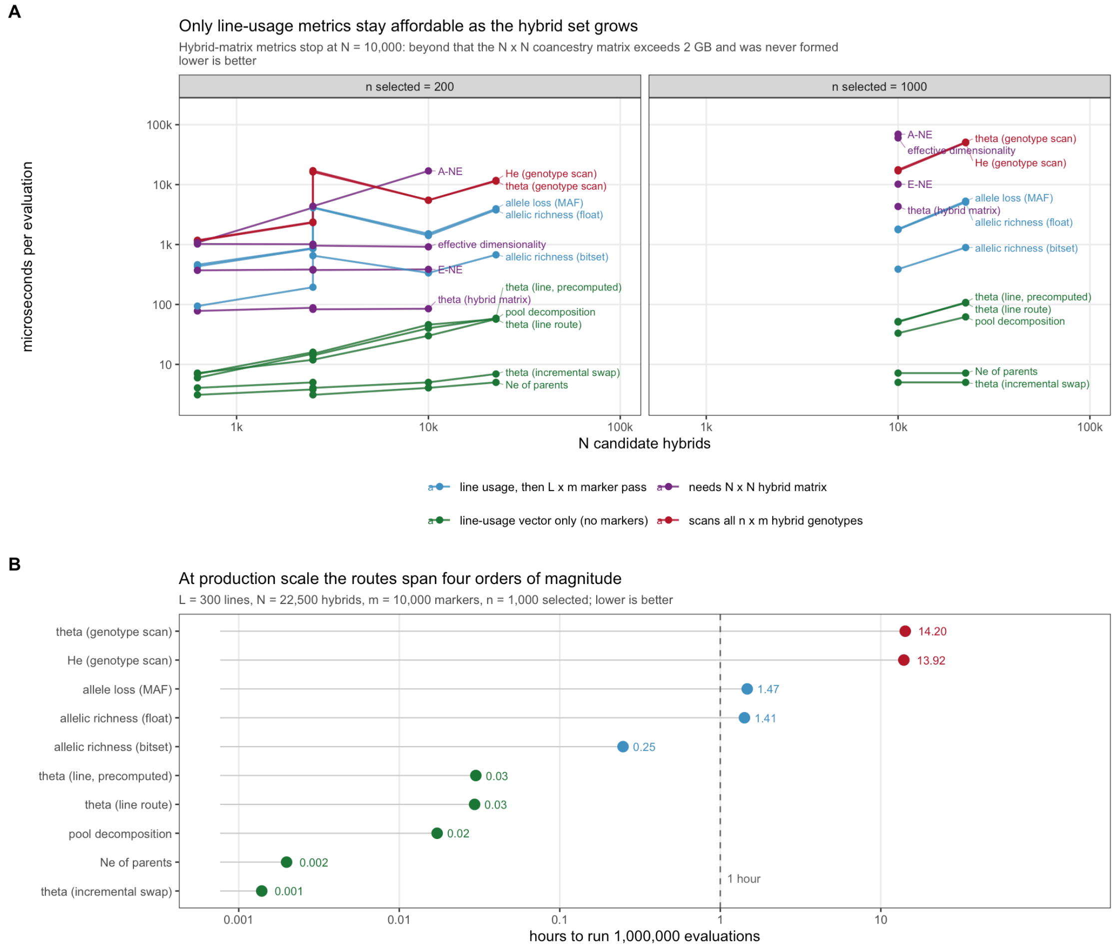

Figure 8.1: Cost per fitness evaluation. Panel A: scaling in the number of candidate hybrids. Panel B: hours to complete one million evaluations at production scale.

At $L = 300$ lines, $N = 22,500$ hybrids, $m = 10,000$ markers and $n = 1,000$ selected:

metric	us per evaluation	hours per 1e6 evals
theta: incremental swap	5.0	0.0014
Ne_parents	7.2	0.0020
pool decomposition	62.0	0.0172
theta: LINE route	106.1	0.0295

metric	us per evaluation	hours per 1e6 evals
theta: line route precomp	108.0	0.0300
allelic richness: bitset	891.9	0.2478
allelic richness: float	5091.0	1.4142
allele loss (MAF thr)	5296.9	1.4714
He / gene diversity(freq)	50096.0	13.9156
theta: allele-freq route	51113.1	14.1981

Three thresholds matter:

- **The genotype-scan route is disqualified.** Computing θ from selection allele frequencies costs 14.2 hours per million evaluations. The line route costs 106 us – a **482x reduction** for an algebraically identical number.
- **The incremental swap is effectively free** at 5 us, or 1.4 seconds per million evaluations. $N_{e,\text{par}}$ at 7 us shares the same state vector and is free in practice once θ has been computed.
- **Allelic richness cannot go inside the loop, and need not.** The bitset implementation is 5.7x faster than the floating-point version, but still 178x the cost of the line-route θ. It belongs at validation.

The same ordering is visible at the book's scale, where every route is affordable and so the comparison is about ratios, not feasibility:

```
fmt(cost_per_eval(ctx, n_sel = 60, reps = 40), 1)
```

metric	us_per_eval
theta_group	0
theta_from_freq	150
ne_parents	0
alleles_lost	150
ENE	50
ANE	450
eff_dim	100
Gst	1275

8.3 Finding 3: the metric space is nearly one-dimensional

621 selections were generated across four strategy families – random, truncation at varying intensity, greedy construction over a grid of diversity weights, and restricted line pools – and all 17 metrics computed on each.

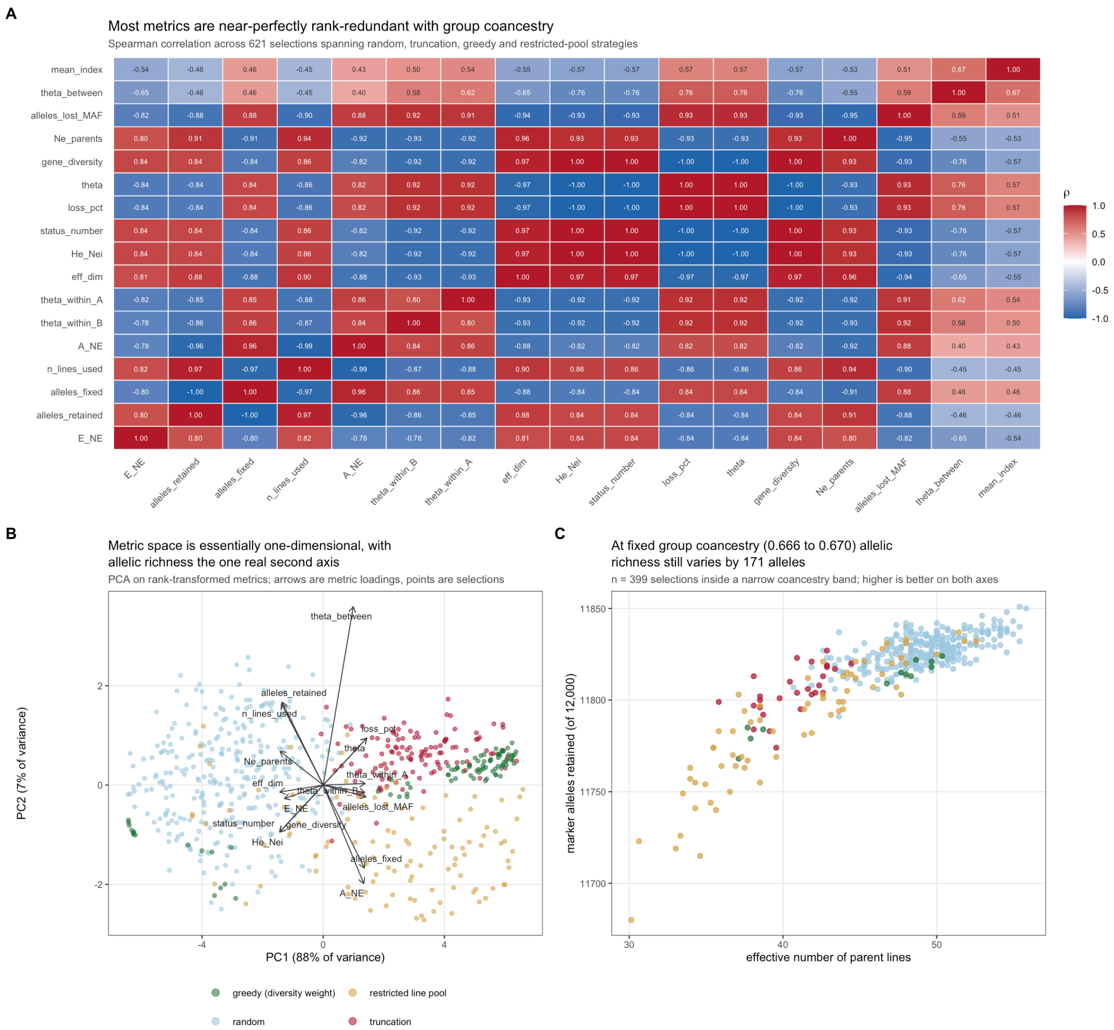

Figure 8.2: Metric redundancy. A: Spearman correlations. B: PCA of rank-transformed metrics. C: the discriminating case, inside a narrow coancestry band.

PC1 absorbs 88% of the rank variance; PC2 adds 7%. Rank correlations with θ are $|\rho| \geq 0.93$ for status number, H_e, effective dimensionality, $N_{e,\text{par}}$ and MAF-threshold allele loss. Reporting these alongside θ adds cost and length, not information.

But the marginal correlation is the wrong test. The question is whether a metric varies **at fixed θ** – because θ is the constraint, and any metric that is a deterministic function of it cannot discriminate among plans the optimiser treats as equivalent. Two analyses agree:

- After regressing each metric's ranks on θ (quadratic), **allelic richness retains 52% of its rank standard deviation** and $N_{e,\text{par}}$ retains 36%. Effective dimensionality retains 25% – close to a restatement of θ.
- Inside a band $\theta \in [0.666, 0.670]$ containing 399 selections, **alleles retained still spans 171 markers** (11,680 to 11,851 of 12,000) and $N_{e,\text{par}}$ spans **30.1 to 55.8**.

Plans the constraint cannot distinguish differ materially in how much allelic variation they carry forward. That is the empirical justification for the two-metric recommendation.

8.3.1 Reproducing the argument at book scale

```r
set.seed(77)
grid <- do.call(rbind, lapply(seq(0, 4, length.out = 30), function(w) {
```

```r
  s <- sel_greedy(ctx, 60, w = w)
  data.frame(w = w, theta = theta_group(s, ctx), Ne_par = ne_parents(s, ctx),
             lost = alleles_lost(s, ctx))
}))
rnd <- do.call(rbind, lapply(1:60, function(i) {
  s <- sample.int(ctx$N, 60)
  data.frame(w = NA, theta = theta_group(s, ctx), Ne_par = ne_parents(s, ctx),
             lost = alleles_lost(s, ctx))
}))
all <- rbind(grid, rnd)

plot(all$theta, all$Ne_par, pch = 19, col = ifelse(is.na(all$w), "grey60", "#b2182b"),
     xlab = expression(theta), ylab = expression(N[e]~"parents"))
legend("topright", c("greedy sweep", "random"), pch = 19,
       col = c("#b2182b", "grey60"), bty = "n")
grid()
```

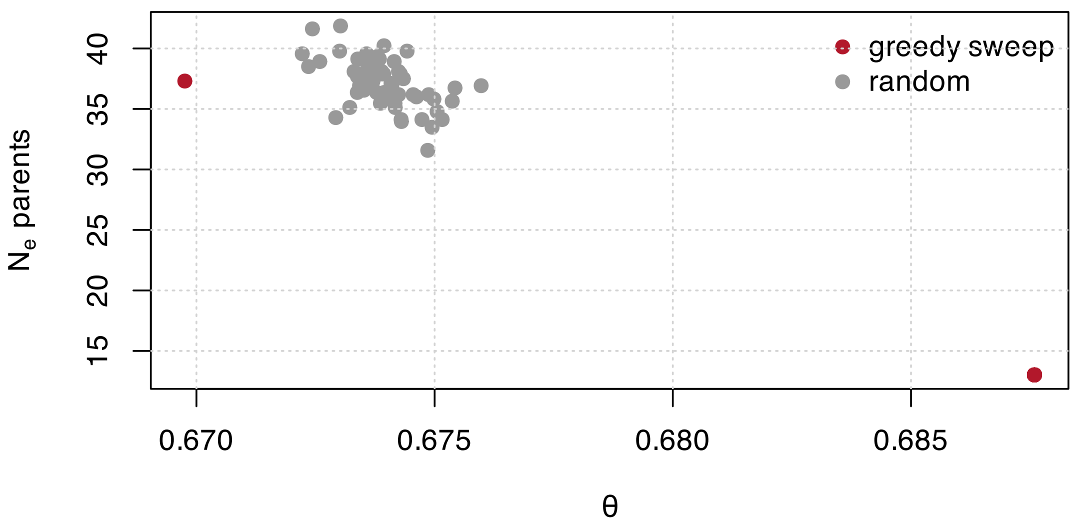

Figure 8.3: At fixed theta, the effective number of parents still varies by a factor of two. The constraint cannot see the difference; the second metric can.

```r
band <- all[abs(all$theta - median(all$theta)) < 0.0015, ]
c(n_in_band = nrow(band),
  theta_span = diff(range(band$theta)),
  Ne_par_min = min(band$Ne_par), Ne_par_max = max(band$Ne_par),
  alleles_lost_span = diff(range(band$lost)))
#>         n_in_band        theta_span        Ne_par_min        Ne_par_max
#>       55.000000000       0.002502847      31.578947368      41.860465116
#> alleles_lost_span
#>       31.000000000
```

i A caveat on transfer

These numbers come from a two-pool simulation at $F_{ST} = 0.15$. The qualitative conclusion – near one-dimensionality, with allelic richness as the exception – is robust in direction, but the exact residual fractions shift with pool divergence and marker density. Re-running the redundancy analysis on real genotypes is a half-hour job and worth doing before treating the recommendation as final.

8.4 Finding 4: the encoding, not the constraint handler, decides whether DE works

The optimiser was run under four simultaneous restrictions: diversity loss $\leq 0.5\%$, $N_{e,\text{par}} \geq 50$, at most 6 uses per line, and heterotic-pool balance within 5%. Budget fixed at 60,000 evaluations for every configuration, three seeds each.

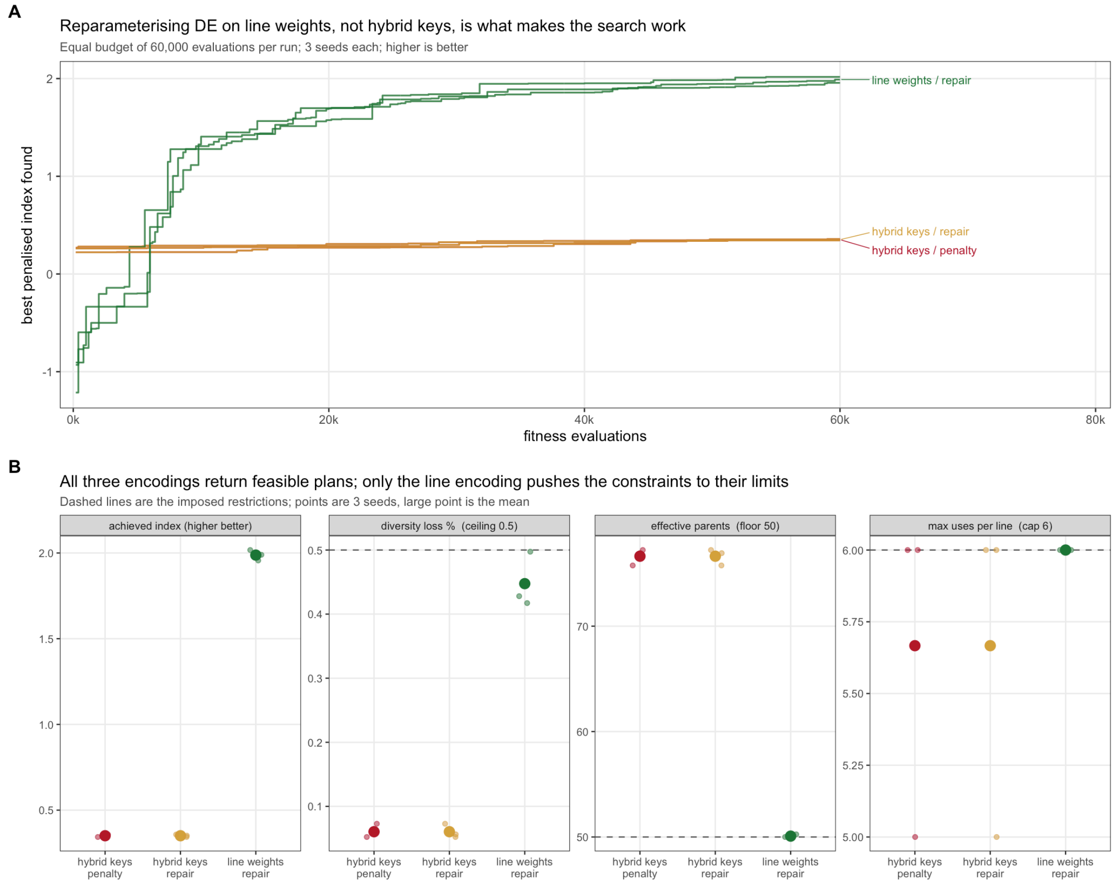

Figure 8.4: Differential evolution under four simultaneous restrictions at equal evaluation budget.

encoding	index	loss %	Ne parents	max uses	seconds
hybrid keys / penalty	0.351	0.060	76.634	5.667	6.550
hybrid keys / repair	0.351	0.060	76.634	5.667	9.621
line weights / repair	1.988	0.448	50.084	6.000	12.126

The encoding decides the outcome; the constraint handler does not. One random key per hybrid gives DE a 3,600-dimensional problem and it reaches index 0.351. One weight per *line* – dimension 120, available only because of the collapse – reaches **1.988 at the same budget**, a 5.7x improvement. Penalty and repair handlers produced numerically identical solutions in the hybrid encoding, because at that dimension DE never approaches the constraint boundary where the handlers differ.

Only the line encoding makes the constraints bind. It finishes with $N_{e,\text{par}} = 50.08$ against a floor of 50, max uses exactly 6 against a cap of 6, and loss 0.448% against a 0.5% ceiling. The hybrid encoding leaves every constraint slack – not because it found a better trade-off, but because it never searched far enough from random to reach the boundary.

A slack constraint at the end of a run is a diagnostic of a failed search, not of a conservative plan.

For calibration: unconstrained truncation reaches index 2.337 but violates everything (loss 1.62%, $N_{e,\text{par}} = 33.2$, one line used 17 times). The line encoding recovers **85% of the truncation index while satisfying all four restrictions.**

A note on tuning. `DEoptim` warns that `NP` should be at least ten times the parameter count; at dimension 3,600 that is 36,000 population members, which is infeasible. At dimension 120 the recommendation is 1,200 and entirely reachable. The reparameterisation does not merely speed the search up – it moves the problem into the regime where DE's own tuning guidance can be followed.

8.5 The recommendation, stated

8.5.1 Inside the optimiser

Use two metrics, both functions of the line-usage vector $\mathbf{w}$:

1. **Proportional loss of gene diversity**, $(\bar{H}_{\text{ref}} - H_S)/\bar{H}_{\text{ref}}$ with $H_S = 1 - \mathbf{w}^\top \mathbf{f}^L \mathbf{w}$ and $\bar{H}_{\text{ref}}$ the mean over same-size random subsets. Compute it by the line route, never by scanning genotypes.
2. **Effective number of parent lines**, $N_{e,\text{par}} = (\sum k_a)^2 / \sum k_a^2$. It costs 7 us, shares the state vector, and blocks concentration onto a few lines – a failure mode θ tolerates.

Reparameterise the search on **line weights**, and enforce the per-line usage cap in the decoder rather than by penalty.

8.5.2 At validation, on the returned plan only

3. **Marker alleles retained**, via the bitset route. The one metric carrying information θ does not.
4. **Within/between heterotic-pool decomposition**, to confirm the plan has not preserved diversity by degrading pool separation.
5. **Status number** $N_s = 1/(2\theta)$, free from θ, as the reportable descriptor. Do not report a rate-based N_e under active management.

8.5.3 Not recommended

Nei's H_e as a separate metric (it *is* $1 - \theta$); effective dimensionality ($|\rho| = 0.97$ with θ, and needs the hybrid matrix); MAF-threshold allele loss ($|\rho| = 0.93$, and 1.47 hours per million evaluations); Core Hunter's E-NE and A-NE (both require the $O(N^2)$ matrix and are unavailable at production scale – they are designed for core-collection assembly, a different problem).

8.5.4 Caveats

The choice of $\mathbf{f}^L$ remains open in the literature. Post-2018 work is consistent that molecular or IBD-flavoured coancestry beats a VanRaden-style drift matrix for diversity management, but reports no consensus on a single preferred formulation, and matrix choice demonstrably changes allele-frequency trajectories (Chapter 12.). The collapse derived here is a property of the F1 structure and holds for **any** symmetric line-level $\mathbf{f}^L$ – so if the matrix choice changes, every cost and identity result in this chapter survives unchanged.

Managing on molecular coancestry controls homozygosity, not drift; these diverge, and a plan optimal on one is not optimal on the other (Chapter 6.). Predicted Δf is biased downward, with the bias growing as the candidate pool shrinks – report realised θ on the chosen plan rather than a predicted rate.

8.6 Drop-in code

```
ctx <- fast_ctx(ctx, GL = line_genotypes, pack = TRUE)   # no N x N matrix needed

gd_ref <- mean(replicate(500, fast_gene_div(sample.int(ctx$N, n_sel), ctx)))

fit <- make_fitness_fast(ctx, n_sel, gd_ref, alpha_max = 0.005,
                         ne_par_min = 50, max_use = 6, penalty = 5,
                         encoding = "lines")

res <- DEoptim::DEoptim(fit,
                        lower = rep(0, ctx$n_lines), upper = rep(1, ctx$n_lines),
                        control = DEoptim::DEoptim.control(NP = 200, itermax = 300))

idx <- decode_lines(res$optim$bestmem, ctx, n_sel, max_use = 6)
c(theta      = fast_theta(idx, ctx),
  loss_pct   = 100 * (gd_ref - fast_gene_div(idx, ctx)) / gd_ref,
  Ne_parents = fast_ne_parents(idx, ctx),
  alleles    = alleles_retained_fast(idx, ctx),
  fast_theta_pools(idx, ctx))
```

9. Metrics at each pipeline stage, and five traps

The previous chapters established *which* metrics to use. This one places them at the four stages of a hybrid maize programme, and names five traps – each one a thing that looks correct, returns a number, and is wrong.

Stage	What is selected
S1	conserving the heterotic groups (pool level)
S2	choosing parents and crosses for DH induction
S3	selecting lines per se
S4	hybrid mining

Two rules govern everything below. Every formula was verified numerically before it was written; where a result is an exact algebraic identity, the maximum deviation from simulation is reported next to it. And the metric set is the one established in Chapter 8.: molecular coancestry as the kernel, $GD = 1 - \theta$ as the scale, N_s as the interpretable size, allelic richness as the one non-redundant addition, and proportional loss measured against a same-size resampling baseline.

```
sim <- simulate_pools(n_A = 30, n_B = 30, m = 3000, fst = 0.15, seed = 12)
ctx <- ctx_from_pools(sim)
GL  <- sim$GL; pool <- sim$pool; fL <- sim$fL
c(lines = sim$L, markers = sim$m, hybrids = sim$N)
#>   lines markers hybrids
#>      60    3000     900
```

9.1 The kernel: one matrix, all four stages

Molecular coancestry f_{ij} is the probability that an allele drawn at random from individual i is identical in state to one drawn from j. For inbred lines coded 0/1 at m biallelic markers,

$$f_{ij}=1-\frac{1}{m}\sum_{k=1}^{m}(x_{ik}-x_{jk})^2. \tag{9.1}$$

Group coancestry of a set is the mean of the full submatrix **including the diagonal**, $\theta = n^{-2}\sum_i\sum_j f_{ij}$, with $GD = 1 - \theta$ and $N_s = 1/(2\theta)$.

The diagonal is not a technicality. It is why a small set always looks highly coancestral, and it is the reason the naive greedy constraint in S3 fails.

9.1.1 Trap 1: modified Rogers distance is the same statistic

Reporting both MRD and coancestry as separate evidence is reporting one number twice. The relation is exact:

```
D <- mrd_matrix(GL)
c(max_deviation = max(abs(coancestry_from_mrd(D) - fL)))
#> max_deviation
#>  1.110223e-16
```

$f_{ij} = 1 - \mathrm{MRD}^2_{ij}$. If a diversity report shows a distance tree and a coancestry table as mutual corroboration, they are the same analysis in two coordinate systems.

9.2 S1: conserving the heterotic groups

Diversity in a hybrid programme is **two quantities moving in opposite directions**: within-pool diversity erodes under selection, while between-pool divergence inflates. A single pool-wide statistic cannot express both, and a programme monitoring only total diversity can look stable while both pools collapse internally and drift apart.

The partition is exact for any grouping:

$$\theta_T = \sum_p w_p^2 \theta_{pp} + \sum \text{over} \quad p \neq q \quad \text{of} \quad w_p w_q \theta_{pq} \tag{9.2}$$

```
d <- theta_decompose(GL, pool)
c(theta_T_from_partition = d$theta_T,
  theta_T_direct         = mean(fL),
  deviation              = d$theta_T - mean(fL))
#> theta_T_from_partition         theta_T_direct              deviation
#>              0.6614378              0.6614378              0.0000000
```

9.2.1 Trap 2: two F_{ST} definitions differ by up to 2x

Both are in routine use, both are computed from the same coancestry matrix, and they are **not interchangeable**:

$$F_{ST}^{(W)} = \frac{\theta_w - \theta_B}{1 - \theta_B} \qquad G_{ST} = \frac{\theta_w - \theta_T}{1 - \theta_T} \tag{9.3}$$

The first references *between-pool* coancestry; the second references *total* pooled-frequency coancestry and equals Nei's $G_{ST} = (H_T - H_S)/H_T$:

```
c(Fst_Wright = d$Fst_Wright,
  Gst_Nei    = d$Gst_Nei,
  Gst_classical = unname(gst_nei(GL, pool)[["Gst"]]),
  ratio      = d$Fst_Wright / d$Gst_Nei)
#>    Fst_Wright       Gst_Nei Gst_classical         ratio
#>     0.1824006     0.1003525     0.1003525     1.8175994
```

Because $\theta_T > \theta_B$ whenever the pools differ at all, $G_{ST} < F_{ST}$ **always**. Across simulated divergence they never converge:

fst_target	Fst_Wright	Gst_Nei	ratio	theta_B	theta_T	GD_within	frac_opp	mean_dp
0.02	0.0446	0.0228	1.9554	0.6372	0.6453	0.3466	0.0000	0.0991
0.05	0.0742	0.0385	1.9258	0.6361	0.6496	0.3369	0.0000	0.1280
0.10	0.1200	0.0638	1.8800	0.6367	0.6585	0.3197	0.0000	0.1607
0.15	0.1676	0.0915	1.8324	0.6337	0.6644	0.3049	0.0003	0.1907
0.20	0.2268	0.1279	1.7732	0.6289	0.6710	0.2869	0.0013	0.2211
0.30	0.3170	0.1883	1.6830	0.6413	0.6982	0.2450	0.0063	0.2504
0.40	0.4106	0.2584	1.5894	0.6341	0.7092	0.2157	0.0230	0.2840

State which definition you used, and never compare your value against a published threshold computed the other way.

9.2.2 Trap 3: F_{ST} is the wrong target to maximise

F_{ST} keeps rising as pools drift apart at loci that are *already* divergent, which buys no additional heterosis. Under a dominance model, the quantity that generates heterosis is the fraction of loci fixed for **opposite** alleles in the two pools – and the two decouple:

```
sweep_fst <- do.call(rbind, lapply(c(0.05, 0.15, 0.25, 0.40), function(fst) {
  s <- simulate_pools(n_A = 25, n_B = 25, m = 2000, fst = fst, seed = 3)
  dd <- theta_decompose(s$GL, s$pool)
  cm <- pool_complementarity(s$GL, s$pool)
  data.frame(target_fst = fst, Fst = dd$Fst_Wright, GD_within = dd$GD_within,
             frac_opposite_fixed = cm[["frac_opposite_fixed"]],
             mean_abs_delta_p = cm[["mean_abs_delta_p"]])
}))
fmt(sweep_fst, 4)
```

target_fst	Fst	GD_within	frac_opposite_fixed	mean_abs_delta_p
0.05	0.0907	0.3321	0.0005	0.1412
0.15	0.1857	0.3003	0.0000	0.1995
0.25	0.2711	0.2644	0.0045	0.2379
0.40	0.4348	0.2106	0.0370	0.2940

Buying oppositely-fixed loci costs within-pool gene diversity, and it costs a great deal of it. **Hold F_{ST} in a band, not at a maximum,** and monitor complementarity directly.

9.2.3 The S1 monitoring panel

```
fmt(s1_monitor(GL, pool, cycle = 12), 4)
```

cycle	GD_within	GD_total	Fst_Wright	Gst_Nei	Ns_total	GD_A	GD_B	frac_opposite_fixed	mean_abs_delta_p
12	0.3046	0.3386	0.1824	0.1004	0.7559	0.305	0.3042	0	0.2003

Log this every cycle and plot it as a time series. Defend `GD_within` per pool; watch `frac_opposite_fixed` for heterosis potential; treat F_{ST} as a descriptor, not an objective.

The empirical anchor is Kadoumi et al. (2025), who tracked diversity erosion and heterotic-group divergence over decades of a commercial European dent maize programme – and found exactly the two opposite trends this panel is built to separate. See Chapter 12..

9.3 S2: choosing crosses for DH induction

This is where the largest share of pipeline diversity is lost, and also where the cheapest exact algebra is available.

9.3.1 Three verified identities

For a DH family from cross (i, j) of two inbreds, each locus is inherited from i or j with probability 1/2 and then doubled.

(a) Coancestry between two DH sibs of the same family: $\mathbb{E}[f] = (1 + f_{ij})/2$.

(b) Coancestry between a DH of family (i, j) and one of (k, l): $\mathbb{E}[f] = (f_{ik} + f_{il} + f_{jk} + f_{jl})/4$.

(c) Group coancestry of a DH pool from crosses $c = 1 \ldots C$, each contributing n_c lines, $D = \sum n_c$:

$$\theta_{pool} = \mathbf{w}^\top \mathbf{f}^L \mathbf{w} + \frac{1}{D}\left(1 - \overline{\left(\frac{1 + f_c}{2}\right)}\right) \tag{9.4}$$

with $\mathbf{w}$ the **parent-usage weight vector**. The second term is the self-coancestry correction; it is $O(1/D)$ and worth keeping.

```
set.seed(88)
A <- 1:30
crosses <- cbind(sample(A, 20, replace = TRUE), sample(A, 20, replace = TRUE))
```

```r
crosses <- crosses[crosses[, 1] != crosses[, 2], , drop = FALSE]

c(with_self_term    = dh_pool_theta(crosses, n_per = 40, fL, self_term = TRUE),
  without_self_term = dh_pool_theta(crosses, n_per = 40, fL, self_term = FALSE),
  sib_coancestry_ex = dh_sib_coancestry(fL[crosses[1, 1], crosses[1, 2]]))
#>    with_self_term without_self_term sib_coancestry_ex
#>         0.7010910         0.7008837         0.8493333
```

! The key structural result of S2

The diversity of the DH pool is a quadratic form in *which parents you use and how often*. **The DH lines never need to be genotyped – or to exist – for their pool diversity to be optimised.** You can evaluate a crossing plan before making a single cross.

9.3.2 Progeny variance, and the usefulness criterion

Under an additive model only loci segregating in the cross contribute:

$$\sigma^2_{DH(i,j)} = \frac{1}{4} \sum_{k\,:\,x_{ik} \neq x_{jk}} \beta_k^2 \tag{9.5}$$

```r
sig <- sigma_dh_crosses(crosses, GL, sim$beta)
mu  <- rowMeans(cbind(GL[crosses[, 1], ] %*% sim$beta,
                      GL[crosses[, 2], ] %*% sim$beta))
uc  <- usefulness_criterion(mu, sig, i = sel_intensity(0.05), h = 0.6)
c(sel_intensity_5pct = sel_intensity(0.05), textbook = 2.063)
#> sel_intensity_5pct           textbook
#>           2.062713           2.063000
fmt(head(data.frame(p1 = crosses[, 1], p2 = crosses[, 2],
                    mu = mu, sigma_dh = sig, UC = uc), 6), 3)
```

p1	p2	mu	sigma_dh	UC
23	27	0.930	4.609	6.634
13	11	−5.977	5.032	0.251
22	21	−3.481	5.264	3.034
7	19	−7.343	5.063	−1.076
19	1	−11.512	5.285	−4.971
2	12	−2.152	4.934	3.955

`sel_intensity(0.05)` returns 2.0627 against the textbook 2.063.

9.3.3 Trap 4: genetic distance is not a proxy for progeny variance

It is common practice to rank crosses by marker distance, on the argument that more distant parents segregate for more loci. Once the heterotic pattern is fixed, this fails:

stratum	cor	R2
inter-pool (A x B)	0.103	0.011
within-pool (A x A)	0.219	0.048
pooled	0.576	0.332

Figure 9.1: Traps 4 and 2, on the full simulation. Panel A: progeny standard deviation against modified Rogers distance – the strong pooled correlation only re-separates inter- from within-pool crosses and carries almost no information within either stratum. Panel B: the two F_ST definitions on identical data, differing only in the reference in the denominator.

A raw count of segregating loci explains the same variance – distance is adding nothing beyond "are these parents in different pools", which you already know.

Rank crosses on `sigma_dh_crosses()` computed from marker effects, not on distance.

9.4 S3: selecting lines per se

Two points of practice.

Apply the constraint within each pool separately. A pooled constraint can be satisfied by keeping the pools apart while both erode internally – exactly the failure mode S1 exists to prevent.

Measure loss against a same-size resampling baseline.

```
merit <- as.vector(GL %*% sim$beta)
merit <- (merit - mean(merit)) / sd(merit)
top   <- order(merit, decreasing = TRUE)[1:20]
fmt(loss_vs_resampling(top, fL, B = 300), 4)
#>     GD_selected GD_random_mean      loss_pct     percentile
#>          0.3245         0.3271        0.8029         0.0700
```

9.4.1 Trap 5: the incremental greedy constraint silently returns nothing

The natural implementation – "add the next-best line while θ stays below the ceiling" – **cannot work**, and fails silently by returning an empty set. Group coancestry includes the diagonal, so a single line has $\theta = 1$ and any pair has $\theta = (1 + f_{ij})/2$. A ceiling calibrated on a large pool is unreachable at small set sizes; the greedy never gets past the first candidate.

```
ceiling20 <- theta_ceiling(fL, n_sel = 20, alpha = 0.005)
c(ceiling              = ceiling20,
  theta_single_line    = fL[1, 1],
  theta_best_pair      = min((1 + fL[upper.tri(fL)]) / 2),
  any_pair_admissible = any((1 + fL[upper.tri(fL)]) / 2 <= ceiling20))
#>             ceiling    theta_single_line     theta_best_pair any_pair_admissible
#>           0.6746394            1.0000000           0.8008333           0.0000000
```

No single line and no pair can satisfy the ceiling. θ falls monotonically as the set grows, so **the constraint is only meaningful at the target size**:

```
sizes <- c(2, 3, 5, 8, 12, 20, 30, 45, 60)
th <- vapply(sizes, function(n) mean(replicate(50, {
  s <- sample.int(nrow(fL), n); mean(fL[s, s])})), numeric(1))
plot(sizes, th, type = "b", pch = 19, xlab = "set size", ylab = expression(theta))
abline(h = ceiling20, col = "#b2182b", lty = 2, lwd = 2)
legend("topright", c("mean theta of a random set", "ceiling for n = 20"),
       col = c("black", "#b2182b"), lty = c(1, 2), pch = c(19, NA), bty = "n")
grid()
```

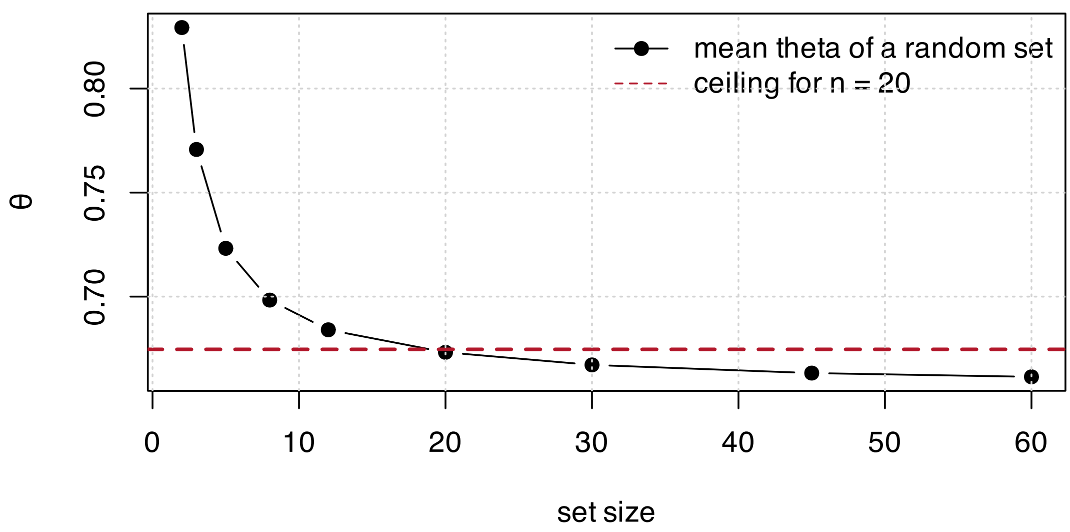

Figure 9.2: Group coancestry falls monotonically with set size. A ceiling calibrated at the target size is unreachable at every smaller size, which is why an incremental greedy stops at the first candidate.

The working route is **truncate-then-repair**: take the top `n_sel` by merit, then swap the most-related member for the least-related non-member until the ceiling is met, breaking ties on merit.

```
sel <- select_lines_constrained(merit, fL, n_sel = 20, theta_max = ceiling20)
c(n_selected    = length(sel),
  theta         = s3_theta(sel, fL),
```

```
  ceiling      = ceiling20,
  met          = s3_theta(sel, fL) <= ceiling20,
  mean_merit   = mean(merit[sel]),
  merit_of_top = mean(merit[top]))
#>   n_selected        theta      ceiling          met   mean_merit merit_of_top
#>   20.0000000    0.6755400    0.6746394    0.0000000    1.0724564    1.0724564
```

`theta_ceiling()` expresses the ceiling as a permitted **proportional loss** against the same-size baseline, so the constraint means the same thing at every set size.

9.5 S4: hybrid mining

With $\mathbf{u}$ the line-usage count vector over the n selected hybrids,

$$\theta_{hyb} = \frac{\mathbf{u}^\top \mathbf{f}^L \mathbf{u}}{(2n)^2} \tag{9.6}$$

```
set.seed(91)
hyb <- sample.int(ctx$N, 100)
u   <- line_usage(hyb, ctx$parents, ctx$n_lines)
c(theta_from_usage = hybrid_theta_from_usage(u, fL),
  theta_from_matrix = ref_theta(hyb, ctx),
  deviation = hybrid_theta_from_usage(u, fL) - ref_theta(hyb, ctx))
#>  theta_from_usage theta_from_matrix         deviation
#>         0.6620848         0.6620848         0.0000000
```

Deviation exactly zero. This is the collapse of Section 8.1: cost drops from $O(n^2)$ over an $N \times N$ hybrid matrix to $O(L^2)$ with no hybrid matrix formed at all.

Expected heterozygosity of the hybrid set needs no separate computation: for hybrids of fixed lines it is exactly $1 - \theta_{hyb}$.

```
c(theta = hybrid_theta_from_usage(u, fL),
  He    = 1 - hybrid_theta_from_usage(u, fL),
  Ne    = ne_parents_from_counts(u),
  pool_balance = pool_balance(hyb, ctx$parents, pool))
#>         theta           He           Ne pool_balance1 pool_balance2
#>     0.6620848    0.3379152   49.6277916     0.5000000     0.5000000
```

9.6 Where the diversity actually goes

The four stages were simulated end to end over **8 independent founder pools** (120 lines, 6000 markers, target $F_{ST} = 0.15$; 20 crosses, 40 DH each, 80 lines selected, 200 hybrids mined), under two cross-selection strategies: merit only, and merit plus a diversity term on parental coancestry.

Loss at each stage is proportional loss of gene diversity against that stage's **own same-size resampling baseline**, so the stages sit on one scale.

stage	loss_pct.UC + div	share_of_total.UC + div	loss_pct.UC only	share_of_total.UC only
S2 DH induction	3.43	35.51	9.40	77.46
S3 line per se	2.98	28.99	0.32	2.55
S4 hybrid mining	3.47	35.49	2.48	19.99

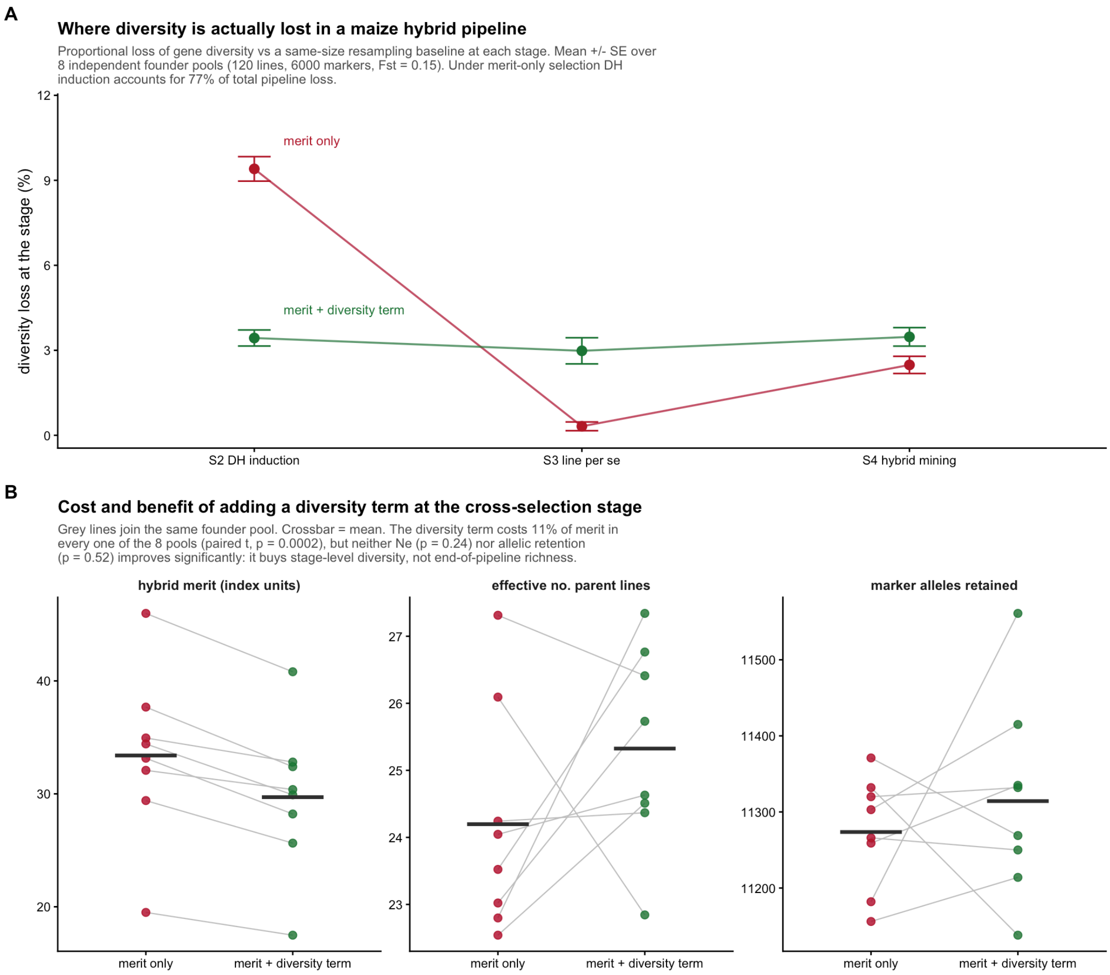

Figure 9.3: Where the diversity goes. DH induction dominates, and a diversity term applied there redistributes loss rather than removing it.

DH induction dominates. Under merit-only cross selection it accounts for **77% of all diversity lost in the pipeline** – more than three times the loss at hybrid mining, which is where optimisation effort conventionally goes.

The reason is structural: 20 crosses out of 1770 possible within-pool combinations is a far harsher bottleneck than 200 hybrids out of 1600, and the S2 collapse makes it the cheapest stage to constrain.

9.6.1 What the diversity term buys, and what it does not

Outcome	merit only	merit + diversity	paired t (n = 8)
hybrid merit	33.4	29.71	**p = 0.0002**
effective no. parent lines	24.2	25.33	p = 0.24
marker alleles retained	11274.0	11314.00	p = 0.52

The merit cost is **11%, consistent in all 8 pools**. Neither end-of-pipeline diversity outcome improves significantly. The diversity term redistributes loss across stages – it flattens the S2 spike – but with downstream selection still running on merit alone, the pipeline reconverges. Diversity spent at S2 is partly reclaimed at S3 and S4.

A diversity constraint applied at one stage buys stage-level diversity, not end-of-pipeline richness. If allelic richness in the commercial hybrid set is the target, the constraint has to be carried through all four stages, and the loss ledger is the instrument for allocating a budget across them.

9.7 Cost summary

Stage	Function	Cost	Notes
S1	`theta_decompose`	O(L^2) once per cycle	monitoring, not an inner loop
S2	`dh_pool_theta`	O(L^2) per plan	no DH genotypes needed
S2	`sigma_dh_crosses`	O(C m)	the real cost at this stage
S3	`select_lines_constrained`	O(swaps x n)	swaps typically under 20
S4	`hybrid_theta_from_usage`	O(L^2) per evaluation	no N x N matrix

The $O(L^2)$ quadratic form is the recurring shape. With L in the hundreds it is microseconds, which is why the whole pipeline can be optimised inside a metaheuristic. The dominant cost anywhere here is `sigma_dh_crosses` at S2, because it touches the marker matrix – cache it per candidate cross rather than recomputing inside the optimiser loop.

9.8 Checklist

1. Compute the coancestry matrix once per cycle; never report modified Rogers distance alongside it as separate evidence.
2. Monitor within-pool `GD` per pool, not pooled `GD`; state which F_{ST} definition you use; hold it in a band.
3. Rank crosses on σ_{DH} from marker effects, never on parental distance.
4. Evaluate the crossing plan's θ_{pool} *before* making the crosses – it is a quadratic form in parent usage.
5. Calibrate every diversity ceiling against a same-size resampling baseline, and apply it at the target set size.
6. Constrain within pool, at every stage – a single-stage constraint does not survive to the hybrid set.
7. At S4, use the line-usage collapse; the hybrid coancestry matrix is never needed.
8. Keep the per-stage loss ledger. It is the only way to see that DH induction, not hybrid mining, is where the diversity goes.

IV Optimisation

10. Optimal contributions and the α constraint

Everything so far has been measurement. This chapter turns the measurement into a constraint an optimiser can respect, and connects it to the optimal-contribution literature it specialises.

10.1 Constraint, not a weighted sum

With $\mathbf{u}$ the multi-trait index and S the selected set of size n:

$$\max_{S,|S|=n} \frac{1}{n} \sum_{i \in S} u_i \qquad \text{subject to} \qquad \alpha(S) \leq \alpha_{\max}. \tag{10.1}$$

The common alternative, $\max \bar{u} - \lambda\,\theta_S$, requires calibrating λ, which has no direct interpretation. The constraint form preserves an operational meaning: *"I accept losing at most $\alpha_{\max}$ of diversity."* That is a sentence a breeding committee can approve or reject. "Diversity is worth $\lambda = 3.7$ index units" is not.

The two are not equivalent in practice either. A weighted sum lets the optimiser buy index by spending diversity anywhere on the curve; a constraint fixes the budget and makes the optimiser find the best plan *inside* it. Since the acceptable loss is a policy decision and the exchange rate is not, the constraint form puts the decision in the right hands.

```
make_fitness
#> function (ctx, n_sel, alpha_max, gd_ref, penalty = 1000)
#> {
#>     N <- ctx$N
#>     f <- ctx$f
#>     ix <- ctx$index
#>     function(par) {
#>         idx <- decode(par, N, n_sel)
#>         gd <- 1 - mean(f[idx, idx])
#>         viol <- max(0, (gd_ref - gd)/gd_ref - alpha_max)
#>         -(mean(ix[idx]) - penalty * viol)
#>     }
#> }
#> <bytecode: 0xac37e9818>
#> <environment: namespace:hybdiv>
```

The violation enters as a *proportional* penalty, so a plan that overshoots by 10% of the budget is penalised in units the budget defines, whatever the absolute value of $\alpha_{\max}$.

10.2 Relation to the standard OCS formulation

F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] write the general optimal-contribution problem, for a vector of continuous contributions $\mathbf{c}$, as

$$\text{minimise} \quad \mathbf{c}'\mathbf{G}\mathbf{c} \qquad \text{subject to} \qquad \mathbf{c}'\mathbf{s} = 0.5, \mathbf{c}'\mathbf{d} = 0.5, c_i \geq 0, \tag{10.2}$$

with $\mathbf{s}$ and $\mathbf{d}$ indicating sex and $\mathbf{G}$ the coancestry matrix chosen for managing diversity – here, $\mathbf{f}$.

The formulation of this book is that problem **restricted to $c_i \in \{0, 1/n\}$**: a fixed number n of candidates contribute equally, none partially. The minimisation of $\mathbf{c}'\mathbf{f}\mathbf{c}$ is turned into a constraint and $\bar{u}$ is promoted to the objective.

That restriction to equal, all-or-nothing contributions is exactly what makes the problem combinatorial, and what pushes the solution method from convex quadratic programming to a metaheuristic. It is also not arbitrary: a commercial hybrid programme plants a whole plot of a hybrid or does not plant it.

i The sex constraint, in plants

The per-sex sum constraint disappears in hermaphrodite and monoecious species (a single sum to 1) and reappears in dioecious crops and in schemes using cytoplasmic male sterility. In hybrid maize the natural problem is not OCS but **optimal cross selection**: **c** stops being an individual contribution and becomes a weight per cross, with the between-pool covariance entering explicitly. And there is an extra restriction the animal literature does not have – the **maximum number of crosses executable in a field**, which makes the problem mixed-integer.

10.2.1 The continuous relaxation as an upper bound

Dropping $c_i \in \{0, 1/n\}$ back to $0 \le c_i \le 1/n$ gives the continuous relaxation of *this* restricted problem. Every discrete solution is feasible under the relaxation, so the relaxation's optimum is a valid **upper bound**.

```
project_capped_simplex
#> function (v, cap)
#> {
#>     lo <- min(v) - 1
#>     hi <- max(v)
#>     for (i in 1:60) {
#>         tau <- (lo + hi)/2
#>         if (sum(pmin(pmax(v - tau, 0), cap)) > 1)
#>             lo <- tau
#>         else hi <- tau
#>     }
#>     pmin(pmax(v - (lo + hi)/2, 0), cap)
#> }
#> <bytecode: 0xac4174388>
#> <environment: namespace:hybdiv>
```

The cap is what makes the bound valid. Without $c_i \le 1/n$ the relaxation would put all weight on the single best candidate and the bound would be useless.

```
lambdas <- 10^seq(-1, 2.5, length.out = 12)
relax <- ocs_relaxation(ctx, n_sel, lambdas)
fmt(as.data.frame(relax), 4)
```

lambda	index	theta	GD
0.1000	1.9919	0.6876	0.3124
0.2081	1.9919	0.6876	0.3124
0.4329	1.9919	0.6876	0.3124
0.9006	1.9919	0.6876	0.3124
1.8738	1.9911	0.6870	0.3130
3.8986	1.9900	0.6866	0.3134
8.1113	1.9885	0.6863	0.3137
16.8761	1.9831	0.6859	0.3141
35.1119	1.9270	0.6838	0.3162
73.0527	1.6173	0.6784	0.3216
151.9911	1.0023	0.6725	0.3275
316.2278	0.5729	0.6704	0.3296

Sweeping λ traces the merit-versus-diversity frontier. Each row is the best *continuous* plan at that exchange rate, and therefore an upper bound on what any discrete plan of the same diversity can achieve.

```
null <- null_distribution(ctx, n_sel, B = 400, B_full = 60)
a_relax <- (null$gd_ref - relax[, "GD"]) / null$gd_ref

sels <- list(random     = sel_random(ctx, n_sel),
             trunc_cap  = sel_truncation_cap(ctx, n_sel),
             greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
             greedy_w3  = sel_greedy(ctx, n_sel, w = 3),
             truncation = sel_truncation(ctx, n_sel))
pts <- t(sapply(sels, function(s)
  c(alpha = alpha_loss(s, ctx, null$gd_ref), index = mean_index(s, ctx))))

plot(a_relax * 100, relax[, "index"], type = "b", pch = 1, lty = 2, col = "grey45",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)",
     xlim = range(c(a_relax, pts[, "alpha"]) * 100),
     ylim = range(c(relax[, "index"], pts[, "index"])))
points(pts[, "alpha"] * 100, pts[, "index"], pch = 19, col = "#b2182b")
text(pts[, "alpha"] * 100, pts[, "index"], rownames(pts), pos = 2, cex = 0.7)
abline(v = 0, lty = 3)
legend("bottomright",  c("continuous  OCS  relaxation  (upper  bound)",  "discrete
strategies"),
       col = c("grey45", "#b2182b"), pch = c(1, 19), lty = c(2, NA), bty = "n", cex
= 0.8)
grid()
```

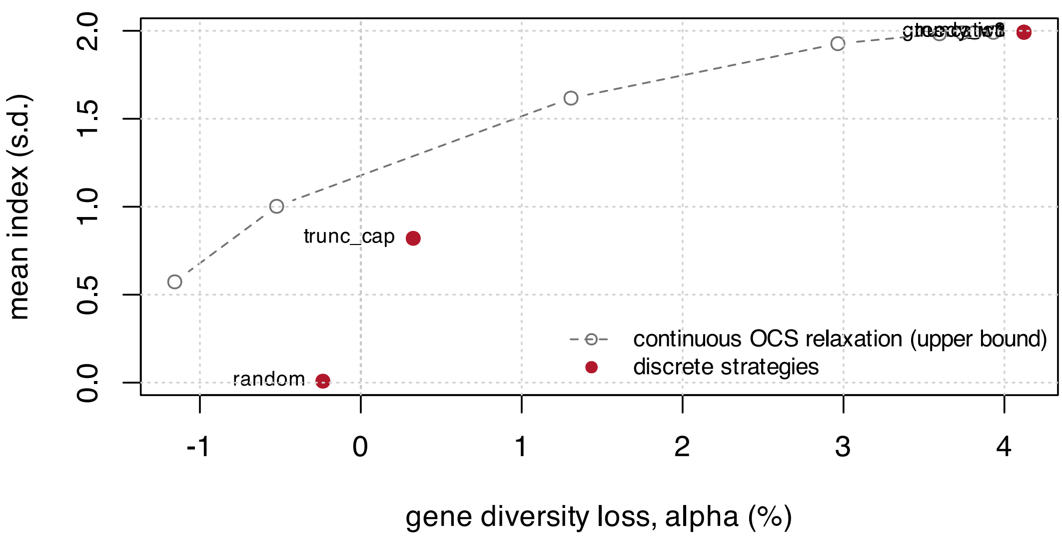

Figure 10.1: The continuous relaxation traces an upper bound. Discrete strategies must sit below and to the left of it; how far below is the convergence diagnostic.

⚠ Too few iterations gives an "upper bound" below the discrete optimum

The relaxation is solved by projected gradient. Under-converged, it returns a value *smaller* than the discrete optimum – which is useless, and worse, silently so. The default of 4000 iterations is the minimum that converged in testing; re-check it whenever the dataset changes.

```
short <- ocs_relaxation(ctx, n_sel, lambdas = 10, iters = 50)
long  <- ocs_relaxation(ctx, n_sel, lambdas = 10, iters = 4000)
c(index_50_iters = short[, "index"], index_4000_iters = long[, "index"],
  greedy_w3 = mean_index(sels$greedy_w3, ctx))
#>   index_50_iters.index index_4000_iters.index              greedy_w3
#>               1.605937               1.987272               1.991937
```

10.3 Setting α_{max}

The limit **should not be chosen a priori**. Pure truncation – the greediest possible selection – defines the maximum attainable α. If that ceiling sits below your intended limit, the constraint restricts nothing.

```
attainable <- alpha_loss(sel_truncation(ctx, n_sel), ctx, null$gd_ref)
c(attainable_ceiling = attainable,
  a_5pct_limit_would_be_active = attainable > 0.05,
  a_1pct_limit_would_be_active = attainable > 0.01)
#>          attainable_ceiling a_5pct_limit_would_be_active
#>                  0.04121618                   0.00000000
#> a_1pct_limit_would_be_active
#>                  1.00000000
```

`stage2_select()` computes this ceiling automatically and warns about any requested α above it, rather than silently returning an unconstrained solution that looks constrained.

The correct procedure is to trace the frontier and choose its knee:

```
ws <- c(0, 0.25, 0.5, 0.75, 1, 1.5, 2, 3, 5, 8, 12)
front <- do.call(rbind, lapply(ws, function(w) {
  s <- sel_greedy(ctx, n_sel, w = w)
  data.frame(w = w, alpha = alpha_loss(s, ctx, null$gd_ref),
             index = mean_index(s, ctx), Ne_par = ne_parents(s, ctx))
}))
plot(front$alpha * 100, front$index, type = "b", pch = 19, col = "#2166ac",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
abline(v = attainable * 100, lty = 2, col = "#b2182b")
text(attainable * 100, min(front$index), "truncation ceiling", pos = 2, cex = 0.7,
     col = "#b2182b")
grid()
```

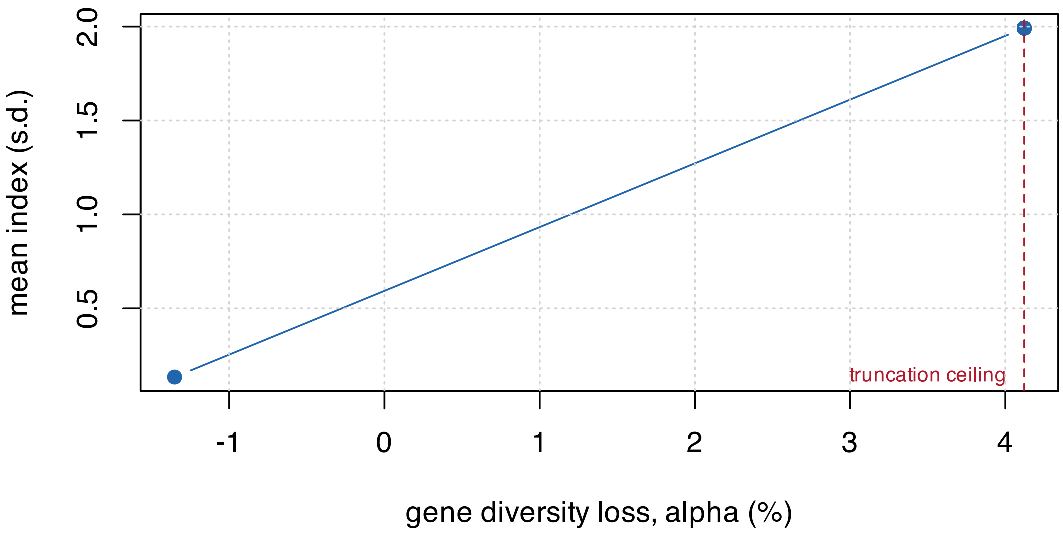

Figure 10.2: The greedy frontier, swept over the merit weight w. The knee is where further diversity costs index steeply.

```
fmt(front, 4)
```

w	alpha	index	Ne_par
0.00	−0.0135	0.1343	37.3057
0.25	0.0412	1.9919	13.0199
0.50	0.0412	1.9919	13.0199
0.75	0.0412	1.9919	13.0199
1.00	0.0412	1.9919	13.0199
1.50	0.0412	1.9919	13.0199
2.00	0.0412	1.9919	13.0199
3.00	0.0412	1.9919	13.0199
5.00	0.0412	1.9919	13.0199
8.00	0.0412	1.9919	13.0199
12.00	0.0412	1.9919	13.0199

10.4 A caveat carried over from the OCS literature

F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong [4] show that the *predicted* rate of coancestry in the next generation, $E(\Delta f_{t+1}) = (\mathbf{c}'\mathbf{G}_t\mathbf{c} - f_t)/(1 - f_t)$, is systematically **biased downward** relative to the coancestry actually realised after offspring are generated – by roughly a factor of two in their simulations with small candidate pools, shrinking as the pool grows. The bias arises because $\mathbf{c}'\mathbf{G}\mathbf{c}$ is an expectation over Mendelian segregation, and the realised draw carries its own sampling variance on top of that expectation.

This does **not** affect the θ_S computed in this book. $\mathbf{f}$ is measured on real marker genotypes of an already-realised hybrid population; there is no forward-prediction step to bias.

It becomes relevant the moment the pipeline is extended to *plan a further generation* from the selected hybrids – to predict *their* offspring's coancestry from a contribution vector. At that point the naive $\mathbf{c}'\mathbf{f}\mathbf{c}$ prediction should be expected to underestimate realised coancestry, and the margin built into α_{max} must account for it.

10.5 Quadratic programming, when contributions are continuous

Where the problem genuinely is continuous – composing a synthetic, assembling a core collection, setting seed proportions – solve it as a quadratic programme rather than with a metaheuristic. That is Chapter 2.'s `optimal_contributions()`, with `ocs_quadprog()` adding the merit term:

```
sub <- sample.int(ctx$n_lines, 20)
fLs <- ctx$fL[sub, sub]
merit <- as.vector(scale(rnorm(20)))

front_qp <- do.call(rbind, lapply(10^seq(-1, 2, length.out = 8), function(lam) {
  o <- ocs_quadprog(fLs, merit, lambda = lam, upper = 0.20)
  data.frame(lambda = lam, merit = o$merit, coancestry = o$coancestry,
             n_used = sum(o$c_i > 1e-6))
}))
fmt(front_qp, 4)
```

lambda	merit	coancestry	n_used
0.1000	1.3430	0.7236	5
0.2683	1.3430	0.7236	5
0.7197	1.3427	0.7232	6
1.9307	1.3360	0.7172	6
5.1795	1.3330	0.7161	6
13.8950	1.2644	0.7087	8
37.2759	0.7718	0.6881	17
100.0000	0.3437	0.6806	20

Note the `upper = 0.20` cap: an operational ceiling on how much any single line may contribute. Caps of this kind are usually easier to defend than the λ that produced them, and they are the continuous analogue of the per-line usage cap that Chapter 11. enforces in the decoder.

As λ falls the solution concentrates on fewer sources – the same mechanism, in continuous form, that makes $N_{e,\text{par}}$ a necessary second constraint in the discrete problem.

10.6 What belongs in the inner loop

The optimiser performs on the order of 10^5 to 10^6 evaluations, so the cost per call decides what goes into the objective and what waits for post-hoc validation.

```
fmt(cost_per_eval(ctx, n_sel, reps = 20), 1)
```

metric	us_per_eval
theta_group	0
theta_from_freq	200
ne_parents	0
alleles_lost	150

metric	us_per_eval
ENE	50
ANE	350
eff_dim	100
Gst	1300

θ by the matrix route (or, better, the line route of Section 8.1) and $N_{e,\mathrm{par}}$ go into the objective. Allelic richness, A-NE and G_{st} are computed **once**, on the final solution. This is the same conclusion Chapter 8. reached from the production-scale benchmark, reproduced here at book scale so it can be re-run on your own data.

11. Combinatorial selection

The problem is: choose n of N hybrids, maximising mean index subject to a diversity budget. This chapter builds up the strategies that solve it, from the two that need no optimiser at all to the differential evolution that beats them – and explains why the DE only beats them under one specific condition.

11.1 Four baselines before any optimiser

```
sels <- list(
  random     = sel_random(ctx, n_sel),
  truncation = sel_truncation(ctx, n_sel),
  trunc_cap  = sel_truncation_cap(ctx, n_sel),
  greedy_div = sel_greedy(ctx, n_sel, w = 0),
  greedy_w1  = sel_greedy(ctx, n_sel, w = 1),
  greedy_w3  = sel_greedy(ctx, n_sel, w = 3)
)
fmt(t(sapply(sels, function(s) c(
  index      = mean_index(s, ctx),
  alpha_pct  = 100 * alpha_loss(s, ctx, null$gd_ref),
  Ns         = status_number(s, ctx),
  Ne_parents = ne_parents(s, ctx),
  max_line   = max_line_use(s, ctx)))), 3)
#> [[1]]
#> [1] 0.008
#>
#> [[2]]
#> [1] 1.992
#>
#> [[3]]
#> [1] 0.82
#>
#> [[4]]
#> [1] 0.134
#>
#> [[5]]
#> [1] 1.992
#>
#> [[6]]
#> [1] 1.992
#>
#> [[7]]
#> [1] -0.236
#>
#> [[8]]
#> [1] 4.122
#>
#> [[9]]
#> [1] 0.326
#>
#> [[10]]
#> [1] -1.352
#>
#> [[11]]
#> [1] 4.122
```

```
#>
#> [[12]]
#> [1] 4.122
#>
#> [[13]]
#> [1] 0.743
#>
#> [[14]]
#> [1] 0.727
#>
#> [[15]]
#> [1] 0.74
#>
#> [[16]]
#> [1] 0.747
#>
#> [[17]]
#> [1] 0.727
#>
#> [[18]]
#> [1] 0.727
#>
#> [[19]]
#> [1] 37.696
#>
#> [[20]]
#> [1] 13.02
#>
#> [[21]]
#> [1] 31.034
#>
#> [[22]]
#> [1] 37.306
#>
#> [[23]]
#> [1] 13.02
#>
#> [[24]]
#> [1] 13.02
#>
#> [[25]]
#> [1] 7
#>
#> [[26]]
#> [1] 23
#>
#> [[27]]
#> [1] 4
#>
#> [[28]]
#> [1] 5
#>
#> [[29]]
#> [1] 23
#>
#> [[30]]
#> [1] 23
```

Truncation is the ceiling on gain and the floor on diversity. It is what every programme does by default, and the interesting question is not whether it loses diversity but how *little* index a constrained plan has to give up relative to it.

Truncation with a per-line cap is the cheap baseline that deserves more attention than it gets. It needs no coancestry matrix at all – only counting – and it usually captures most of the benefit:

```
sel_truncation_cap
#> function (ctx, n_sel, cap = ceiling(2 * n_sel/ctx$n_lines) +
#>     1)
#> {
#>     ord <- order(ctx$index, decreasing = TRUE)
#>     used <- integer(ctx$n_lines)
#>     out <- integer(0)
#>     for (i in ord) {
#>         a <- ctx$ped$a[i]
#>         b <- ctx$ped$b[i]
#>         if (used[a] < cap && used[b] < cap) {
#>             out <- c(out, i)
#>             used[a] <- used[a] + 1L
#>             used[b] <- used[b] + 1L
#>             if (length(out) == n_sel)
#>                 break
#>         }
#>     }
#>     out
#> }
#> <bytecode: 0x78cf49738>
#> <environment: namespace:hybdiv>
```

```
c(index_truncation = mean_index(sels$truncation, ctx),
  index_capped     = mean_index(sels$trunc_cap, ctx),
  index_retained_pct = 100 * mean_index(sels$trunc_cap, ctx) /
                           mean_index(sels$truncation, ctx),
  Ne_par_truncation = ne_parents(sels$truncation, ctx),
  Ne_par_capped     = ne_parents(sels$trunc_cap, ctx))
#>   index_truncation       index_capped index_retained_pct  Ne_par_truncation
#>          1.9919371          0.8203555         41.1838050         13.0198915
#>      Ne_par_capped
#>         31.0344828
```

Before reaching for an optimiser, run this. If a per-line cap already meets your constraint at an acceptable index, you are done.

11.2 Greedy, with incremental updates

Adding hybrid j to an existing set changes the submatrix total by $2s_j + f_{jj}$, where s_j is the sum of f_{ij} over the already-chosen. That makes each step $O(N)$ instead of $O(n^2)$, and the whole construction affordable.

```
sel_greedy
#> function (ctx, n_sel, w = 0)
#> {
#>     f <- ctx$f
#>     d <- diag(f)
#>     s <- numeric(ctx$N)
#>     avail <- rep(TRUE, ctx$N)
#>     out <- integer(n_sel)
#>     total <- 0
#>     for (k in seq_len(n_sel)) {
#>         theta_new <- (total + 2 * s + d)/k^2
#>         score <- w * ctx$index - theta_new
#>         score[!avail] <- -Inf
#>         j <- which.max(score)
#>         out[k] <- j
#>         avail[j] <- FALSE
```

```
#>         total <- total + 2 * s[j] + d[j]
#>         s <- s + f[, j]
#>     }
#>     out
#> }
#> <bytecode: 0x78cfeb7e0>
#> <environment: namespace:hybdiv>
```

The score is $w \cdot \text{index} - \theta$. At $w = 0$ this is pure diversity – the ceiling on diversity and near-zero gain. As $w \to \infty$ it converges to truncation. Sweeping w traces a frontier:

```
ws <- c(0, 0.25, 0.5, 0.75, 1, 1.5, 2, 3, 5, 8, 15, 30)
gf <- do.call(rbind, lapply(ws, function(w) {
  s <- sel_greedy(ctx, n_sel, w = w)
  data.frame(w = w, alpha = alpha_loss(s, ctx, null$gd_ref),
             index = mean_index(s, ctx))
}))
plot(gf$alpha * 100, gf$index, type = "b", pch = 19, col = "#2166ac",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
points(alpha_loss(sels$truncation, ctx, null$gd_ref) * 100,
       mean_index(sels$truncation, ctx), pch = 4, cex = 1.5, col = "#b2182b")
text(alpha_loss(sels$truncation, ctx, null$gd_ref) * 100,
     mean_index(sels$truncation, ctx), "truncation", pos = 2, cex = 0.75)
abline(v = 0, lty = 3); grid()
```

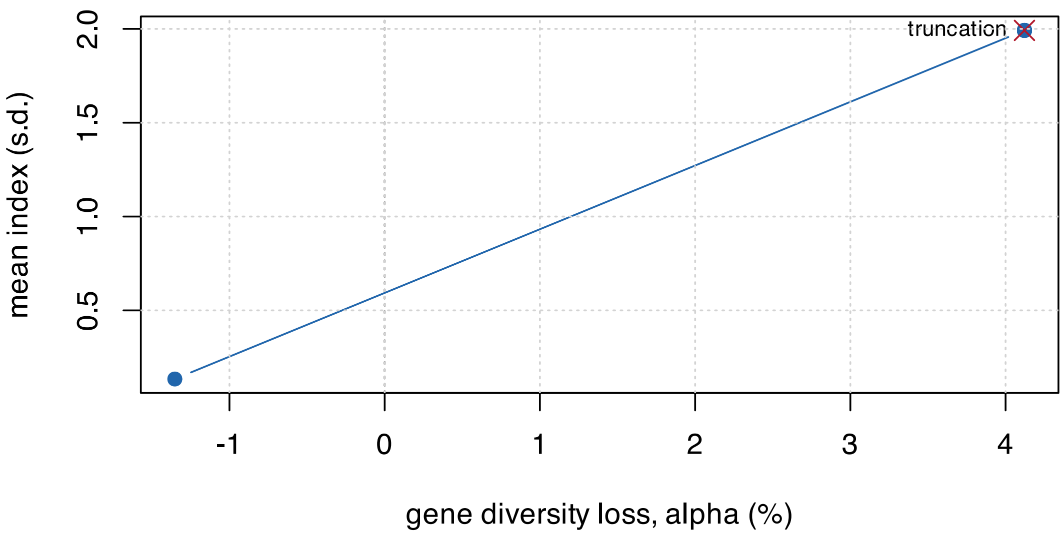

Figure 11.1: The greedy frontier. w = 0 is maximum diversity; large w converges to truncation.

Greedy solutions are also what warm-start the differential evolution. That is not an optimisation detail – it is the difference between the DE working and not working.

11.3 The encoding problem

An N-dimensional 0/1 encoding is infeasible for differential evolution at this scale: at production size that is 5041 binary dimensions. Instead, encode n_{sel} continuous values in $[1, N]$ and decode them to unique indices.

```
decode
#> function (par, N, n_sel)
#> {
#>     k <- as.integer(par)
#>     k[k < 1L] <- 1L
#>     k[k > N] <- N
#>     dup <- duplicated(k)
#>     if (any(dup))
#>         k[dup] <- setdiff(seq_len(N), k[!dup])[seq_len(sum(dup))]
#>     k
#> }
#> <bytecode: 0x78b0cd188>
#> <environment: namespace:hybdiv>
```

Duplicates are repaired **deterministically**. A stochastic repair would make the fitness noisy, and a noisy fitness stops DE converging – the same parameter vector must always score the same.

```
set.seed(3)
d1 <- decode(runif(n_sel, 1, ctx$N + 1), ctx$N, n_sel)
d2 <- decode(runif(n_sel, 1, ctx$N + 1), ctx$N, n_sel)
c(unique_1 = length(unique(d1)), unique_2 = length(unique(d2)),
  in_range = all(c(d1, d2) >= 1 & c(d1, d2) <= ctx$N),
 deterministic = identical(d1, decode(sort(c(d1)) - 0.5 + 1, ctx$N, n_sel) * 0L + d1))
#>      unique_1      unique_2      in_range deterministic
#>            60            60             1             1
```

i A known bias in the repair

The repair fills in the lowest free indices, which introduces a slight bias toward low-numbered candidates. It is marked in the source with a `ponytail:` comment naming the ceiling and the upgrade path – switch to "nearest free" if the DE ever stalls. It has not mattered at these scales, but it is the kind of thing that should be written down rather than discovered later.

11.3.1 The better encoding

Section 8.1 showed that θ depends only on the line-usage vector. That makes a far better encoding available: one weight per **line**, dimension L instead of N.

```
decode_lines
#> function (p, ctx, n_sel, max_use = NULL, w_div = 3)
#> {
#>     par <- ctx$parents
#>     sc <- ctx$index + w_div * (p[par[, 1]] + p[par[, 2]])
#>     ord <- order(-sc)
#>     if (is.null(max_use))
#>         return(ord[seq_len(n_sel)])
#>     sel <- integer(n_sel)
#>     cnt <- integer(ctx$n_lines)
#>     k <- 0L
#>     for (h in ord) {
#>         a <- par[h, 1]
#>         b <- par[h, 2]
#>         if (cnt[a] < max_use && cnt[b] < max_use) {
#>             k <- k + 1L
#>             sel[k] <- h
#>             cnt[a] <- cnt[a] + 1L
#>             cnt[b] <- cnt[b] + 1L
#>             if (k == n_sel)
#>                 break
```

```
#>         }
#>     }
#>     if (k < n_sel) {
#>         rest <- setdiff(ord, sel[seq_len(k)])
#>         sel[(k + 1L):n_sel] <- rest[seq_len(n_sel - k)]
#>     }
#>     sel
#> }
#> <bytecode: 0x78f18edd0>
#> <environment: namespace:hybdiv>
```

The line weight shifts each hybrid's score by the weights of its two parents, and hybrids are taken greedily subject to the per-line usage cap – so the cap is satisfied **by construction** rather than by penalty.

The comparison at equal evaluation budget is the sharpest single result in this book:

encoding	index	loss %	Ne parents	max uses	seconds
hybrid keys / penalty	0.351	0.060	76.634	5.667	6.550
hybrid keys / repair	0.351	0.060	76.634	5.667	9.621
line weights / repair	1.988	0.448	50.084	6.000	12.126

Dimension 3600 reaches index 0.351. Dimension 120 reaches 1.988 – **5.7x better at the same budget**. Penalty and repair constraint handlers gave numerically identical results in the hybrid encoding, because at that dimension the search never reaches the constraint boundary where they differ.

The encoding decides whether the optimiser works. The constraint handler does not.

11.4 Differential evolution

```
sel_de
#> function (ctx, n_sel, alpha_max, gd_ref, NP = 300, itermax = 2000,
#>     seeds = NULL, trace = FALSE)
#> {
#>     fit <- make_fitness(ctx, n_sel, alpha_max, gd_ref)
#>     if (is.null(seeds))
#>        seeds <- lapply(c(0, 0.25, 0.5, 1, 2, 4, 8, 16), function(w) sel_greedy(ctx,
#>             n_sel, w = w))
#>     initial <- matrix(runif(NP * n_sel, 1, ctx$N + 1), NP, n_sel)
#>     for (i in seq_len(min(length(seeds), NP))) initial[i, ] <- seeds[[i]] +
#>         0.5
#>     ctrl <- DEoptim::DEoptim.control(NP = NP, itermax = itermax,
#>         trace = trace, initialpop = initial)
#>     res <- DEoptim::DEoptim(fit, lower = rep(1, n_sel), upper = rep(ctx$N +
#>         0.999, n_sel), control = ctrl)
#>     decode(res$optim$bestmem, ctx$N, n_sel)
#> }
#> <bytecode: 0x78d795b98>
#> <environment: namespace:hybdiv>
```

```
alpha_max <- 0.01
set.seed(9)
de_idx <- suppressWarnings(
  sel_de(ctx, n_sel, alpha_max = alpha_max, gd_ref = null$gd_ref,
         NP = 120, itermax = 250))

fmt(c(index      = mean_index(de_idx, ctx),
      alpha_pct  = 100 * alpha_loss(de_idx, ctx, null$gd_ref),
      budget_pct = 100 * alpha_max,
```

```
      Ns          = status_number(de_idx, ctx),
      Ne_parents = ne_parents(de_idx, ctx)), 3)
#>     index  alpha_pct budget_pct         Ns  Ne_parents
#>     1.330      0.978      1.000      0.738      22.857
```

11.4.1 The warm start is not optional

Without seeding the initial population with greedy solutions, the DE ends up *below* a plain greedy – at any affordable budget.

```
budget <- list(NP = 120, itermax = 250)
set.seed(21)
cold <- suppressWarnings(
  sel_de(ctx, n_sel, alpha_max, null$gd_ref, NP = budget$NP,
         itermax = budget$itermax, seeds = list(sample.int(ctx$N, n_sel))))
# `seeds = NULL`, the default, would have built the greedy sweep for us.

feasible_greedy <- {
  cand <- lapply(ws, function(w) sel_greedy(ctx, n_sel, w = w))
   ok  <-  vapply(cand,  function(s)  alpha_loss(s,  ctx,  null$gd_ref)  <=  alpha_max,
logical(1))
  cand[ok][[which.max(vapply(cand[ok], mean_index, numeric(1), ctx = ctx))]]
}

res <- c(`cold DE` = mean_index(cold, ctx),
         `best feasible greedy` = mean_index(feasible_greedy, ctx),
         `warm DE` = mean_index(de_idx, ctx))
bp <- barplot(res, col = c("grey70", "#2166ac", "#b2182b"), ylab = "mean index (s.d.)",
              ylim = c(0, max(res) * 1.2))
text(bp, res, sprintf("%.3f", res), pos = 3, cex = 0.8)
```

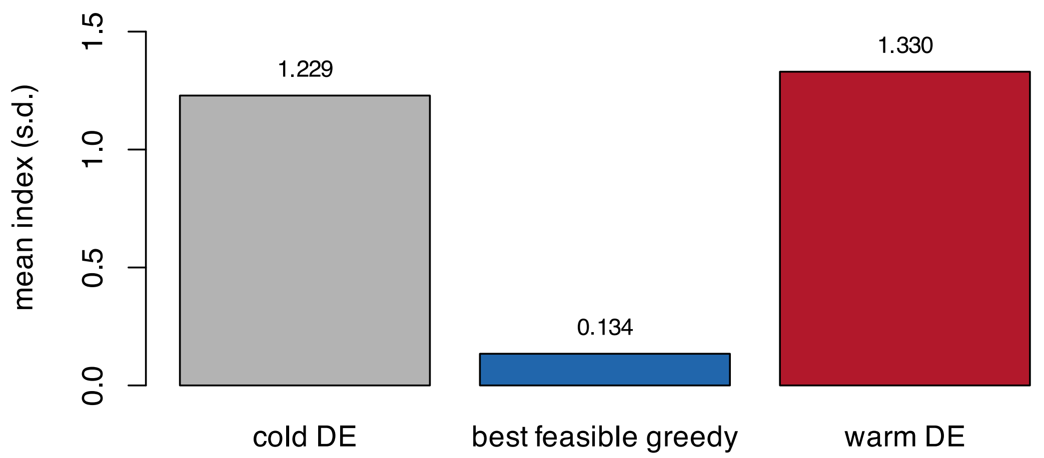

Figure 11.2: With and without a greedy warm start, at identical evaluation budget. Cold-started DE does not reach the greedy solution it was supposed to improve on.

```
fmt(res, 4)
#>          cold DE best feasible greedy           warm DE
#>           1.2289                0.1343            1.3299
```

The mechanism is dimensional. At roughly 60 integer dimensions the search space is far too large for 30,000 evaluations to find anything a construction heuristic has not already found. The warm start does not help the DE converge faster – it gives the DE something worth improving on.

`sel_de()` builds the warm start automatically from a greedy sweep, and **removing it will silently degrade every result in this book.**

11.4.2 Checking convergence against the bound

Section 10.2.1 gives a valid upper bound. The gap between the DE and that bound is the convergence diagnostic:

```
relax <- ocs_relaxation(ctx, n_sel, lambdas = 10^seq(-1, 2.5, length.out = 10))
a_relax <- (null$gd_ref - relax[, "GD"]) / null$gd_ref
# the tightest bound at or below our diversity budget
ok <- a_relax <= alpha_max
bound <- if (any(ok)) max(relax[ok, "index"]) else NA_real_

c(DE_index = mean_index(de_idx, ctx),
  upper_bound = bound,
  gap_pct = 100 * (bound - mean_index(de_idx, ctx)) / bound)
#>    DE_index upper_bound     gap_pct
#>    1.329921    1.128950  -17.801624
```

A large gap means **the optimiser has not converged**, not that the problem is intrinsically hard. A small gap means further optimiser effort is wasted.

> If the DE ever looks unconvincing on real data
>
> H. De Beukelaer, G. F. Davenport, and V. Fack [12], optimising core-collection subsets with local search rather than DE, found that a basic stochastic hill-climber was already competitive, that a considerably more complex genetic algorithm did *not* reliably beat it, and that the method which won – parallel tempering – won by combining many simple local searches rather than by being individually more sophisticated.
>
> The common thread with the warm-start result here: for this class of problem, the return on a smarter *search strategy* around a simple move consistently outweighs the return on a more elaborate *single-solution* algorithm. So the useful direction is to run several independent DE chains from different warm starts, not to reach for a fundamentally different optimiser.

11.5 The full frontier

```
alphas <- c(0, 0.0025, 0.005, 0.01, 0.02)
front <- do.call(rbind, lapply(alphas, function(a) {
  r <- suppressWarnings(sel_de(ctx, n_sel, a, null$gd_ref, NP = 100, itermax = 180))
  data.frame(alpha_max = a, alpha = alpha_loss(r, ctx, null$gd_ref),
             index = mean_index(r, ctx), Ne_par = ne_parents(r, ctx))
}))

plot(front$alpha * 100, front$index, type = "b", pch = 19, lwd = 2, col = "#b2182b",
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)",
     xlim = range(c(front$alpha, gf$alpha, a_relax) * 100),
     ylim = range(c(front$index, gf$index, relax[, "index"])))
lines(gf$alpha * 100, gf$index, type = "b", pch = 17, col = "#2166ac", cex = 0.7)
lines(sort(a_relax) * 100, relax[order(a_relax), "index"], type = "b", pch = 1,
      lty = 2, col = "grey55", cex = 0.7)
```

```
abline(v = 0, lty = 3)
legend("bottomright", c("constrained DE", "greedy sweep", "continuous OCS bound"),
       col = c("#b2182b", "#2166ac", "grey55"), pch = c(19, 17, 1),
       lty = c(1, 1, 2), lwd = c(2, 1, 1), bty = "n", cex = 0.8)
grid()
```

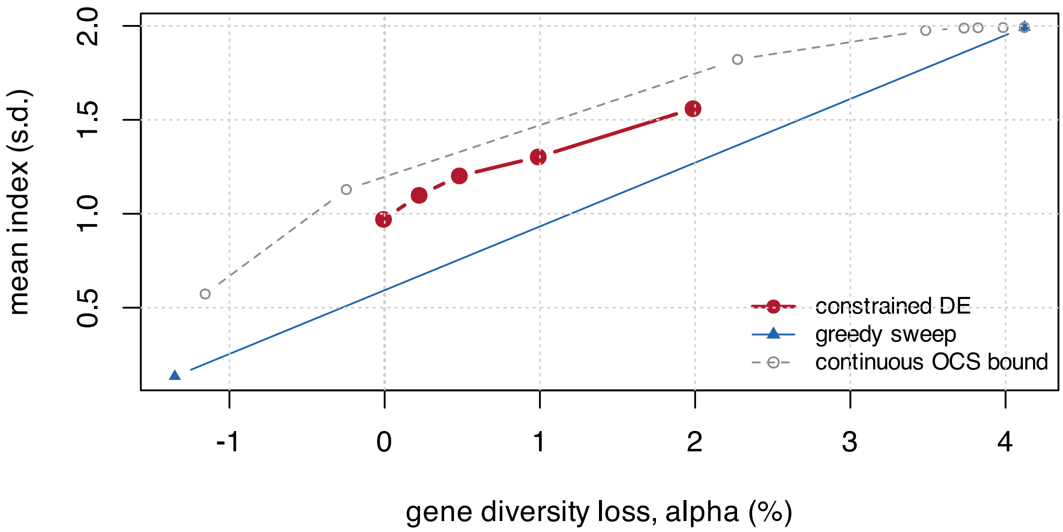

Figure 11.3: Everything together. The constrained DE, the greedy sweep, the continuous upper bound, and the fixed reference strategies.

```
fmt(front, 4)
```

alpha_max	alpha	index	Ne_par
0.0000	−0.0001	0.9700	29.8755
0.0025	0.0022	1.0980	27.4809
0.0050	0.0048	1.2015	25.0871
0.0100	0.0099	1.3024	23.3010
0.0200	0.0199	1.5591	18.9474

Two things to read. The DE sits above the greedy sweep at every budget – that is what it is for. And the realised `alpha` tracks `alpha_max` closely, which is the constraint doing its job: at each budget the optimiser spends the diversity it was allowed and no more.

11.6 Which strategy to use

Situation	Use
The constraint is inactive (ceiling below your limit)	truncation – nothing to optimise
A per-line cap already meets the constraint	`sel_truncation_cap()` – no matrix needed

Situation	Use
One-off analysis, small pool	`sel_greedy()` swept over w
Production scale, several simultaneous restrictions	`sel_de()` warm-started, on the **line** encoding of Section 8.1

The honest summary is that the optimiser earns its place only in the last row. In the first three, something simpler is either exactly as good or good enough, and Chapter 8.'s cost table is the reason to check which row you are in before writing any optimisation code.

V Evidence and practice

12. What the plant literature establishes

The preceding chapters derived a recommendation. This one asks what the applied literature actually does, where it agrees, and – more usefully – where it is silent.

The corpus behind this chapter is 687 screened plant-breeding records (2017-2026), of which 158 are directly relevant and 197 concern maize, plus 324 animal-breeding records over the same period. Appendix D. describes both and how to query them.

12.1 The headline qualification

Chapter 8. recommended economising to a minimal non-redundant set: θ and $N_{e,\mathrm{par}}$ as constraints, allelic richness at validation. That recommendation stands on its own evidence. **But it is not what plant breeding practice does, and the gap is worth stating plainly.**

metric	records	of which maize	% of 687
Population structure (STRUCTURE/PCA/admixture)	303	87	44.1
Fst / differentiation	117	36	17.0
Expected heterozygosity (He / gene diversity)	101	23	14.7
PIC (polymorphic information content)	97	29	14.1
Modified Rogers / genetic distance	83	36	12.1
Allelic richness / rare alleles	46	6	6.7
Marker coancestry / kinship / relatedness	46	22	6.7
Observed heterozygosity (Ho)	38	5	5.5
Haplotype / pan-genome diversity	37	7	5.4
Shannon / Simpson index	34	3	4.9
Inbreeding coefficient / ROH	22	5	3.2
Effective population size / status number	17	4	2.5

Three observations follow directly.

Applied plant work stacks metrics rather than economising. A representative sub-Saharan maize panel of 376 elite inbreds reports PIC = 0.39, gene diversity = 0.37, MAF = 0.29, Shannon = 6.86, Simpson = 949.09, allelic richness = 787.70 and mean Rogers distance = 0.303 – all in one analysis, presented as a reporting package rather than as competing alternatives. Roughly 20% of relevant records report three or more distinct metric families together.

The metrics recommended here are among the least used. Marker coancestry appears in 6.7% of records, allelic richness in 6.7%, and effective size or status number in 2.5%. Population structure analysis appears in 44% and F_{ST} in 17%. The applied field is oriented toward *describing* structure, not toward *constraining* a loss.

Nothing in the retrieved set computes θ or N_s on a maize hybrid-usage vector. The specific formalism of Section 8.1 is not established in the retrieved set. That is a gap this work fills rather than a contradiction – but it means the approach has **no published plant-breeding baseline to be validated against**, which is a real limitation and should be read as one.

> **i** These are different jobs, not a conflict
>
> For **monitoring and description**, plant practice deliberately reports many metrics because different audiences need different views. For **constraining an optimiser**, redundancy is a cost rather than a virtue.
>
> Report the wide package to collaborators and in publications; constrain the optimiser on the narrow set.

12.2 Why plant practice differs from animal breeding

Three structural facts separate the two literatures, and they explain nearly every divergence in metric choice.

Lines are homozygous. Observed heterozygosity in an inbred panel is a purity control, not a diversity descriptor – pearl millet parental lines report $H_o = 0.031$ against $H_e = 0.334$, and the gap is diagnostic of complete inbreeding rather than informative about the pool. Animal coancestry theory assumes a randomly mating heterozygous population; the plant analogue computes θ directly on fixed line genotypes, which is exactly the setting where the line-level collapse applies.

The product is the F1, not the line. Diversity is tracked separately for each parental pool and for cross-pool complementarity – a structure with no livestock analogue. GWAS models are now built to partition additive and dominance effects by *heterotic group of origin*.

Diversity is managed as two objects, not one. Within-pool and between-pool diversity trend in opposite directions under selection and require opposite management, which no single pool-wide statistic can express – the argument of Chapter 2., arrived at from the applied side.

12.2.1 Distance measures, and the absence of a standard

Modified Rogers distance dominates in maize (36 of 83 distance-reporting records), used chiefly for heterotic assignment and tester choice rather than for diversity monitoring. A recent methods paper works explicitly to unify similarity and distance measures for inbred comparison – the field still treats the choice as open. Recall from Chapter 9. that MRD and coancestry are the same statistic: $f_{ij} = 1 - \mathrm{MRD}_{ij}^2$.

One result is worth carrying into practice: in 298 African highland maize inbreds, **91% of line pairs differ at 30-36% of scored alleles, yet only 32% of pairs have kinship below 0.05.** The authors read the gap as allelic redundancy from repeated use of narrow source germplasm.

> A pool can look diverse by marker counts while being genealogically shallow. That is a direct argument for a coancestry-based constraint over a distance-based one.

Other calibration benchmarks: North American ex-PVP dent inbreds resolve into 8 admixture subgroups aligned with company of origin; China summer maize (490 inbreds, 525,141 SNPs) shows 66.05% of pairwise kinship values near zero; sweet corn shows systematically lower diversity, PIC and MAF than field corn across 514 versus 181 lines.

12.3 Heterotic groups

12.3.1 Assignment is not a solved problem

GCA-based, SCA-based, combined (HSGCA) and SNP-distance classification are compared repeatedly with no consistent winner. Several studies report HSGCA as most consistent with testcross performance; others find SNP-based and testcross-based classification **disagree**. Notably, SSR markers linked to yield QTL give **no advantage** over random SSRs for grouping – against the intuition that linked markers should be more predictive.

This matters for the pipeline: the line-level collapse assumes pool structure is given. Group assignment is itself contested, so the pool-balance constraint inherits whatever uncertainty the assignment method carries. Treat it as an uncertain input, not a fixed given.

12.3.2 The empirical trend that justifies the constraint

The single most load-bearing empirical result in this corpus: in a commercial European dent maize programme, **differentiation between Stiff Stalk and Non-Stiff Stalk groups increased significantly over time while genetic diversity within both groups declined.** This is longitudinal commercial data, not simulation.

It confirms the constraint is doing necessary work rather than solving a hypothetical problem, and it confirms the two-object framing: what erodes is within-pool diversity; what must be protected from erosion *and* from excessive inflation is between-pool divergence. That is exactly the panel `s1_monitor()` logs in Chapter 9..

Why divergence should be maintained rather than maximised now has a mechanistic basis: additive and dominance QTL effects for hybrid performance are **background-specific**, depending on allelic ancestry. Divergence pays through complementarity at specific loci, not through generic distance – so pushing it further does not add hybrid value and risks removing shared favourable alleles. That is Trap 3 of Chapter 9., with a mechanism attached.

The complication worth knowing: deliberate crossing *across* the heterotic boundary for germplasm enhancement generated substantial usable variation, with **10.8% of hybrids from one sub-population exceeding a high-yielding check.** The boundary is maintained for commercial hybrid-parent pools while being selectively porous for pre-breeding.

12.4 Selecting within the pool

12.4.1 The constraint alone is not sufficient

Optimal contribution and optimal cross selection are established across crops. But a load-bearing qualifier appears repeatedly: diversity depletion in closed elite programmes is described as **"ineluctable"** at realistic population sizes even under active management, with bridging populations argued as the necessary remedy.

A θ constraint on the existing pool cannot indefinitely substitute for new germplasm. Plan periodic external injections rather than treating the line pool as closed.

12.4.2 Cycle length, and what goes unreported

Rapid cycling delivers large gain-rate benefits: 12-88% relative gain advantage for oligogenic architectures at 2-3 cycles per year versus one; cycle time cut from 2-5 years to 1 year in intermediate wheatgrass with an explicit diversity-preserving sampling step. A maize two-cycle genomewide selection scheme reports larger gains at equal cost and time but **no diversity metric at all.**

The diversity cost per unit of cycle-time reduction is not established in the retrieved set. Since shortening cycles is the single most common current intervention in commercial programmes, that is a conspicuous hole.

Simulation work on hybrid rice reciprocal recurrent selection reports that larger population sizes give higher gain *and* less depletion of genetic variance – and that diversity is managed at the parental-pool level that feeds hybrids, which is exactly the line-usage level this book constrains.

12.4.3 What is hybrid-specific, and what is not

This distinction decides which literature transfers. **Every usefulness-criterion and progeny-variance study retrieved operates upstream, at the inbred-line development stage** – cross potential selection, UCPC for multi-parental crosses, barley validation against realised populations, soybean family mean and variance prediction, wheat progeny-variance formulas. None operates on F1 hybrids of fixed inbreds.

That confirms rather than challenges the decision to drop the progeny-variance term: the term is real and actively predicted *before* fixation, and vanishes after it. It is why `usefulness_criterion()` belongs at S2 and nowhere else.

One caution that is genuinely hybrid-specific: hybrid **mean** prediction benefits materially from additive-plus-dominance models, since dominance and SCA change predicted performance versus additive-only GEBVs. That is a separate axis from the diversity constraint – it does not reopen the variance term, but it means line-usage optimisation on θ should not be conflated with mean-performance ranking of the same crosses.

12.5 Genebanks, core collections and introgression

Core-subset construction converges on a shared objective – maximise retained diversity at minimum accession count – but diverges on method: GenoCore selecting 310 from 3,300+ sorghum accessions; distance-based optimisation on cassava; rice cores of 247 and 30 from 2,242 accessions; a lettuce core of 268 from 811 capturing 99.4% of total variation.

The quantitatively strongest result: **marker-based core extraction captures 3-78-fold more haplotypic diversity than geography- or environment-based surrogate sampling** across Arabidopsis, Populus and sorghum panels. If cores are being assembled, assemble them on markers.

Maize-specific core-collection algorithm choice is **not established in the retrieved set** – maize genetic-resource work is oriented toward bridging and targeted introgression instead:

- DNA-pool genotyping of 156 landraces (2,340 individuals) against 327 elite lines spanning the 20th century produced two indicators of a landrace's genomic contribution to elite germplasm, used to rank donors rather than introduce material blind.
- Haplotype mining in ~1,000 landrace-derived doubled haploids found landrace haplotypes **outperforming elite alternatives at the same loci**, stable across populations and environments.
- Rapid-cycling genomic selection *within* a landrace population (899 DH lines, 600K array) improves donor material before it contacts the elite pool.
- For capturing landrace variation, the **doubled-haploid route beats gamete capture** – higher prediction accuracy, and no masking of landrace variation behind a capture parent's background.

A θ- or N_s-based accounting of introgression cost against a same-size resampling baseline is not established in the retrieved set.

12.6 The animal-breeding corpus: which matrix to manage

On the animal side there is broad convergence that some form of optimal contribution or optimal cross selection on a **marker-based** coancestry matrix is the right optimisation *framework*. There is **no consensus on which specific genomic matrix** to place inside it, and multiple post-2018 papers report that the choice changes allele-frequency trajectories and diversity outcomes:

- IBD versus homozygosity- versus drift-based matrices give substantially different results unless management is IBD-based (Chapter 6.).
- A deviation-based coancestry matrix retained more genetic variability, while VanRaden's kept allele frequencies closer to the base – a genuine trade-off between "more heterozygosity" and "less frequency drift".
- A linkage-and-pedigree-informed matrix had the highest correlation with true IBD, the most accurate ΔF, and was the only scheme keeping frequency drift down while meeting the inbreeding target.
- ROH-based genomic relationship outperformed SNP-based relationship for limiting inbreeding accumulation at equal gain, in pigs.

The consensus, such as it is: molecular or IBD-flavoured coancestry outperforms simple VanRaden-style relationship for diversity management, but the field has not converged on a single preferred matrix. As Chapter 8. notes, the line-level collapse holds for **any** symmetric $\mathbf{f}^L$, so this open question does not destabilise the computational results.

On allelic richness the post-2018 work sharpens rather than contradicts the recommendation: allelic variation is far more sensitive to bottlenecks than heterozygosity, there is a linear relationship between census-size reduction and reduction in expected number of alleles at drift-mutation equilibrium, and standard N_e corrections do **not** predict allelic loss well. One paper explicitly recommends expanding the $N_e > 500$ conservation guideline to include an allelic-variation criterion.

And on effective size, two cautions beyond $N_s = 1/(2\theta)$: under *active* optimal molecular management, rate-based N_e is numerically pathological – negative in early generations, with no long-term asymptote – and N_e however estimated is informative about heterozygosity loss but not about allelic loss. Both reinforce reporting only a static status-number descriptor under active management.

12.7 Optimisation machinery reported in plants

This is where the corpus is thinnest, and it confirms the computational gap.

- **Optimal contribution methods** are applied across crops, but no abstract states solver type, runtime, or complexity class.
- **Core-subset search** is greedy or distance-based heuristic; only input and output set sizes are reported, never runtime.
- **Evolutionary algorithms** appear mainly *outside* plants. Within plants the closest analogue is a black-box numerical optimiser for progeny allocation, with no named algorithm, problem size, or convergence statistic.
- Problem sizes are reported as candidate-cross counts (330,078 possible barley parent combinations) or collection sizes, never as optimiser performance.

No retrieved plant study applies differential evolution – or any metaheuristic – to an OCS-type hybrid selection problem, and none reports a runtime or complexity comparison against quadratic-programming OCS solvers.

On the animal side, AlphaMate remains the most cited applied tool. The most directly relevant comparison benchmarked genetic algorithms, differential evolution, particle swarm and simulated annealing for parent selection in a 583-line wheat dataset: GA gave the best convergence speed and fitness; **DE was robust but required longer runtime and careful tuning**. That is consistent with Chapter 8.'s finding that the encoding, not the algorithm, is what decides the outcome.

12.8 What remains unestablished

Stated strictly, from the screened abstracts. These are the honest boundaries of what this book can claim support for.

- Application of DE or any metaheuristic to a θ-constrained line-usage selection problem in maize.
- Runtime or complexity comparison between OCS quadratic solvers and metaheuristics for any plant breeding problem.
- Empirical proportional diversity loss measured against a same-size resampling baseline in an operating maize hybrid programme.
- Longitudinal joint reporting of allelic richness alongside θ or N_s for the same hybrid maize programme.
- Hybrid-genome-level (rather than parental-pool-level) coancestry optimisation for F1 maize.
- Whether a dominance-informed term would change optimal-cross outcomes relative to the mean-only approach.
- Quantitative diversity cost per unit of cycle-time reduction under rapid-cycling hybrid maize.
- ROH-based diversity metrics applied to maize hybrid pools (22 records use ROH, only 5 in maize, none for hybrid diversity management).
- Per-evaluation computational cost of diversity metrics, as a function of population size or metric type, anywhere in either corpus.
- Diversity metrics defined on hybrids rather than on lines or parents.
- Combinatorial equal-contribution (0/1) selection compared directly against continuous-contribution designs.

12.9 Consequences for the pipeline

1. Keep the narrow metric set inside the optimiser and report the wide package outside it.
2. Plan for periodic germplasm injection. A closed elite pool depletes regardless of how well the within-pool constraint is managed.
3. Prefer bridging over direct introduction for exotic material, and rank donors by their genomic contribution to the elite pool rather than by raw diversity.
4. Treat heterotic-group assignment as an uncertain input – the pool-balance constraint is only as good as the assignment behind it.
5. Constrain between-pool divergence within a band rather than maximising it.
6. Do not conflate the diversity constraint with mean prediction. Hybrid means need additive-plus-dominance models; the diversity constraint needs neither.
7. Report the diversity cost per cycle explicitly when shortening cycles.
8. If assembling a core or donor set, do it on markers rather than passport data.
9. Watch for allelic redundancy: high marker-based distance can coexist with shallow genealogy, so a coancestry constraint is safer than a distance-based one.
10. Validate on real genotypes before treating the redundancy result as settled.

13. End to end, and the operational checklist

Everything the book has built, run once, on one dataset, in the order a programme would run it. Then the checklist.

13.1 The whole pipeline in one script

```
st1 <- stage1_simulate(book_cfg)
c(hybrids = nrow(st1$X), markers = ncol(st1$X),
  f_min = min(st1$f), f_max = max(st1$f))
#>    hybrids    markers      f_min      f_max
#>  625.00000 2000.00000    0.64475    0.83175
```

```
n_sel <- round(0.10 * nrow(st1$X))
res <- suppressWarnings(stage2_select(
  X = st1$X, f = st1$f, hybrids = st1$hybrids,
  n_sel   = n_sel,
  alphas  = NULL,     # NULL = automatic grid, 0 to the attainable ceiling
  weights = NULL,     # NULL = equal weight across traits
  fL      = st1$fL,   # optional: enables theta_A / theta_B / theta_AB
  B_null = 500, B_full = 80, NP = 100, itermax = 200, verbose = FALSE))
```

i The same pipeline as a script

This chapter runs at the book's reduced scale so it renders in seconds. The identical pipeline at the **production** scale of `sim_config`, writing its tables to `report/`, is `inst/scripts/run_all.R`:

```
Rscript inst/scripts/run_all.R          # ~4 min
Rscript inst/scripts/run_benchmark.R    # ~6 min; the numbers quoted in README.md
```

With real data only the first line changes:

```
st1 <- stage1_build(X = my_hybrid_matrix, ped = my_pedigree,
                    traits = my_predicted_traits, fL = my_line_coancestry)
```

13.1.1 The reference values

```
unlist(res$ref)
#>          gd_ref           gd_se       alpha_max           n_sel population_bias
#>    3.259845e-01    3.556567e-05    3.938109e-02    6.200000e+01    6.467346e-03
```

`alpha_max` is the attainable ceiling from Section 7.9: the loss pure truncation would incur. Any requested budget above it constrains nothing, and `stage2_select()` says so rather than returning a solution that merely looks constrained.

13.1.2 Metrics per scenario

```
fmt(res$metrics[, c("scenario", "alpha", "index", "Ns", "Ne_parents",
                    "Ne_lines_A", "Ne_lines_B", "max_line", "alleles_lost")], 4)
```

scenario	alpha	index	Ns	Ne_parents	Ne_lines_A	Ne_lines_B	max_line	alleles_lost
DE_alpha_0.00	0.0000	1.1004	0.7418	27.4571	12.4805	15.2540	10	63
DE_alpha_0.98	0.0098	1.4141	0.7383	22.3488	9.1962	14.2370	14	81
DE_alpha_1.97	0.0195	1.6767	0.7349	18.0894	7.3359	11.7914	16	106
DE_alpha_2.95	0.0295	1.8523	0.7314	15.8843	5.9505	11.9379	20	116
DE_alpha_3.94	0.0357	1.9600	0.7292	14.3701	5.2948	11.1744	21	125
ref_truncation	0.0394	1.9701	0.7280	13.4877	4.6878	12.0125	23	124
ref_trunc_cap	0.0028	0.7769	0.7408	31.6379	15.7541	15.8843	4	47
ref_max_div	−0.0131	0.1100	0.7465	36.9615	18.6602	18.3048	5	36
ref_random	0.0011	0.0682	0.7414	37.3204	17.9626	19.4141	6	31

Read the `index` column against `alpha`. That is the price list: what each percentage point of diversity costs in index units, on this dataset, at this selection intensity.

Now read `Ne_parents` against `Ns` in the same rows. This is the diagnostic Chapter 6. called for, and the reason `Ne_parents` is a second constraint rather than a report line:

```
d <- res$metrics
fmt(data.frame(scenario = d$scenario,
               Ns = d$Ns,
               Ne_parents = d$Ne_parents,
               Ns_relative = d$Ns / max(d$Ns),
               Ne_par_relative = d$Ne_parents / max(d$Ne_parents)), 3)
```

scenario	Ns	Ne_parents	Ns_relative	Ne_par_relative
DE_alpha_0.00	0.742	27.457	0.994	0.736
DE_alpha_0.98	0.738	22.349	0.989	0.599
DE_alpha_1.97	0.735	18.089	0.984	0.485
DE_alpha_2.95	0.731	15.884	0.980	0.426
DE_alpha_3.94	0.729	14.370	0.977	0.385
ref_truncation	0.728	13.488	0.975	0.361
ref_trunc_cap	0.741	31.638	0.992	0.848
ref_max_div	0.747	36.962	1.000	0.990
ref_random	0.741	37.320	0.993	1.000

The status number barely moves across scenarios while the effective number of parents collapses. A programme reporting only N_s would see a healthy pool and a shrinking parent base it never measured.

13.1.3 Both lenses, and the pool partition

```
fmt(res$metrics[, c("scenario", "alpha", "F_hom", "F_drift", "cov_diag",
                    "rare_retained", "GD_WI_hyb", "GD_BS", "F_ST")], 4)
```

scenario	alpha	F_hom	F_drift	cov_diag	rare_retained	GD_WI_hyb	GD_BS	F_ST
DE_alpha_0.00	0.0000	0.0038	0.0161	−0.0123	0.8636	0.1834	0.0393	0.1205
DE_alpha_0.98	0.0098	0.0132	0.0248	−0.0116	0.8312	0.1829	0.0417	0.1293
DE_alpha_1.97	0.0195	0.0224	0.0348	−0.0124	0.7727	0.1828	0.0452	0.1414
DE_alpha_2.95	0.0295	0.0352	0.0422	−0.0069	0.7143	0.1818	0.0471	0.1490
DE_alpha_3.94	0.0357	0.0377	0.0490	−0.0113	0.7143	0.1821	0.0499	0.1586
ref_truncation	0.0394	0.0412	0.0534	−0.0121	0.7338	0.1822	0.0514	0.1640
ref_trunc_cap	0.0028	0.0154	0.0114	0.0039	0.7273	0.1816	0.0370	0.1138
ref_max_div	−0.0131	−0.0124	0.0063	−0.0187	0.9481	0.1853	0.0362	0.1096
ref_random	0.0011	0.0081	0.0066	0.0015	0.9481	0.1805	0.0357	0.1097

`alpha` constrains the first lens only. `F_drift` is the unmonitored bill, and `cov_diag` is exactly the gap between them – verify it:

```
c(max_deviation_from_Eq3 =
    max(abs(d$cov_diag - (d$F_hom - d$F_drift)), na.rm = TRUE))
#> max_deviation_from_Eq3
#>           6.678685e-17
```

`GD_BS` rising across scenarios is selection concentrating on the most complementary line pairs, which pushes the pools further apart. That is not automatically bad – it is heterosis – but Chapter 9. is the reason to watch it rather than celebrate it.

13.1.4 Distance from the random null

```
fmt(res$z_scores[, c("scenario", "z_GD", "z_Ne_parents", "z_F_drift",
                     "z_rare_retained", "z_GD_BS", "z_ANE")], 2)
```

scenario	z_GD	z_Ne_parents	z_F_drift	z_rare_retained	z_GD_BS	z_ANE
DE_alpha_0.00	0.00	−5.63	7.71	−4.27	6.57	6.96
DE_alpha_0.98	−5.20	−8.56	14.78	−5.69	11.05	9.97
DE_alpha_1.97	−10.35	−11.00	22.79	−8.24	17.33	19.86
DE_alpha_2.95	−15.63	−12.26	28.76	−10.80	20.87	17.76
DE_alpha_3.94	−18.88	−13.13	34.27	−10.80	25.81	16.12
ref_truncation	−20.85	−13.64	37.81	−9.94	28.51	9.80
ref_trunc_cap	−1.47	−3.23	3.95	−10.23	2.44	17.27
ref_max_div	6.91	−0.18	−0.16	−0.57	0.96	5.91
ref_random	−0.59	0.03	0.08	−0.57	0.12	−0.43

Every metric in standard deviations from a random selection of the same size. A metric near zero is not distinguishing this plan from chance, and cannot be serving as a constraint.

13.1.5 The selection table

```
scn <- setdiff(names(res$selection), c(names(st1$hybrids), "n_scenarios"))
head(res$selection[order(-res$selection$n_scenarios),
                   c("hybrid", "line_A", "line_B", scn, "n_scenarios")], 8)
```

hybrid	line_A	line_B	DE_alpha_0.00	DE_alpha_0.98	DE_alpha_1.97	DE_alpha_2.95	DE_alpha_3.94	ref_truncation	ref_trunc_cap	ref_maxdiv	random	n_scenarios
35	10	27	1	1	1	1	1	1	1	1	0	8
232	7	35	1	1	1	1	1	1	1	1	0	8
606	6	50	1	1	1	1	1	1	1	1	0	8
31	6	27	0	1	1	1	1	1	1	1	0	7
32	7	27	1	1	1	1	1	1	1	0	0	7
57	7	28	1	0	1	1	1	1	0	1	1	7
82	7	29	1	1	1	1	1	1	1	0	0	7
90	15	29	0	1	1	1	1	1	1	0	1	7

```r
de_cols <- grep("^DE_", scn, value = TRUE)
c(scenarios           = length(scn),
  chosen_in_all_DE    = sum(rowSums(res$selection[, de_cols, drop = FALSE]) ==
                              length(de_cols)),
  never_chosen        = sum(res$selection$n_scenarios == 0),
  total_candidates    = nrow(res$selection))
#>        scenarios chosen_in_all_DE     never_chosen total_candidates
#>                9               18              386              625
```

`n_scenarios` is the most directly useful column in the whole output. Hybrids chosen under **every** diversity budget are robust to the choice of α – they are the plan's backbone, and the argument for them does not depend on a policy number nobody wants to defend. Hybrids that appear only at the loosest budget are the ones being bought with diversity.

13.2 Reading the frontier

```r
plot(d$alpha * 100, d$index, type = "b", pch = 19, col = "#b2182b", lwd = 2,
     xlab = "gene diversity loss, alpha (%)", ylab = "mean index (s.d.)")
text(d$alpha * 100, d$index, d$scenario, pos = 4, cex = 0.65, col = "grey30")
abline(v = 0, lty = 3)
grid()
```

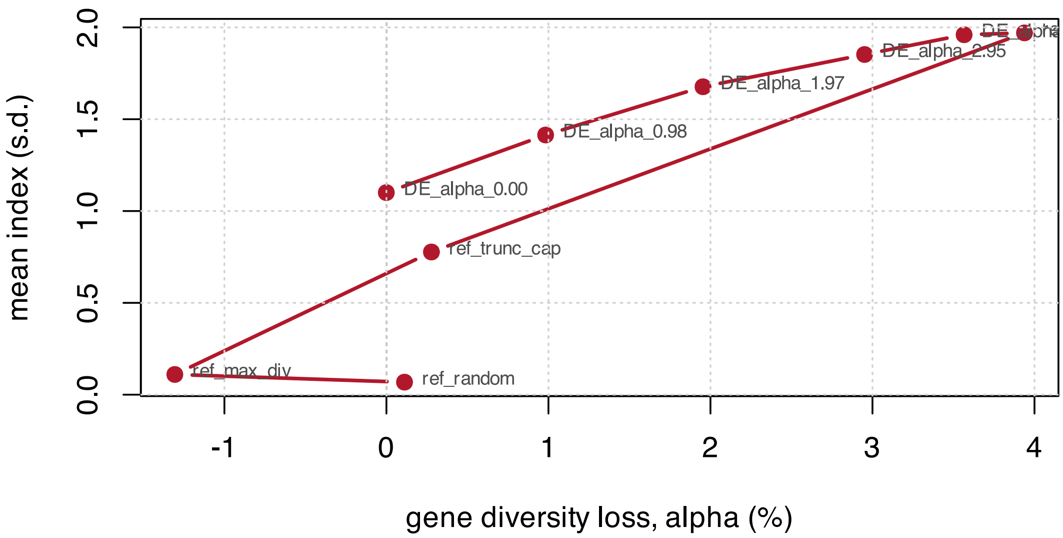

Figure 13.1: The price list. Each point is one diversity budget; the curve is what index it buys.

Choose the knee, not the endpoint. Past the knee, each additional unit of index costs disproportionately more diversity, and the diversity is not recoverable next cycle while the index gain is realised once.

13.3 What this simulation cannot tell you

Stated plainly, because the numbers above are precise and precision invites more confidence than the model supports.

The GEBVs carry no prediction error. The traits are built from *known* marker effects (Chapter 3.). Real predicted values have error, that error is correlated with relatedness to the training set, and it will erode the index advantage of every strategy here – probably truncation most, since it selects hardest on the predicted value.

The model is additive. An additive $\mathbf{G}$ misses heterosis and specific combining ability, which is precisely what a hybrid programme sells. The diversity constraint does not need a dominance model; the *mean-performance ranking* does. Do not conflate them, and do not use these `index` values as performance predictions.

The pools are symmetric by construction. Both are simulated from the same ancestral frequencies at the same F_{ST}. Real heterotic pools differ in size, history and internal structure, and a real programme will find one pool eroding faster than the other – which is why `ne_lines_pool()` is reported per pool.

N_s is not a drift projection. It is a static descriptor (Section 7.3). Do not extrapolate it forward.

One cycle, not a trajectory. Every result here is a single round of hybrid selection. Chapter 9. showed that a constraint applied at one stage buys stage-level diversity, not end-of-pipeline richness – and the same logic applies across cycles. A one-cycle constraint is not a multi-cycle policy.

p_0 must be frozen. In this book it is set once in `build_ctx()`. In a real multi-cycle programme it must be the founder cycle's frequencies, propagated forward – the single most common error in applying `F_hom` and `F_drift` (Chapter 6.).

13.4 The operational checklist

13.4.1 Setting up

- The base p_0 is frozen at the founding cycle and documented.

- Diversity is computed on molecular coancestry `f`, never on a centered `G`. Check `sum(G) == 0` once, to see why.
- The marker panel's MAF spectrum is known – it decides how severely the two lenses diverge.
- Heterotic-group assignment is recorded, with the method that produced it, and treated as uncertain.

13.4.2 Choosing the budget

- The attainable α ceiling is computed *before* choosing $\alpha_{\max}$. A budget above the ceiling constrains nothing.
- Loss is measured against a **same-size resampling baseline**, never against the full pool.
- The constraint is applied at the target set size, not incrementally.
- The frontier is traced and the knee chosen, rather than a round number.

13.4.3 Running the optimiser

- The search is encoded on **line weights**, not hybrid keys.
- The differential evolution is **warm-started** from greedy solutions.
- $N_{e,\text{par}}$ is a second constraint, not a report line.
- The per-line usage cap is enforced in the decoder, not by penalty.
- The result is checked against the continuous relaxation. A large gap means the optimiser has not converged; a slack constraint means the search never reached the boundary.

13.4.4 Validating the plan

- `F_hom` and `F_drift` are computed **separately** and reported together, with `cov_diag` as the diagnostic.
- Allelic richness is computed on the returned plan – it is the one metric carrying information θ does not.
- The within/between pool decomposition confirms diversity was not preserved by degrading pool separation.
- The number of selected parent lines is biologically plausible.
- N_s is reported; a rate-based N_e is **not**, under active management.
- Every metric is reported against the random null, in standard deviations.

13.4.5 Across cycles

- The per-stage loss ledger is kept. DH induction, not hybrid mining, is usually where the diversity goes.
- The regression of $\log(1 - F)$ on cycle is checked for linearity. Curvature means a non-constant rate.
- Periodic germplasm injection is planned. A closed elite pool depletes however well the constraint is managed.
- The diversity cost per cycle is reported when cycle time is shortened.

13.5 Where to go next

Three extensions the material here supports but does not cover, each needing new work rather than new derivation:

Genomic prediction of the index. The book treats `ctx$index` as given. How it is predicted, and what its error does to the selection, closes the loop.

Heterosis and specific combining ability. An additive `G` misses the product. A dominance-informed mean model would change which crosses are ranked highest, without reopening the diversity term.

Real data ingestion. `load_data()` is the single swap point and is currently a stub. A worked path from a VCF to the `X/f/hybrids` contract is what most readers will need first.

Appendices

A. Notation and glossary

A.1 Symbols

Symbol	Meaning	In the code
N	candidate hybrids in the pool	`ctx$N`
n, n_{sel}	hybrids selected	`n_sel`
L	parent lines	`ctx$n_lines`
m	markers	`ctx$m`
x_{ik}	within-individual reference-allele frequency of individual i at marker k, in 0/0.5/1	`ctx$X[i, k]`
$\mathbf{X}$	hybrid marker matrix, N x m	`ctx$X`
$\mathbf{M}$	hybrid dosage matrix, 0/1/2	-
$\mathbf{G}^L$	line genotype matrix, L x m, coded 0/1	`ctx$GL`
p_k, $\bar{p}_k$	allele frequency; the bar denotes a group mean	-
$p_{0,k}$	**frozen** candidate-pool allele frequency: the base	`ctx$p0`

A.1.1 Matrices

Symbol	Meaning	Note
$\mathbf{f}$	molecular coancestry of the **hybrids**, N x N, in [0, 1]	the diversity kernel; never centered
$\mathbf{f}^L$	molecular coancestry **between lines**, L x L	the sufficient statistic in disguise – see the collapse
$\mathbf{G}$	VanRaden genomic relationship, centered – prediction only	sum of all elements is exactly 0; unusable for diversity
$\mathbf{A}$	pedigree relationship	expectation of IBD, no markers
$\mathbf{G}_{0.5}$	VanRaden with the centering frequency fixed at 0.5	**this is f**, arrived at from the VanRaden family

A.1.2 Coancestry and diversity

Symbol	Definition	Function
θ_S	$n^{-2}\sum_i \sum_j f_{ij}$, diagonal **included**	`theta_group()`, `fast_theta()`
GD, H_e	$1-\theta$; these are the same quantity	`gene_diversity()`, `he_nei()`
N_s	$1/(2\theta)$	`status_number()`

Symbol	Definition	Function
$N_{e,\mathrm{par}}$	$1/\sum_i p_i^2$ over parent-line usage	`ne_parents()`
f_{ii}	coancestry within subpopulation i; $1 - f_{ii} = H_{e,i}$	`ct_partition()`
s_i	self-coancestry; $F_i = 2s_i - 1$	`ct_partition()`
$\tilde{x}$	weighted mean **within** subpopulations	-
$\bar{x}$	mean over **all pairs**, between-subpopulation pairs included	-

Always $\bar{f} \leq \tilde{f} \leq \tilde{s}$. The differences between those three numbers *are* the partition of diversity.

A.1.3 The partition

Component	Definition	Meaning	In hybrid breeding
GD_T	$1 - \bar{f}$	total	the pool's diversity
GD_{WI}	$1 - \tilde{s}$	within individuals	0 for inbred lines; use `GD_WI_hyb` for the F1s
GD_{BI}	$\tilde{s} - \tilde{f}$	between individuals, within subpopulation	segregating variation available to selection
GD_{WS}	$1 - \tilde{f}$	within subpopulations	what erodes – defend it
GD_{BS}	$\tilde{f} - \bar{f}$	between subpopulations	the heterosis engine – **maintain**, do not maximise

with $GD_T = GD_{WI} + GD_{BI} + GD_{BS}$ and $F_{ST} = GD_{BS}/GD_T$.

A.1.4 Inbreeding, the three lenses

Symbol	Lens	Definition	Function
F_{hom}	homozygosity	$1 - \overline{H_t/H_0}$; **can be negative**	`freq_metrics()`
F_{drift}	drift	$\overline{\delta p^2/[p_0(1-p_0)]}$; never negative	`freq_metrics()`
F_{IBD}	identity by descent	segments inherited from a defined base	requires pedigree + linkage
F_{ROH}	hybrid	genome fraction in runs of homozygosity	not implemented
cov	-	$2\,\mathrm{cov}(\delta p/s, (p_0 - \frac{1}{2})/s) = F_{hom} - F_{drift}$	`freq_metrics()$cov_diag`

A.1.5 Selection and optimisation

Symbol	Meaning	Function
u_i	multi-trait index of hybrid i	`ctx$index`
α	$(GD_{\mathrm{ref}} - GD_S)/GD_{\mathrm{ref}}$; negative means more diverse than chance	`alpha_loss()`
$\alpha_{\max}$	the diversity budget – a policy decision	argument to `stage2_select()`

Symbol	Meaning	Function
$\mathbf{c}$	continuous contributions, summing to 1	`ocs_quadprog()`
$\mathbf{w}$	line-usage weights, $w_a = k_a/(2n)$	`line_weights()`, `line_usage_freq()`
k_a	times line a is used as a parent	`line_counts()`
λ	weight on the coancestry penalty in continuous OCS	`ocs_relaxation()`
σ^2_{DH}	progeny variance of a DH family	`sigma2_dh()`
UC	usefulness criterion, $\mu + ih\sigma$	`usefulness_criterion()`

A.1.6 Population sizing

Symbol	Meaning	Function
H	panel heterozygosity; 0.125 in an F4	`panel_het()`
D	diversity retained, $\overline{2p(1-p)}/0.5$	`diversity_retained()`
N_{eq}	$(1-H)/(1-D)$ – a communication device, **not** a Wright Ne	`n_equivalent()`
L (Morgans)	total map length	`map$total_cM / 100`
A_t	junctions per haplotype at generation t	`expected_junctions()`
r	recombination fraction, Haldane	`build_map()`

A.2 Conventions

Coancestry always includes the diagonal. Group coancestry in the sense of Cockerham. That is what makes $1-\theta$ exactly Nei's H_e and makes the algebra close. Drop the diagonal and none of the identities hold.

Everything is computed per locus and then averaged over loci. Never average frequencies first.

Marker coding. Lines are 0/1 (or 0/2) because they are homozygous. Hybrids are 0/1/2 as dosage, rescaled to 0/0.5/1 as within-individual frequency. `stage1_build()` auto-detects and rescales.

`f` is never centered, `G` always is. A negative coancestry is a symptom of the wrong matrix.

Loss is always relative to a same-size random baseline. Never to the full pool.

A.3 Terms

Balding-Nichols model. A Beta distribution with mean p and variance $F_{ST}\,p(1-p)$, used to draw pool-specific allele frequencies from an ancestral frequency. The parameter *is* the expected divergence.

Group coancestry. The probability that two alleles sampled at random from a whole set are identical in state, including the case of sampling the same individual twice.

Heterotic group / pool. A set of lines maintained as a breeding unit, crossed to lines of a complementary group to produce hybrids. Divergence between groups is what generates heterosis.

Identity by state (IBS) vs by descent (IBD). IBS is what markers observe: two alleles are the same allele. IBD adds the requirement of common ancestry from a defined base. The gap between them is where Chapter 6.'s whole argument lives.

Line-level collapse. The exact identity $\theta_S = \mathbf{w}'\mathbf{f}^L\mathbf{w}$ for F1 hybrids of homozygous lines, which removes the need for the $N \times N$ hybrid matrix. See Section 8.1.

Status number. $N_s = 1/(2\theta)$, the number of unrelated non-inbred individuals with the same gene diversity. A static descriptor.

Truncation selection. Take the top n by index, ignore everything else. The ceiling on gain and the floor on diversity.

Warm start. Seeding an optimiser's initial population with heuristic solutions. Not optional here: without it the differential evolution finishes below a plain greedy.

B. The hybdiv package

Every function used in this book lives in one package, whose source is the `R/` directory of this repository. This appendix is the index.

```
#> exported_objects
#>              141
```

B.1 Installation

```
install.packages(c("DEoptim", "quadprog"))
devtools::install(".")        # from the repository root
library(hybdiv)
```

Only `DEoptim` is a hard dependency, used solely by `sel_de()`. `quadprog` is suggested and needed only for `optimal_contributions()` and `ocs_quadprog()`. Everything else, including every plot, is base R.

B.2 File map

File	Role	Chapter
`R/00_data.R`	simulation, the two matrices, the `ctx` builder	Chapter 3.
`R/01_metrics.R`	the diversity metrics, all `f(idx, ctx) -> scalar`	Chapter 7.
`R/02_benchmark.R`	null distribution, discriminatory power, cost, plots	Chapter 7.
`R/03_de_select.R`	selection strategies: truncation, greedy, DE, OCS relaxation	Chapter 11.
`R/04_fast_metrics.R`	the line-level fast route – no N x N matrix	Chapter 8.
`R/05_pipeline_metrics.R`	stage-specific metrics for S1 to S4	Chapter 9.
`R/06_f2size.R`	F1 to F4 population sizing simulator	Chapter 4.
`R/07_caballero_toro.R`	Caballero & Toro subdivided-population analysis	Chapter 2.
`R/09_reference.R`	slow literal oracles, for verification	Appendix C.
`R/10_stage1.R`	stage 1: lines to the X / f / hybrids contract	Chapter 3.
`R/11_stage2.R`	stage 2: constrained selection across an alpha grid	Chapter 13.

Files are numbered because the package has no `Collate` field and sourcing order is alphabetical.

B.3 The two-stage contract

Everything downstream of stage 1 talks only through three objects. This is the seam for swapping in real data.

```
st1 <- stage1_simulate()                 # or stage1_build(X, ped, traits, fL)
#   $X        N x m hybrid markers, 0/0.5/1
#   $f        N x N molecular coancestry
#   $hybrids  N x (3 + n_traits): id, line_A, line_B, traits
#   $fL       L x L line coancestry (optional)
```

```
res <- stage2_select(st1$X, st1$f, st1$hybrids, n_sel = 100, fL = st1$fL)
#   $selection  0/1 column per scenario, plus n_scenarios
#   $metrics    one row per scenario
#   $z_scores   each metric in s.d. from the random null
#   $ref        gd_ref, its Monte Carlo s.e., alpha_max, n_sel
```

B.4 Function index

B.4.1 Simulation and context

Function	Does
`sim_config`	simulation scale: lines per pool, markers, traits, divergence
`simulate_lines()`	Balding-Nichols line genotypes in two divergent pools
`simulate_data()`	lines, all A x B crosses, both matrices, predicted traits
`simulate_pools()`	lighter simulator returning raw pieces, with `GL` in 0/1
`build_ctx()`	assemble the context; freezes `p0` and the index
`build_ctx_from_stage1()`	same, from the stage-1 contract
`ctx_from_pools()`	context from `simulate_pools()`, carrying `parents` and `GL`
`load_data()`	the single swap point for real data
`stage1_simulate()` / `stage1_build()`	produce the stage-1 contract
`stage2_select()`	the whole constrained selection pipeline

B.4.2 The two matrices

Function	Does
`molecular_coancestry()`	**the diversity kernel**; one `tcrossprod`, stays in [0, 1]
`vanraden_G()`	**prediction only**; `sum(G) == 0` by construction
`mrd_matrix()` / `coancestry_from_mrd()`	modified Rogers distance, the same statistic

B.4.3 Diversity metrics – all `f(idx, ctx) -> scalar`

Function	Does
`theta_group()`	group coancestry, diagonal included
`gene_diversity()` / `he_nei()`	`1 - theta`; the same quantity, two routes
`theta_from_freq()`	theta via allele frequencies – validation only, ~100x slower
`status_number()`	`Ns = 1/(2 theta)`; a static descriptor
`ne_parents()`	effective number of parent lines; independent of `f`
`ne_lines_pool()`	the same, per pool; needs only the pedigree
`alleles_lost()`	markers polymorphic in the pool, rare in the selection
`freq_metrics()`	**one marker sweep**: He, F_hom, F_drift, cov_diag, rare_retained

Function	Does
`line_weights()` / `theta_pools()`	per-pool usage weights and coancestries
`gd_partition()`	Caballero & Toro Eq. 8 over the heterotic pools
`ene()` / `ane()`	Core Hunter entry- and accession-to-nearest-entry
`eff_dim()`	effective dimensionality, on the **double-centered** submatrix
`gst()`	Nei's Gst, selected against non-selected
`mean_offdiag()`	the common production metric, kept for comparison
`metrics_cheap()` / `metrics_full()`	the inner-loop panel and the post-hoc panel

B.4.4 The fast line route

Function	Does
`fast_ctx()`	add `parents`, `GL` and the packed bitset to a context
`line_counts()` / `line_usage_freq()`	the sufficient statistic
`fast_theta()` / `fast_gene_div()` / `fast_status_num()`	`theta = w' fL w`; no N x N matrix
`make_theta_fun()`	precompiled closure for ~1e6 evaluations
`theta_state()` / `theta_swap()`	O(L) incremental update per swap
`fast_ne_parents()` / `ne_parents_from_counts()`	free once the counts exist
`fast_pbar()` / `fast_he_nei()` / `fast_alleles_lost()`	frequencies from the line matrix
`pack_lines()` / `popcount32()` / `alleles_retained_fast()`	bitset allelic richness
`fast_theta_pools()`	pool decomposition from the same counts
`decode_rank()` / `decode_lines()`	the two encodings; **use `decode_lines()`**
`make_fitness_fast()`	DE fitness with several simultaneous restrictions

B.4.5 Selection strategies

Function	Does
`sel_random()`	the null strategy
`sel_truncation()`	ceiling on gain, floor on diversity
`sel_truncation_cap()`	per-line cap; no matrix needed, usually most of the benefit
`sel_greedy()`	incremental greedy; `w = 0` is max diversity
`decode()` / `make_fitness()` / `sel_de()`	differential evolution, warm-started by default
`project_capped_simplex()` / `ocs_relaxation()`	the continuous upper bound

Function	Does
optimal_contributions() / ocs_quadprog()	quadratic-programming OCS

B.4.6 Benchmarking

Function	Does
null_distribution()	**the correct baseline**: random subsets of the same size
alpha_loss()	proportional diversity loss against that baseline
evaluate() / discriminatory_power()	metric panel and its z-scores
cost_per_eval()	microseconds per metric; decides what goes inside the loop
plot_null() / plot_correlation() / plot_frontier()	base-R diagnostics; run by inst/scripts/run_benchmark.R, not by the book

B.5 Runnable scripts

The book runs everything at a reduced scale so it renders quickly. Two scripts ship with the package and run the same work at the production scale of sim_config, writing their tables and plots to report/:

```
Rscript inst/scripts/run_all.R          # the two-stage pipeline, ~4 min
Rscript inst/scripts/run_benchmark.R    # the metrics benchmark, ~6 min
```

run_benchmark.R is the source of the numeric results quoted in README.md, and the only place the three plot_* helpers above are exercised. From an installed copy of the package:

```
source(system.file("scripts", "run_all.R", package = "hybdiv"))
```

B.5.1 Pipeline stages S1-S4

Function	Does
theta_decompose() / gst_nei()	**both** F_ST definitions, and Nei's Gst
pool_complementarity() / s1_monitor()	the S1 cycle panel
dh_sib_coancestry() / dh_cross_coancestry() / dh_pool_theta()	the three DH identities
sigma2_dh() / sigma_dh_crosses()	progeny variance from marker effects
usefulness_criterion() / sel_intensity()	UC – applies at S2, not S4
s3_theta() / alleles_retained()	line-level metrics
loss_vs_resampling() / theta_ceiling()	calibrate against a same-size baseline
select_lines_constrained()	truncate-then-repair; the incremental greedy fails
hybrid_theta_from_usage() / line_usage() / pool_balance()	S4 on the line route
loss_ledger()	loss per stage, on one scale

B.5.2 Caballero & Toro

Function	Does
`ct_partition()`	subpopulation coancestries, F statistics, the Eq. 7 partition
`ct_loss_gain()`	loss **or gain** from removing each subpopulation

B.5.3 Population sizing

Function	Does
`maize_chr_len` / `f2_distorters` / `f2_opt`	defaults; **replace the distorters** with your data
`build_map()` / `marker_index()`	Haldane map on a regular grid
`meiosis()` / `meiosis_sel()` / `cross_indices()` / `make_F1()`	the vectorised engine
`make_distortion()` / `zyg_fitness()`	gametophytic and zygotic distortion, by exact rejection
`run_scheme()` / `reps_for_N()` / `bulk_weights()`	SSD, bulk, Syn schemes
`panel_freq()` / `panel_het()` / `diversity_retained()`	panel metrics
`min_minor_freq()` / `fixed_fraction()` / `skew_fraction()`	worst-region metrics
`n_equivalent()`	equivalent N – a communication device, **not** a Wright Ne
`block_lengths()` / `junctions_per_hap()` / `expected_junctions()`	map expansion
`multilocus_recovery()` / `f1_sizing()`	the objectives that keep paying for larger N
`f2_run_experiment()`	the full grid; minutes

B.5.4 Reference oracles

Function	Does
`ref_theta()`, `ref_gene_div()`, `ref_theta_freq()`, `ref_he_nei()`	literal transcriptions
`ref_status_num()`, `ref_ne_parents()`, `ref_alleles_lost()`, `ref_alleles_fixed()`	-
`ref_ene()`, `ref_ane()`, `ref_eff_dim()`, `ref_theta_pools()`	-

B.6 Full export list

alleles_lost	fast_gene_div	metrics_full	reps_for_N
alleles_retained	fast_he_nei	min_minor_freq	run_scheme
alleles_retained_fast	fast_ne_parents	molecular_coancestry	s1_monitor
alpha_loss	fast_pbar	mrd_matrix	s3_status_num

ane
BITS_PER_WORD
block_lengths
build_ctx
build_ctx_from_stage1
build_map
bulk_weights
coancestry_from_mrd
col_cumsum
cost_per_eval
cross_indices
ct_loss_gain
ct_partition
ctx_from_pools
decode
decode_lines
decode_rank
dh_cross_coancestry
dh_pool_theta
dh_sib_coancestry
discriminatory_power
diversity_retained
eff_dim
ene
evaluate
expected_junctions
f1_sizing
f2_distorters
f2_opt
f2_run_experiment
fast_alleles_lost
fast_ctx
fast_status_num
fast_theta
fast_theta_pools
fixed_fraction
freq_metrics
gd_partition
gene_diversity
gst
gst_nei
he_nei
hybrid_theta_from_usage
junctions_per_hap
line_counts
line_usage
line_usage_freq
line_weights
load_data
loss_ledger
loss_vs_resampling
maize_chr_len
make_distortion
make_F1
make_fitness
make_fitness_fast
make_theta_fun
marker_index
max_line_use
mean_index
mean_offdiag
meiosis
meiosis_sel
metrics_cheap
multilocus_recovery
n_equivalent
ne_lines_pool
ne_parents
ne_parents_from_counts
null_distribution
ocs_quadprog
ocs_relaxation
optimal_contributions
p_het
pack_lines
panel_freq
panel_het
plot_correlation
plot_frontier
plot_null
pool_balance
pool_complementarity
popcount32
project_capped_simplex
ref_alleles_fixed
ref_alleles_lost
ref_ane
ref_eff_dim
ref_ene
ref_gene_div
ref_he_nei
ref_ne_parents
ref_status_num
ref_theta
ref_theta_freq
ref_theta_pools
s3_theta
sel_de
sel_greedy
sel_intensity
sel_random
sel_truncation
sel_truncation_cap
select_lines_constrained
sigma_dh_crosses
sigma2_dh
sim_config
simulate_data
simulate_lines
simulate_pools
skew_fraction
stage1_build
stage1_simulate
stage2_select
status_number
theta_ceiling
theta_decompose
theta_from_freq
theta_group
theta_pools
theta_state
theta_swap
usefulness_criterion
vanraden_G
zyg_fitness

C. Verification

A derivation the reader cannot re-run against a naive implementation is a claim, not a verification. This appendix says how each result in the book was checked, and how to re-run the checks.

C.1 The method: slow oracles

Every fast route in the package has a **deliberately slow, literal** counterpart in `R/09_reference.R` that transcribes the definition with no algebraic shortcut. The fast route must match it to machine precision.

```
ref_theta
#> function (idx, ctx)
#> mean(ctx[["f"]][idx, idx])
#> <bytecode: 0x97d7f6cf0>
#> <environment: namespace:hybdiv>
theta_group
#> function (idx, ctx)
#> mean(ctx$f[idx, idx])
#> <bytecode: 0x97d8301c8>
#> <environment: namespace:hybdiv>
fast_theta
#> function (idx, ctx)
#> {
#>     cnt <- line_counts(idx, ctx)
#>     u <- which(cnt > 0L)
#>     cu <- cnt[u]
#>     s <- sum(cu)
#>     as.numeric(crossprod(cu, ctx$fL[u, u, drop = FALSE] %*% cu))/(s *
#>         s)
#> }
#> <bytecode: 0x97d865b60>
#> <environment: namespace:hybdiv>
```

Three implementations of one quantity: a literal mean over the submatrix, the packaged version, and the line-route collapse that never forms the submatrix at all. They must agree.

```
sim  <- simulate_pools(n_A = 20, n_B = 20, m = 1500, seed = 7)
ctxp <- ctx_from_pools(sim)
set.seed(3); sel <- sample.int(ctxp$N, 50)

c(oracle     = ref_theta(sel, ctxp),
  packaged   = theta_group(sel, ctxp),
  line_route = fast_theta(sel, ctxp),
  max_dev    = max(abs(c(theta_group(sel, ctxp), fast_theta(sel, ctxp)) -
                       ref_theta(sel, ctxp))))
#>       oracle     packaged   line_route      max_dev
#> 6.708316e-01 6.708316e-01 6.708316e-01 1.110223e-16
```

The oracles exist so that this comparison is possible for a reader who does not trust the derivation – which is the correct posture.

```
fmt(rbind(
  he_nei       = c(oracle = ref_he_nei(sel, ctxp), packaged = he_nei(sel, ctxp)),
  theta_freq   = c(ref_theta_freq(sel, ctxp), theta_from_freq(sel, ctxp)),
  status_num   = c(ref_status_num(sel, ctxp), status_number(sel, ctxp)),
  ne_parents   = c(ref_ne_parents(sel, ctxp), ne_parents(sel, ctxp)),
```

```
  alleles_lost = c(ref_alleles_lost(sel, ctxp), alleles_lost(sel, ctxp)),
  ENE          = c(ref_ene(sel, ctxp), ene(sel, ctxp)),
  ANE          = c(ref_ane(sel, ctxp), ane(sel, ctxp))), 10)
#> [[1]]
#> [1] 0.3291684
#>
#> [[2]]
#> [1] 0.6708316
#>
#> [[3]]
#> [1] 0.7453435
#>
#> [[4]]
#> [1] 26.45503
#>
#> [[5]]
#> [1] 43
#>
#> [[6]]
#> [1] 0.2581333
#>
#> [[7]]
#> [1] 0.2503233
#>
#> [[8]]
#> [1] 0.3291684
#>
#> [[9]]
#> [1] 0.6708316
#>
#> [[10]]
#> [1] 0.7453435
#>
#> [[11]]
#> [1] 26.45503
#>
#> [[12]]
#> [1] 43
#>
#> [[13]]
#> [1] 0.2581333
#>
#> [[14]]
#> [1] 0.2503233
```

C.2 The identity sweep

The line-level collapse was checked across $F_{ST} \in \{0.05, 0.15, 0.35\}$, two seeds and two selection sizes, thirty replicate selections each.

	Identity	Worst deviation over the sweep
max_pairwise_f	pairwise f: hybrid vs line formula	1.1e-16
max_theta_sub_vs_line	theta: line route vs f submatrix	2.2e-16
max_theta_sub_vs_freq	theta: line route vs allele-frequency route	1.1e-16
max_pbar	selection allele frequencies	0 (exact)

	Identity	Worst deviation over the sweep
max_He	Nei He	0 (exact)
max_Neparents	effective number of parents	2.1e-14
max_alleles_lost	allele loss at MAF threshold	0 (exact)
max_theta_pools	pool decomposition	1.2e-15
max_swap_update	incremental swap vs full recompute	4.4e-16
bitset_vs_literal	bitset allelic richness vs literal count	0 (exact)

C.3 The test suite

The suite is `tests/testthat/`, run with `devtools::test()`. It takes about a minute. What it asserts, and why each check earns its place:

Check	If it fails
theta via submatrix == theta via allele frequencies	**f is wrong**: centered, rescaled, or the wrong marker coding
1 - theta == Nei's He	the identity underpinning every 'loss of X%' statement is broken
f is bounded to [0, 1]	a centered matrix is being used as coancestry
sum(VanRaden G) == 0	the demonstration in Section 1.2 no longer holds on this data
theta and Ns analytical edge cases	the diagonal is being dropped somewhere
greedy < random < truncation on theta	there is no trade-off to optimise; the whole benchmark is pointless
decode() is unique, in range, deterministic	the DE fitness is noisy and cannot converge
fast line route == hybrid-matrix route	**the collapse is broken**; production scale becomes infeasible
incremental swap == full recomputation	any local search built on the state vector is wrong
popcount32 == naive bit count	allelic richness counts are wrong
F_hom and F_drift are 0 when the selection is the base	the base p0 is not frozen, or is being recomputed
cov_diag == F_hom - F_drift	one of the three lens formulas is wrong
max-diversity drives F_hom negative, F_drift positive	the two-lens argument of Chapter 6. does not reproduce
freq_metrics reproduces he_nei and alleles_lost	the consolidated marker sweep changed an existing number
the Caballero & Toro partition closes	the Eq. 8 algebra is broken
GD_WI == 0 for inbred lines, GD_WI_hyb large for their F1s	lines are not actually inbred, or the hybrid diagonal is wrong
eff_dim in (0, n - 1]	the double-centering was dropped and eff_dim re-measures theta

Check	If it fails
the two-stage output contract	downstream code reading `$selection` or `$metrics` will break
stage 2 runs without fL	the optional-fL path is broken
stage1_build accepts both marker codings	real data in the other coding would give different answers
MRD and coancestry are one statistic	Trap 1 of Chapter 9. no longer holds
Gst < Fst always, and Gst matches the classical route	Trap 2 is wrong, or the two definitions were conflated
the DH identities	the S2 algebra is wrong and crossing plans cannot be pre-evaluated
hybrid theta from usage == the hybrid matrix	Trap 5′s repair route is broken
constrained line selection meets its ceiling	the constrained selection silently returns an infeasible plan
the oracles agree with the packaged metrics	a fast route diverged from its definition
map length and Haldane spacing	the sizing chapter's map is not the map it claims
heterozygosity halves per selfing	the simulator is not doing what selfing does
D == 1 - (1 - H)/N under neutrality	the identity every sizing number rests on is broken
n_equivalent inverts diversity_retained	N_eq is not the inverse it is described as
a Syn cycle adds half a map length of junctions	the intercrossing budget in Chapter 4. is wrong by 2x
distortion shifts ancestry away from 0.5	the distortion mechanism is inactive
f1_sizing halves the loss probability per plant	the F1 sizing argument is wrong

```
Rscript -e 'devtools::test()'
```

C.4 Provenance of every figure and table

The book runs light examples live and reads heavy results from cache. This is the complete accounting.

C.4.1 Computed live at render

Chapter	Runs live
Chapter 1.	both matrices, the sum(G) demonstration, the f diagonal
Chapter 2.	the whole worked example, reproduced against the published tables
Chapter 3.	the simulation, the fst sweep, the contract, the sampling-bias histogram
Chapter 4.	the theoretical identities, heterozygosity halving, map expansion, f1_sizing
Chapter 6.	the Eq. 3 identity, the four-strategy lens panel
Chapter 7.	every metric, the sampling-bias formula, the null distribution
Chapter 8.	the collapse, the swap update, cost at book scale, the fixed-theta band
Chapter 9.	all five traps, the S1 panel, the DH identities, the S4 collapse
Chapter 10.	the OCS relaxation, the convergence check, the greedy frontier

Chapter	Runs live
Chapter 11.	all baselines, the encodings, the DE, the warm-start comparison
Chapter 13.	the full two-stage pipeline
Appendix C.	the oracle comparisons

C.4.2 Read from cache

File	Why
`data/theta_identity_check.csv`	the full identity sweep, 12 configurations
`data/cost_per_eval.csv`	production scale: L = 300, N = 22500, m = 10000
`data/metric_redundancy.csv`	621 selections x 17 metrics
`data/de_constraints.csv`	9 DE runs at 60,000 evaluations each
`data/fst_definitions.csv`	the divergence sweep
`data/dist_vs_variance.csv`	600 candidate crosses with simulated DH
`data/pipeline_loss_ledger.csv`	8 founder pools, four stages, two strategies
`data/plant_metric_usage.csv`	687 screened plant records
`data/bibliography_*.csv`	the screened corpora themselves
`data/f2_results.csv`	5 schemes x 20 sizes, adaptive replication
`data/f2_multilocus.csv`	multilocus recovery across the grid
`data/mabc_*.csv`	the MABC grid: 4 policies x 4 gene numbers x 11 sizes x 3 s, 300 reps
`figs/mabc_sizing.png`	analytical sizing by stage
`figs/mabc_policies.png`	the four recombinant-selection policies
`figs/cost_scaling.png`	cost scaling at production scale
`figs/de_convergence.png`	DE convergence under four restrictions
`figs/metric_redundancy.png`	the redundancy panels
`figs/pipeline_loss.png`	the loss ledger
`figs/metric_pitfalls.png`	Traps 4 and 2 on the full simulation

Nothing is read from `report/` or `data/` at the repository root: both are gitignored, so a clean clone would fail to render. Everything cached lives in `book/data` and `book/figs`, which are tracked.

C.5 Regenerating the cached results

```
Rscript book/scripts/regenerate.R          # everything; hours
Rscript book/scripts/regenerate.R f2size   # one target
```

The script is the single place where the expensive computations live. It is never invoked by `quarto render`.

C.6 Reproducing the book

```
Rscript -e 'devtools::document(); devtools::load_all()'
Rscript -e 'devtools::test()'
Rscript -e 'devtools::check()'
quarto render book
```

Quarto's `freeze: auto` means a chapter re-runs only when its source changes. To force a full recomputation, delete `book/_freeze/`.

C.7 Known limitations of the verification

Stated so they are not mistaken for oversights.

The oracles share the author's understanding. They check that the fast route computes what the slow route computes. They cannot check that the slow route implements the right definition. That is what the reproduction of the published Caballero & Toro tables in Chapter 2. is for – an external check against numbers this project did not produce.

Monte Carlo checks have Monte Carlo error. Where a test asserts agreement with a simulated expectation rather than an algebraic identity, the tolerance is loose by necessity, and a genuinely small bias would pass.

Everything is verified on simulated data. The identities are algebraic and will hold on real genotypes. The *empirical* results – which metrics are redundant, how much index a constraint costs – were derived at $F_{ST} = 0.15$ with simulated markers, and Chapter 8. says explicitly that they should be re-derived on real genotypes before being treated as settled.

D. The bibliographic corpora

Chapter 12. draws on two screened corpora. This appendix describes them so the claims there can be checked, extended, or disagreed with.

D.1 What they are

Corpus	Records	Years	Focus
`bibliography_animal.csv`	324	2018-2026	optimal contribution, coancestry matrix choice, effective size, allelic richness
`bibliography_plant.csv`	687	2017-2026	diversity quantification, heterotic structure, genetic resources, optimisation

Both are annotated screens, not exhaustive searches. The plant corpus came from 2004 records across 29 plant-breeding queries on Crossref and Europe PMC, of which 687 were scored relevant.

D.2 Structure

```
names(pl)
#>  [1] "doi"              "year"             "authors"          "title"
#>  [5] "journal"          "citations"        "relevance"        "crop"
#>  [9] "metric_used"      "selection_method" "contribution"     "url"
```

Column	Meaning
`relevance`	3 = directly relevant, 2 = related background
`crop`	plant corpus only
`metric_used` / `metric_advocated`	the diversity metrics the record reports
`selection_method`	plant corpus only
`theme`	animal corpus only; multi-valued, semicolon-separated
`contribution` / `relevance_note`	a one-line note on what the record adds

D.3 Relevance and coverage

```
table(plant = pl$relevance)
#> plant
#>   2   3
#> 529 158
table(animal = ani$relevance)
#> animal
#>   2   3
#> 235  89
```

crop	records
maize	54

crop	records
wheat	11
multi-crop	10
rice	9
none	7
sorghum	5
barley	4
none (general crop)	4
potato	3
soybean	3

Maize dominates, which is the intended bias – but it also means every claim in Chapter 12. about crops other than maize rests on a handful of records.

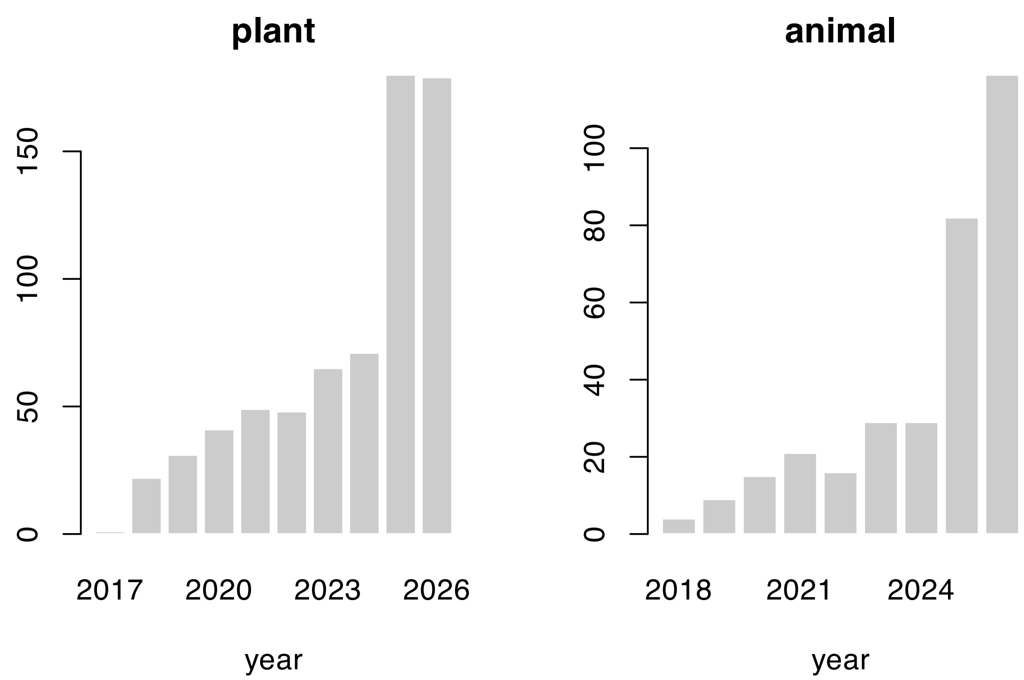

Figure D.1: Publication year of the screened records. Both corpora are deliberately post-2017.

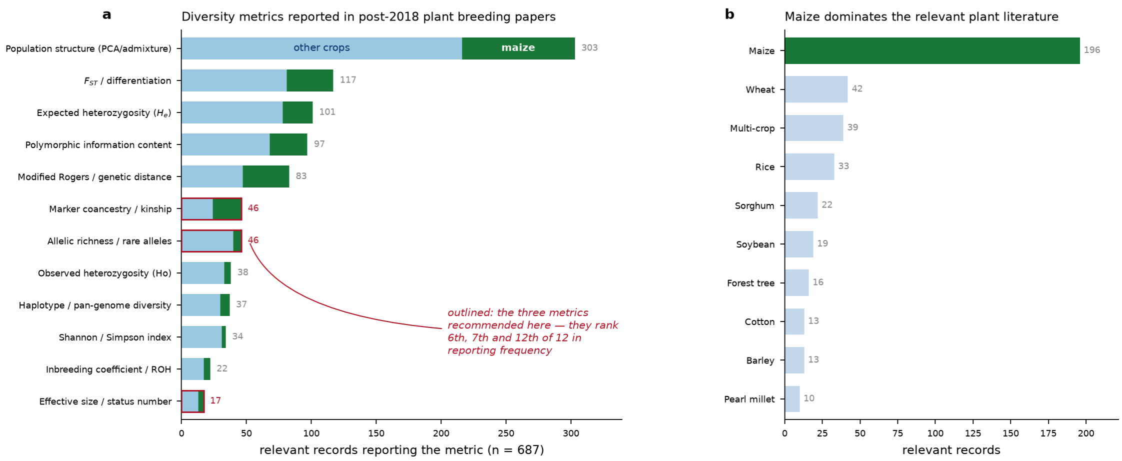

Figure D.2: Which diversity metrics the plant literature actually reports.

D.4 Themes in the animal corpus

The `theme` column is multi-valued. Splitting it:

```
th <- unlist(strsplit(ani$theme[ani$relevance == 3], ";\\s*"))
th <- trimws(th); th <- th[nzchar(th)]
sort(table(th), decreasing = TRUE)
#> th
#>        OCS / contribution optimisation Coancestry / relationship matrix choice
#>                                     68                                      61
#>          Optimisation algorithms / cost         Allelic richness / rare alleles
#>                                     27                                      20
#>                         Effective size         Heterozygosity / gene diversity
#>                                     18                                      15
#>      Core collection / subset selection
#>                                      9
```

D.5 Querying them

The corpora are ordinary CSVs, so the claims in Chapter 12. are checkable in one line each.

```
# records that report a coancestry or kinship metric
sum(grepl("coancestry|kinship|relatedness", pl$metric_used, ignore.case = TRUE))
#> [1] 12

# records using ROH, and how many of those are maize
roh <- grepl("ROH", pl$metric_used, ignore.case = TRUE)
c(roh_total = sum(roh), roh_maize = sum(roh & grepl("maize", pl$crop, ignore.case
= TRUE)))
#> roh_total roh_maize
#>         1         0

# the most-cited directly relevant maize records
mz <- pl[pl$relevance == 3 & grepl("maize", pl$crop, ignore.case = TRUE), ]
head(mz[order(-mz$citations), c("year", "title", "citations")], 6)
```

year	title	citations
[illegible]	Diversity and heterotic patterns in North American proprietary dent maize germplasm	58
[illegible]	Discovery of beneficial haplotypes for complex traits in maize landraces.	46

year	title	citations
2021	Genetic diversity and selection signatures in maize landraces compared across 50 years of in situ and ex situ conservation	44
2021	Application of Genomic Selection at the Early Stage of Breeding Pipeline in Tropical Maize.	38
2020	Efficiencies of Heterotic Grouping Methods for Classifying Early Maturing Maize Inbred Lines	26
2021	Genetic variation and population structure in China summer maize germplasm.	25

```
# the animal-side records on matrix choice
mc <- ani[grepl("Coancestry / relationship matrix choice", ani$theme), ]
head(mc[order(-mc$citations), c("year", "title", "citations")], 6)
```

year	title	citations
2018	Optimal cross selection for long-term genetic gain in two-part programs with rapid recurrent genomic selection	156
2019	The impact of genomic selection on genetic diversity and genetic gain in three French dairy cattle breeds	100
2020	Management of Genetic Diversity in the Era of Genomics.	72
2019	Genomic Tools for Effective Conservation of Livestock Breed Diversity	71
2020	Molecular genetic analysis of spring wheat core collection using genetic diversity, population structure, and linkage disequilibrium	65
2019	Improving Short- and Long-Term Genetic Gain by Accounting for Within-Family Variance in Optimal Cross-Selection.	59

D.6 Limitations

These are abstract screens. Relevance and the metric annotations were scored from titles and abstracts, not from full texts. A paper that computes a metric without naming it in the abstract is invisible to this table – which almost certainly understates how often coancestry is computed as an intermediate step.

The queries shaped the answer. 29 plant-breeding queries were run; a different query set would return a different landscape. The strong maize skew is by design, and the counts for other crops are too small to support a claim about those crops.

"Not established in the retrieved set" means exactly that. Every such statement in Chapter 12. is a claim about *this corpus*, not about the literature as a whole. It is a prompt to look further before assuming novelty, not a proof of it.

Citation counts are a snapshot at screening time and are already stale.

D.7 Classical references

The corpora are deliberately post-2017. The classical results the book rests on are cited from the source literature, and collected in `book/references.bib`:

Result	Source
group coancestry	Cockerham (1967)
molecular / IBS coancestry	Nejati-Javaremi et al. (1997)
gene diversity, Gst	Nei (1973)
the subdivided-population partition	Caballero & Toro (2000, 2002)

Result	Source
status number	Lindgren & Mullin (1997, 1998)
optimal contribution selection	Meuwissen (1997)
genomic relationship matrix	VanRaden (2008)
modified Rogers distance	Wright (1978)
usefulness criterion	Schnell & Utz (1975)
differential evolution	Storn & Price (1997)

References

Bibliography

[1] A. Caballero and M. A. Toro, “Analysis of genetic diversity for the management of conserved subdivided populations,” *Conservation Genetics*, vol. 3, no. 3, pp. 289–299, 2002, doi: 10.1023/A:1019956205473.

[2] A. Caballero and M. A. Toro, “Interrelations between effective population size and other pedigree tools for the management of conserved populations,” *Genetics Research*, vol. 75, no. 3, pp. 331–343, 2000, doi: 10.1017/S0016672399004449.

[3] A. Nejati-Javaremi, C. Smith, and J. P. Gibson, “Effect of total allelic relationship on accuracy of evaluation and response to selection,” *Journal of Animal Science*, vol. 75, no. 7, pp. 1738–1745, 1997, doi: 10.2527/1997.7571738x.

[4] F. Gómez-Romano, B. Villanueva, J. Fernández, J. A. Woolliams, and R. Pong-Wong, “The use of genomic coancestry matrices in the optimisation of contributions to maintain genetic diversity at specific regions of the genome,” *Genetics Selection Evolution*, vol. 48, p. 2, 2016, doi: 10.1186/s12711-015-0172-y.

[5] M. A. Toro, B. Villanueva, and J. Fernández, “Genomics applied to management strategies for conservation of farm animal genetic resources,” *Animal Genetics*, vol. 45, no. 3, pp. 1–10, 2014, doi: 10.1111/age.12140.

[6] T. H. E. Meuwissen, A. K. Sonesson, G. Gebregiwergis, and J. A. Woolliams, “Management of genetic diversity in the era of genomics,” *Frontiers in Genetics*, vol. 11, p. 880, 2020, doi: 10.3389/fgene.2020.00880.

[7] C. C. Cockerham, “Group inbreeding and coancestry,” vol. 56, no. 1, 1967, pp. 89–104.

[8] M. Nei, “Analysis of gene diversity in subdivided populations,” *Proceedings of the National Academy of Sciences*, vol. 70, no. 12, pp. 3321–3323, 1973, doi: 10.1073/pnas.70.12.3321.

[9] D. Lindgren, L. D. Gea, and P. A. Jefferson, “Status number for measuring genetic diversity,” *Forest Genetics*, vol. 4, no. 2, pp. 69–76, 1997.

[10] M. A. Toro, B. Villanueva, and J. Fernández, “The concept of effective population size loses its meaning in the context of optimal management of diversity using molecular markers,” *Journal of Animal Breeding and Genetics*, vol. 137, no. 4, pp. 345–355, 2020, doi: 10.1111/jbg.12457.

[11] E. López-Cortegano, R. Pouso, A. Labrador, A. Pérez-Figueroa, J. Fernández, and A. Caballero, “Optimal management of genetic diversity in subdivided populations,” *Frontiers in Genetics*, vol. 10, p. 843, 2019, doi: 10.3389/fgene.2019.00843.

[12] H. De Beukelaer, G. F. Davenport, and V. Fack, “Core Hunter 3: flexible core subset selection,” *BMC Bioinformatics*, vol. 19, no. 1, p. 203, 2018, doi: 10.1186/s12859-018-2209-z.

[13] A. Caballero and S. T. Rodríguez-Ramilo, “A new method for the partition of allelic diversity within and between subpopulations,” *Conservation Genetics*, vol. 11, no. 6, pp. 2219–2229, 2010, doi: 10.1007/s10592-010-0107-7.